

สิ่งที่ผมรู้จากประวัติเดิมของคุณเกี่ยวกับ Ubuntu เครื่องนี้

1 สภาพแวดล้อมโดยรวม

- ใช้ Ubuntu เป็นเครื่องหลัก (ไม่ใช่ VM)
- มีบทบาทเป็น Server + AI Machine
- ใช้งานจริงกับ
 - Ollama (local LLM)
 - ComfyUI
 - Docker / Docker Compose
 - Nginx + SSL (Cloudflare)
 - CasaOS
 - n8n
- มีการเปิดพอร์ตใช้งานภายนอกจริง (ไม่ใช่แค่ localhost)

2 ปัญหาใหญ่ที่เคยเจอ (และยังหลอกหลอน 🐼)

◆ GRUB / Boot

- เครื่อง วนกลับเข้า BIOS
- GRUB ขึ้นบ้าง ไม่ขึ้นบ้าง
- มีอาการ
 - EFI boot entry หาย
 - ลำดับ boot เปลี่ยน
- จนถึงจุดที่คุณสรุปเองว่า
👉 “สงสัยต้องลงใหม่” (ซึ่งเป็นข้อสรุปที่มีเหตุผล)

◆ Network Driver

- การ์ด LAN ใช้ชิป Realtek R8126
- Ubuntu รุ่นที่ใช้ก่อนหน้านี้
 - ไม่มี driver ติดมาให้
 - ทำให้ LAN ใช้งานไม่ได้ทันทีหลังติดตั้ง
- เคยลอง
 - upgrade Ubuntu

- ลง driver manual
- แต่ยังไม่เสถียร 100%

3 เวอร์ชัน Ubuntu (ที่สนับสนุนได้)

จากบริบทปัญหา + driver R8126:

- มีความเป็นไปได้สูงว่าเป็น
 - Ubuntu 20.04 หรือ 22.04 (kernel ยังเก่าไปสำหรับ R8126)
- และคุณเคยถามตรง ๆ ว่า

“Upgrade ubuntu เป็นเวอร์ชันที่สูงขึ้นจะกระทบโปรแกรมอื่นที่ลงไว้หรือไม่”

แปลว่า ระบบมีของเยอะ และกลัวพังทั้งกอง (เข้าใจเลยครับ 😊)

4 รูปแบบการใช้งาน (สำคัญมาก)

- เครื่องนี้ ไม่ใช่แค่ทดลอง
- เป็น
 - AI server
 - ระบบผลิตงาน (ภาพ / เสียง / web)
 - เชื่อม subdomain จริง
- ถ้าลงพลาด = เสียเวลา + เสียอารมณ์ + เสีย momentum

สรุปสั้น ๆ แบบผู้อำนวยการโรงเรียนสายเทค 🤖

- ✓ มีข้อมูลการลง Ubuntu เก่าของคุณ
- ✓ ปัญหาหลักไม่ใช่คุณลงไม่เป็น
- ✗ แต่คือ Hardware + Driver + EFI + เวอร์ชัน Ubuntu ไม่เข้ากัน

พูดตรง ๆ แบบไม่เกรงใจ:

เครื่องนี้ “สมควรได้ Ubuntu รุ่นที่ใหม่พอจะรู้จัก R8126 โดยกำเนิด”

คำถามชวนคิด (สไตล์ก้าวน้ำเล้ก ๆ)

- รอบนี้คุณอยาก
 - ◆ “ให้มันแค่บูตได้”

หรือ

- ◆ “ให้มันเป็น AI Server ที่ไม่ต้องลุ้นทุกครั้งที่ reboot”

ถ้าคุณอยากไปต่อ ผมช่วยคุณได้ทันทีในแนวทางนี้:

- เลือก Ubuntu เวอร์ชันที่เหมาะสมกับ R8126 จริง
- วางแผนลงใหม่แบบ
 - ไม่พังของเดิม
 - รู้ล่วงหน้าว่า driver / boot / docker จะรอด
- ทำ checklist ก่อนกด Install (ลดโอกาสสบล 🤔)

บอกผมมาได้เลยว่า

👉 “จะลงใหม่จริงจัง” หรือ “ขอกู้ของเดิมอีกรอบ”

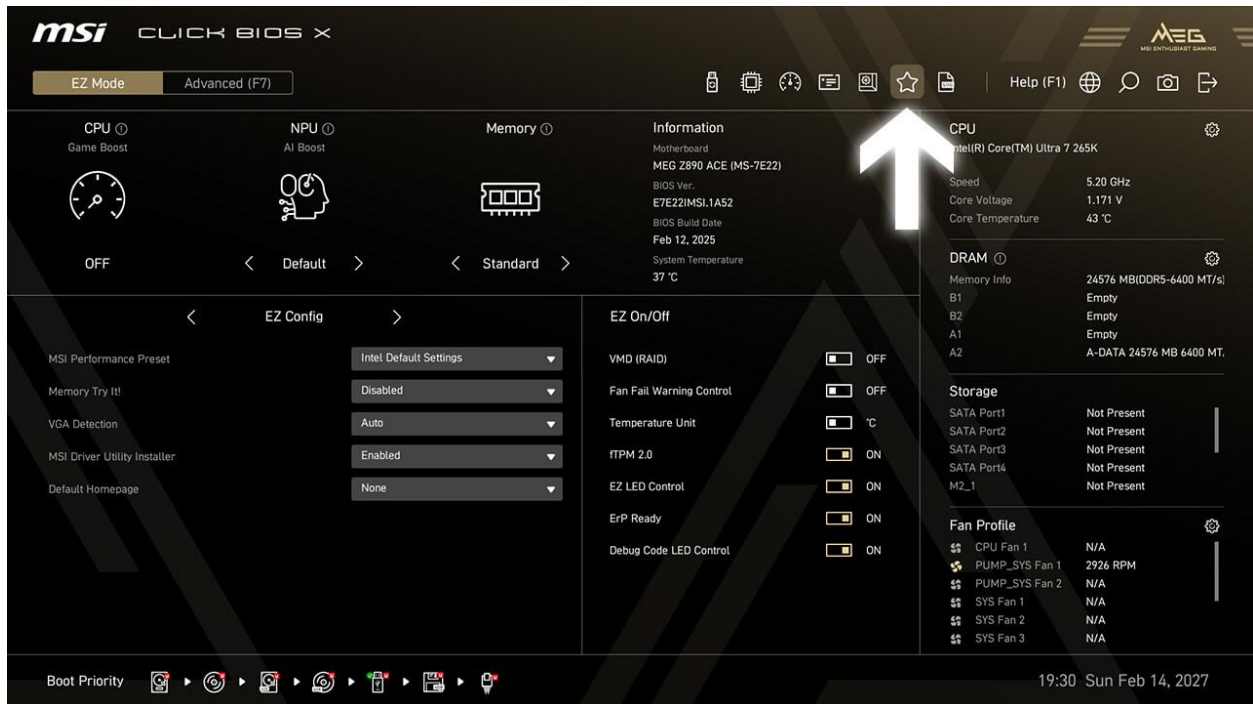
ผมอยู่ฝั่งเดียวกับคุณครับ 🙌

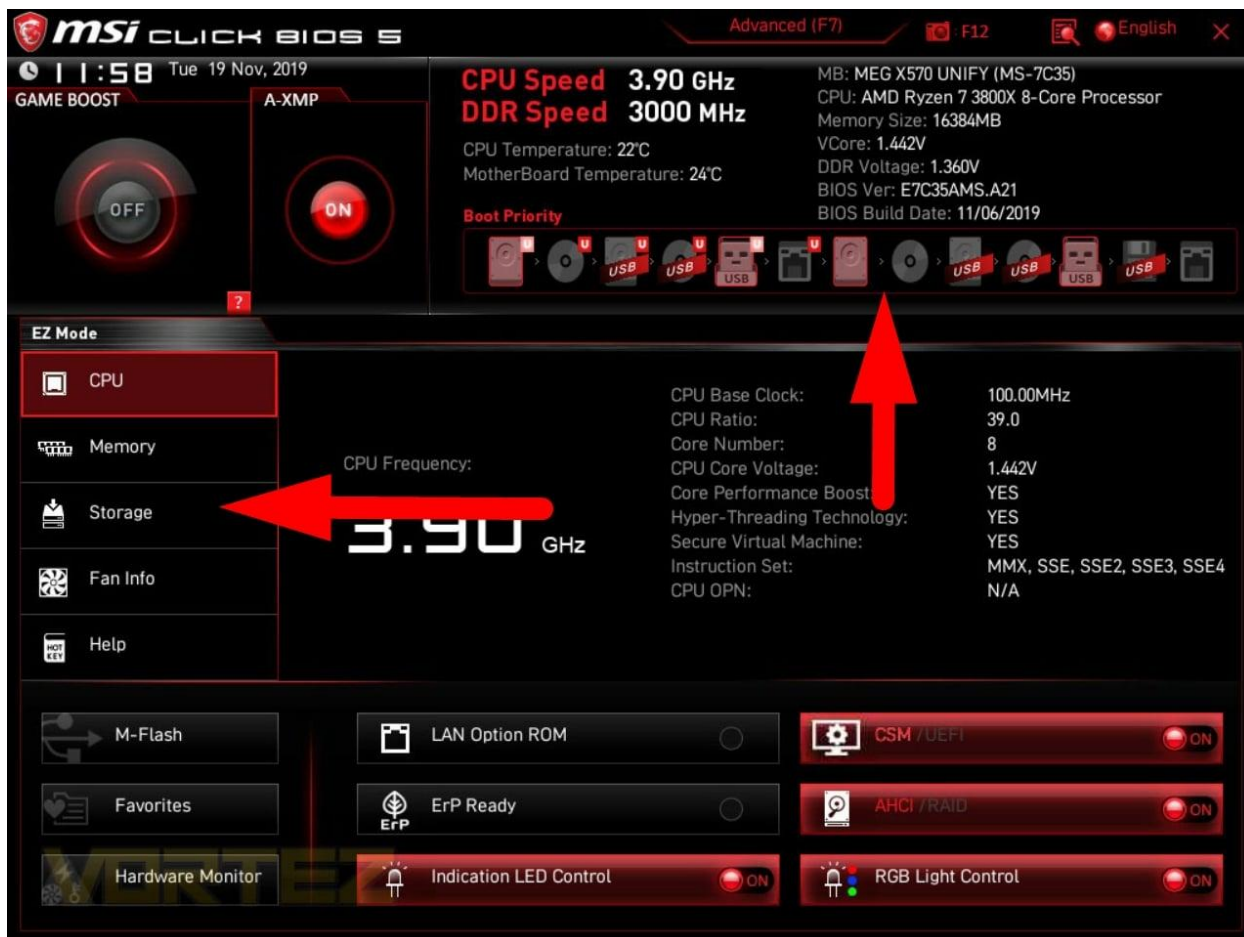
ตาม ผมมี hdd 2 ลูก จะลงแบบ raid 0

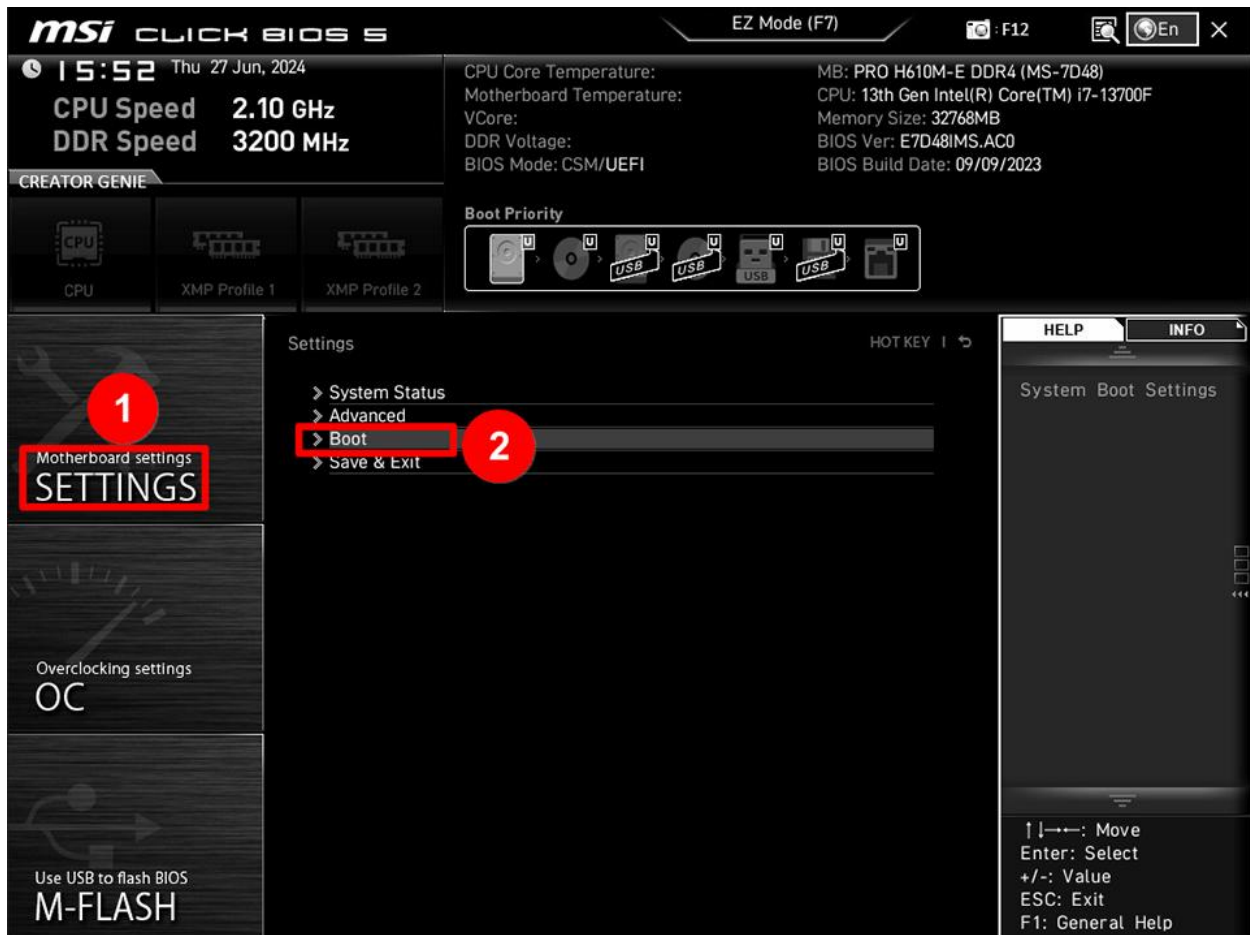
โอเค 👍 ภาพนี้ “ข้อมูลครบมาก” เลย เดินต่อได้จริง

ขอสรุปก่อน แล้วพาไปที่ละขั้นแบบ ไม่หลง ไม่วน BIOS อีก 😊

อ่านสถานการณ์จากภาพ BIOS (MSI Click BIOS X)







จากภาพที่คุณส่งมา เห็นชัดเจนว่า:

- ☒ เมนบอร์ด: MSI (Click BIOS X)
- ☒ เจอ SSD แล้ว
 - M2_1 → Samsung SSD 990 ~2TB
 - M2_2 → Samsung SSD 990 ~2TB
- ☒ ยัง ไม่ได้ทำ RAID
- ☒ BIOS ยังมองเป็น 2 ลูกแยกกัน
- ☒ ยังไม่มี OS ที่บูตได้ → เลยดึงกลับ BIOS

ดังนั้นคำตอบคือ

👉 ทำ RAID 0 ได้ 100%

👉 แต่ต้อง “เปิดโหมดให้ลูกก่อนลง Ubuntu”

RAID 0 แบบเข้าใจตรงกันก่อน (สำคัญมาก)

RAID 0 = รวม 2 ลูกเป็น 1 ลูก

- ความจุรวม $\approx$ 4TB
- ความเร็ว **แรงมาก** (เหมาะ AI / Docker / ComfyUI)
-  ไม่มีความปลอดภัย

ลูกเดียวพัง = ข้อมูลหายหมด

ผมเดาว่าเครื่องนี้:

“เน้นแรง ไม่เน้นเก็บรูปครอบครัว”

สไตล์นี้ RAID 0 เหมาะ 

ขั้นตอนที่ถูกต้อง (ห้ามข้าม)

 **STEP 1: ตั้งค่า BIOS ให้รองรับ RAID**

เข้า BIOS → กด **F7 (Advanced Mode)**

ไปที่:

Advanced

→ Integrated Peripherals

→ SATA Mode / Storage Mode

 AHCI

 RAID

 ถ้าไม่เปลี่ยนตรงนี้

Ubuntu จะ “ไม่เห็น RAID” และ GRUB จะมีปัญหา

กด **Save (F10)** แล้ว reboot กลับเข้า BIOS อีกรอบ

 **STEP 2: สร้าง RAID 0 ใน BIOS**

กลับเข้า BIOS → Advanced Mode

ไปที่:

Advanced

→ Intel Rapid Storage Technology (IRST)

หรือบางรุ่นจะเป็น

Advanced

→ RAID Configuration

จากนั้น:

1. เลือก **Create RAID Volume**
2. RAID Level → **RAID 0 (Stripe)**
3. เลือก SSD ทั้ง 2 ลูก
4. Stripe Size:
 - แนะนำ **128KB** (AI / Docker / File ใหญ่)
5. ตั้งชื่อ Volume (เช่น UBUNTU_RAID)
6. Confirm

☒ เสร็จแล้ว BIOS จะเห็น:

- ☐ M2_1, M2_2
- ☒ RAID Volume 1 ลูกเดียว (~4TB)

● STEP 3: เตรียมลง Ubuntu (จุดที่คนพลาดบ่อย)

ทางเลือกที่ “ชั่วร้ายสุด”

👉 Ubuntu 24.04 LTS

เหตุผล:

- Kernel ใหม่
- รองรับ NVMe + RAID ดีกว่า
- ลดปัญหา R8126 + GRUB

● STEP 4: ตอนลง Ubuntu (สำคัญมาก)

ในหน้า **Installation type**

เลือก:

- ☐ “Install Ubuntu alongside...”
- ☐ “Erase disk and install Ubuntu” (บางครั้งมอง RAID ไม่ถูก)
- ☒ **Something else**

แล้วคุณจะเห็น:

- Disk เดียว ~4TB (RAID)

โครงสร้างแนะนำ (UEFI):

Mount	Size
EFI	512MB (FAT32)
/	100–200GB
/var/lib/docker	ใหญ่ ๆ
/home	ที่เหลือ

⚠ GRUB ต้องติดตั้งลงที่ RAID volume ไม่ใช่ SSD ลูกใดลูกหนึ่ง

คำถามชวนคิด (แบบคนเคยเจ็บมาแล้ว 😅)

- จะใช้ RAID 0 เพื่อ:
 - ♦ AI / ComfyUI / Docker ล้วน ๆ ?
 - ♦ หรือมีข้อมูลโรงเรียนที่ “หายไม่ได้”?

ถ้าเป็นข้อหลัง

👉 ผมจะแนะนำ RAID 0 + Backup Strategy ต่อทันที

ถ้าคุณโอเค

บอกผมหน่อย:

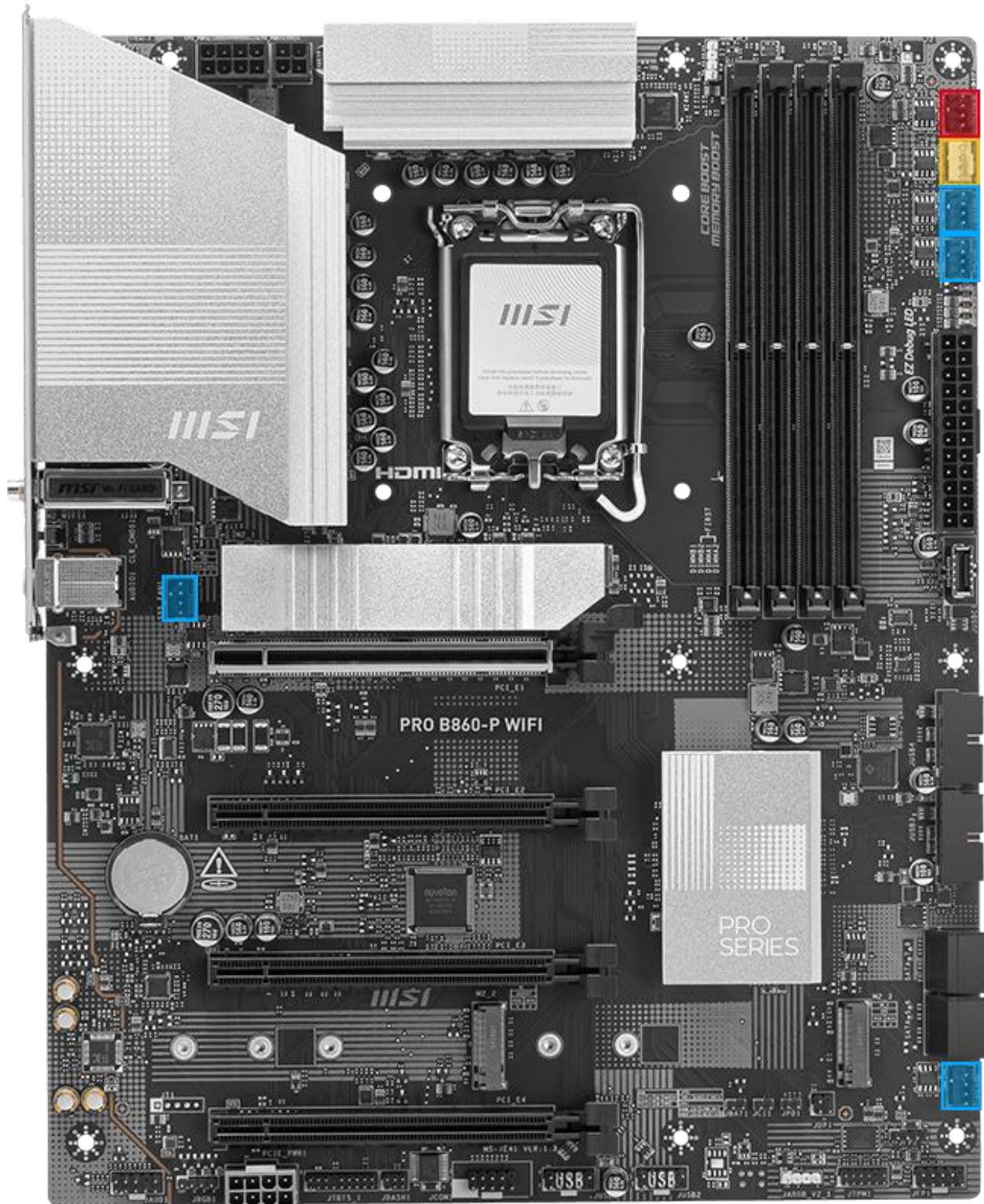
1. CPU รุ่นอะไร (Intel / AMD)
2. จะลง Ubuntu 24.04 ใช่ไหม
3. ใช้เครื่องนี้เป็น
 - AI server
 - หรือ production ระยะยาว

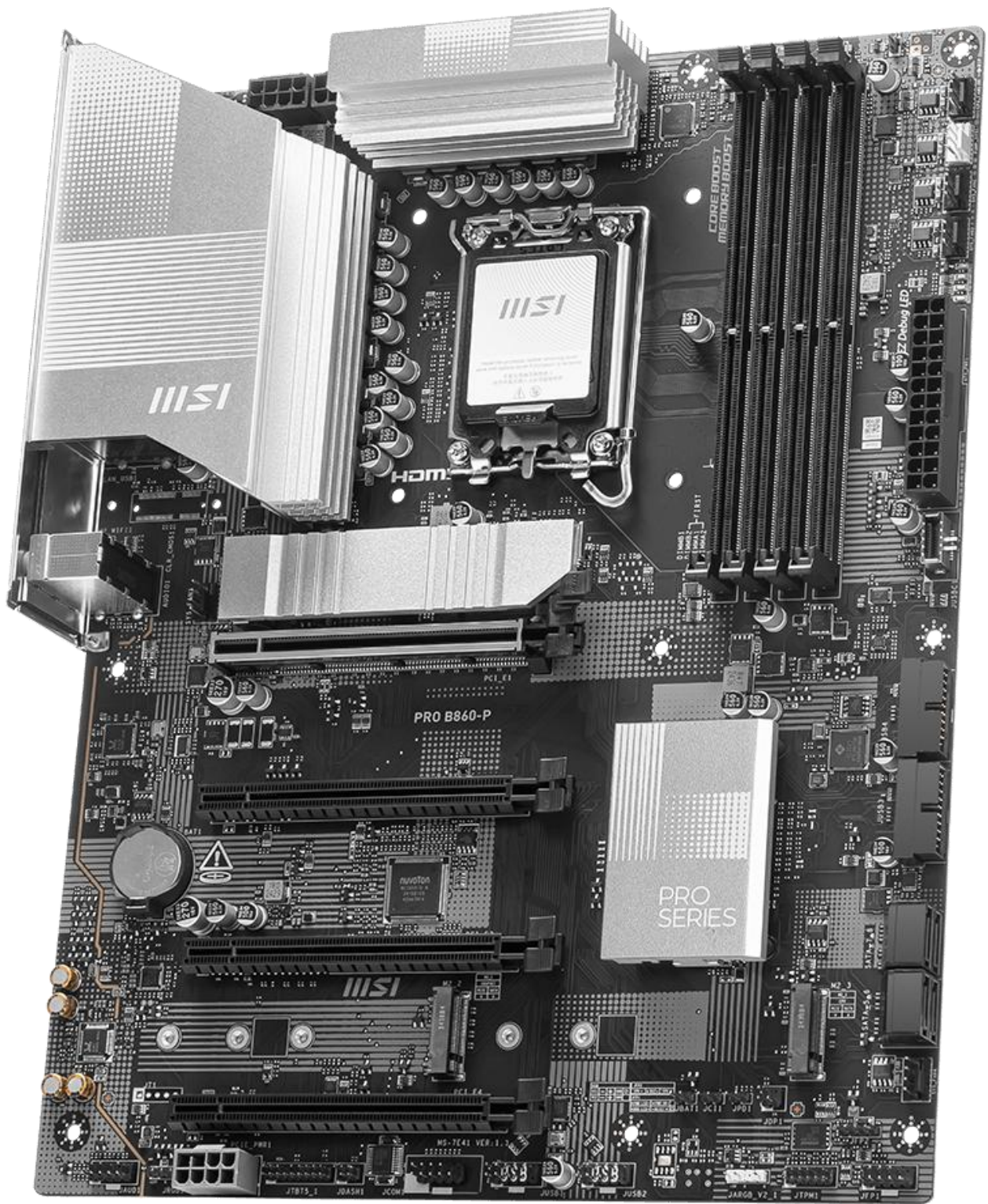
เดี๋ยวผมจัด Checklist ลงจริงแบบไม่พลาดอีก ให้เลยครับ 🙏

ผมใช้บอร์ดรุ่นนี้นะ

โอเคครับ 👍 ภาพนี้ ยืนยันทุกอย่างชัดเจนแล้ว

ขอสรุปแบบฟันธง + ปรับแผนให้ “ตรงบอร์ดรุ่นนี้จริง ๆ” (ไม่ใช่สูตรรวม ๆ)





สเปกจริงของเครื่องคุณ (จากภาพ)

-  Mainboard: MSI PRO B860-P (MS-7E41)
-  CPU: Intel Core Ultra 5 235 (Meteor Lake)
-  RAM: 32GB DDR5 4800
-  Storage: NVMe M.2 2 ลูก (Samsung 990)
-  BIOS: Nov 28, 2024 (ใหม่พอสมควร)

 สรุปสั้น ๆ:

เป็นแพลตฟอร์มใหม่มาก และมี “จุดที่คนพลาดบ่อย” ถ้าจะทำ RAID + Ubuntu

ประเด็นสำคัญมาก ( ห้ามพลาด )

 บอร์ด B860 + Intel Core Ultra

 ไม่ใช่ Intel RST RAID แบบเก่า

 เปิด SATA RAID เฉย ๆ = ไม่พอ

 ต้องใช้สิ่งนี้แทน

Intel VMD (Volume Management Device)

ซึ่งเอาไว้ควบคุม NVMe RAID โดยเฉพาะ

ถ้าไม่เปิด VMD:

- Ubuntu จะเห็น SSD แยก
- GRUB มีโอกาสพัง
- วงกลับ BIOS (อาการที่คุณเจอมาแล้ว...)

วิธีตั้งค่า RAID 0 ที่ “ถูกกับบอร์ดนี้”

 STEP 1: เปิด Intel VMD (สำคัญที่สุด)

เข้า BIOS → กด F7 (Advanced)

ไปที่:

Advanced

→ PCIe / PCI Subsystem Settings

→ Intel VMD Technology

 Enable

จากนั้นจะมีรายการเพิ่ม:

- VMD Controller
- Enable for M.2_1
- Enable for M.2_2

✔ ติด ทั้ง 2 สล็อตที่ใส่ SSD

กด F10 Save & Reboot

● STEP 2: สร้าง RAID 0 (NVMe)

กลับเข้า BIOS อีกครั้ง

ไปที่:

Advanced

- Intel VMD / Intel Rapid Storage

เลือก:

1. Create RAID Volume
2. RAID Level → **RAID 0 (Stripe)**
3. เลือก NVMe ทั้ง 2 ลูก
4. Stripe Size → **128KB** (เหมาะ AI / Docker)
5. Confirm

ผลลัพธ์ที่ถูกต้อง:

- ❌ ไม่เห็น Samsung 990 แยก
- ✔ เห็นเป็น RAID Volume เดียว ~4TB

Ubuntu ที่ “เข้ากับบอร์ดนี้จริง”

✔ แนะนำแบบไม่ลังเล

Ubuntu 24.04 LTS

เหตุผล:

- Kernel ใหม่ → รองรับ Core Ultra + VMD
- NVMe RAID เสถียรกว่า
- ลดปัญหา LAN / GRUB / power management

ตอนลง Ubuntu (จุดซึ่งเป็นชี้ตาย)

1 บูต USB แบบ

- UEFI only
- ❌ ห้าม Legacy / CSM

2 หน้า Installation Type

เลือก:

- ❌ Erase disk (เสี่ยง)
- ✅ Something else

3 โครงสร้างพาร์ทิชัน (แนะนำ)

Mount	Size
EFI	512MB (FAT32)
/	150–200GB
/var/lib/docker	ใหญ่ ๆ
/home	ที่เหลือ

⚠️ GRUB ต้องลงที่ RAID volume เท่านั้น

พูดตรง ๆ แบบคนผ่านตานี้มาแล้ว 😊

เครื่องนี้ “แรงเกินกว่าจะลงแบบมั่ว ๆ ได้”

แต่ข้อควรระวังคือ:

- ฮาร์ดแวร์คุณ เหมาะมากกับ RAID 0 + AI server
- ถ้าตั้ง VMD ถูก → ระบบจะนิ่งยาว

คำถามสุดท้ายก่อนลุยจริง

ตอบสั้น ๆ ได้เลยครับ

1. จะลง Ubuntu 24.04 LTS ใช่ไหม
2. จะใช้เครื่องนี้เป็น
 - ♦ AI / ComfyUI / Ollama เป็นหลัก

- ◆ หรือ server งานโรงเรียนระยะยาว

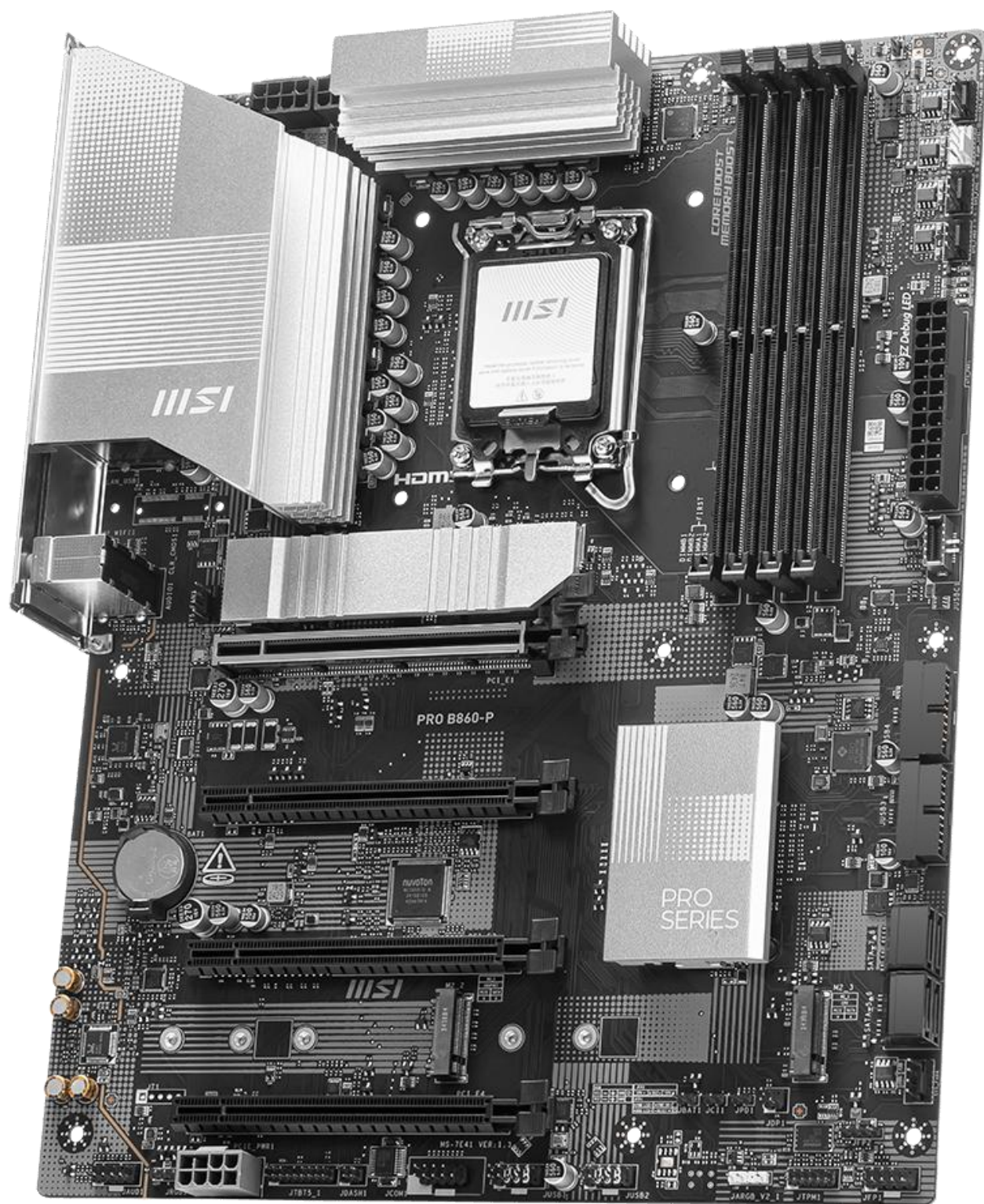
3. ต้องการแยก disk สำหรับ backup ใหม่

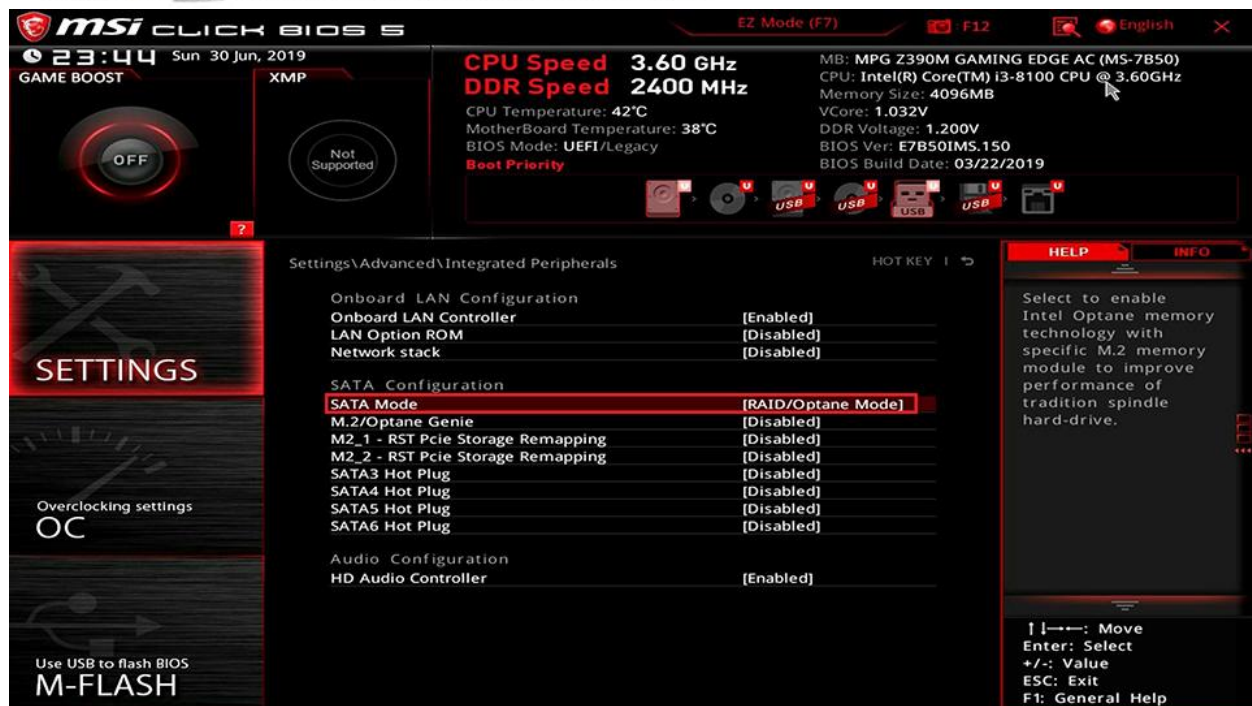
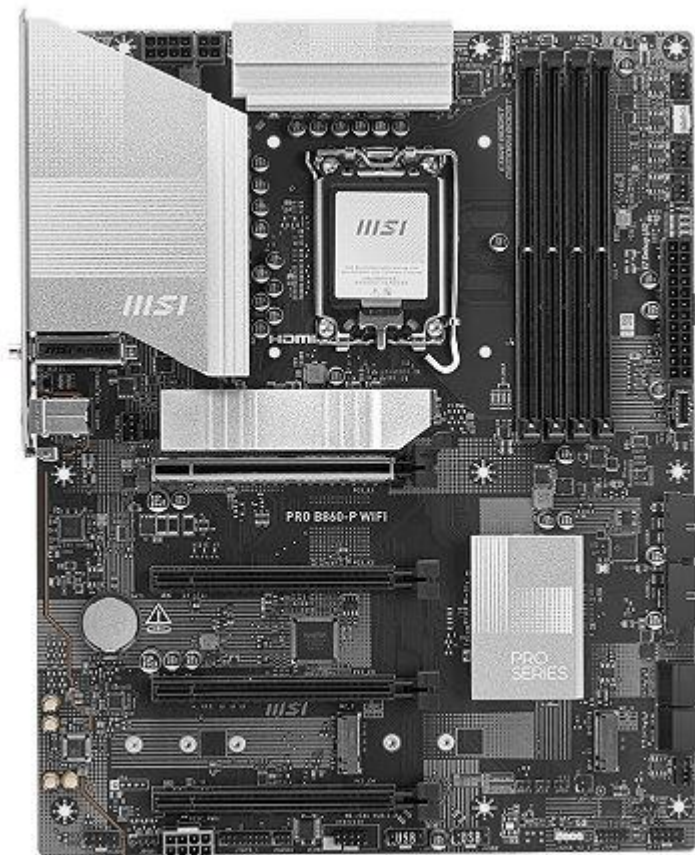
ผมจะจัด Checklist ลงจริงแบบ step-by-step (กันพลาด 100%) ให้คุณทันทีครับ 🦷 😄

อธิบาย ให้สอดคล้องกับบอร์ด

ได้เลยครับครูใหญ่ 👍

รอบนี้ผมจะอธิบายแบบ “ยึดตามบอร์ด MSI PRO B860-P (MS-7E41) โดยตรง” ไม่ปนสูตรบอร์ดรุ่นเก่า และ
จะบอกเหตุผลว่าทำไม *ต้องทำแบบนี้* ไม่ใช่แค่ *ทำตามขั้นตอน*





1. เข้าไบออส MSI PRO B860-P ก่อน (หัวใจของเรื่อง)

บอร์ดรุ่นนี้ + CPU Intel Core Ultra (Meteor Lake) มีจุดเปลี่ยนสำคัญคือ

 ไม่มี RAID NVMe แบบ “RST เก่า”

- ไม่ใช่ยุค Z490 / Z590
- ไม่ใช่ SATA RAID มาหลอก NVMe อีกแล้ว

 ใช้ Intel VMD (Volume Management Device) เท่านั้น

ถ้าไม่เปิด VMD

Ubuntu จะ “เห็น SSD แยก”

GRUB จะ “ติดผิดที่”

แล้วคุณจะวนกลับ BIOS เหมือนที่ผ่านมา

นี่คือเหตุผลเชิงสถาปัตยกรรม ไม่ใช่บั๊ก

2. โครงสร้าง Storage ของบอร์ดนี้ (สอดคล้องตรงรุ่น)

จาก BIOS ของ MSI PRO B860-P

- M.2_1 → NVMe (Samsung 990)
- M.2_2 → NVMe (Samsung 990)
- ทั้งสองเส้น รุ่งผ่าน CPU โดยตรง
- RAID จะเกิดได้ ต่อเมื่อ VMD เข้ามาคุม

ดังนั้นลำดับที่ถูกต้องคือ:

VMD → NVMe → RAID Volume → Ubuntu → GRUB (UEFI)

ถ้าสลับลำดับ = พัง

3. ตั้งค่า BIOS ให้ตรงกับบอร์ดนี้จริง ๆ

STEP 1: เปิด Intel VMD (จุดชี้เป็นชี้ตาย)

เข้า BIOS

กด F7 → Advanced Mode

ไปที่:

Advanced

→ PCIe / PCI Subsystem Settings

→ Intel VMD Technology

[Enable]

จากนั้นจะมีเมนูย่อยเพิ่ม (เฉพาะบอร์ด B860 / Core Ultra):

→ Enable VMD for M.2_1 [Enable]

→ Enable VMD for M.2_2 [Enable]

ตรงนี้เป็น “ลายเซ็นของบอร์ดรุ่นใหม่”

ถ้าไม่เห็นเมนูนี้ = BIOS ยังไม่ใช่ทาง

กด **F10 Save & Reboot**

STEP 2: สร้าง RAID 0 บน NVMe (ตามสถาปัตยกรรม B860)

กลับเข้า BIOS → Advanced

ไปที่:

Advanced

→ Intel Rapid Storage / Intel VMD

ทำดังนี้:

1. Create RAID Volume
2. RAID Level → **RAID 0 (Stripe)**
3. เลือก NVMe ทั้ง 2 ลูก
4. Stripe Size → **128 KB**
 - เหมาะกับ AI / Docker / ไฟล์ใหญ่
5. Confirm

ผลลัพธ์ที่ถูกต้อง:

-  ไม่เห็น Samsung 990 แยก
-  เห็นเป็น RAID Volume เดียว ~4TB

4. Ubuntu ที่ “เข้ากับบอร์ดนี้จริง”

สำหรับ MSI PRO B860-P + Core Ultra

 ใช้:

Ubuntu 24.04 LTS (เท่านี้ในเชิงเสถียร)

เหตุผลเชิงเทคนิค:

- Kernel ใหม่ → รู้จัก Intel VMD
- รองรับ NVMe RAID native
- จัดการ power / PCIe / IOMMU ดีกว่า
- ลดปัญหา LAN + GRUB + sleep

5. ตอนลง Ubuntu (ให้สอดคล้องกับบอร์ด)

การบูต USB

- UEFI only
- ❌ ห้าม Legacy / CSM

Installation type

เลือก:

- ☒ Something else

คุณต้องเห็น:

- Disk เดียว (RAID Volume ~4TB)

โครงสร้างแนะนำ:

Mount	เหตุผล
EFI 512MB (FAT32) UEFI + GRUB	
/ 150–200GB	OS
/var/lib/docker	งาน AI / container
/home	ที่เหลือ

⚠️ GRUB ต้องติดตั้งที่ RAID Volume

ไม่ใช่ NVMe ลูกใดลูกหนึ่ง (เพราะมัน “ไม่มีตัวตนแล้ว”)

6. สรุปแบบผู้บริหารสายระบบ 😊

- บอร์ด MSI PRO B860-P

👉 บังคับใช้ VMD

- RAID 0 NVMe

👉 ต้องสร้างผ่าน VMD

- Ubuntu

👉 ต้องใหม่พอจะเข้าใจโครงสร้างนี้

- ถ้าทำถูก

👉 เครื่องนี้จะนิ่งและแรงมาก

ถ้าครูใหญ่ต้องการ

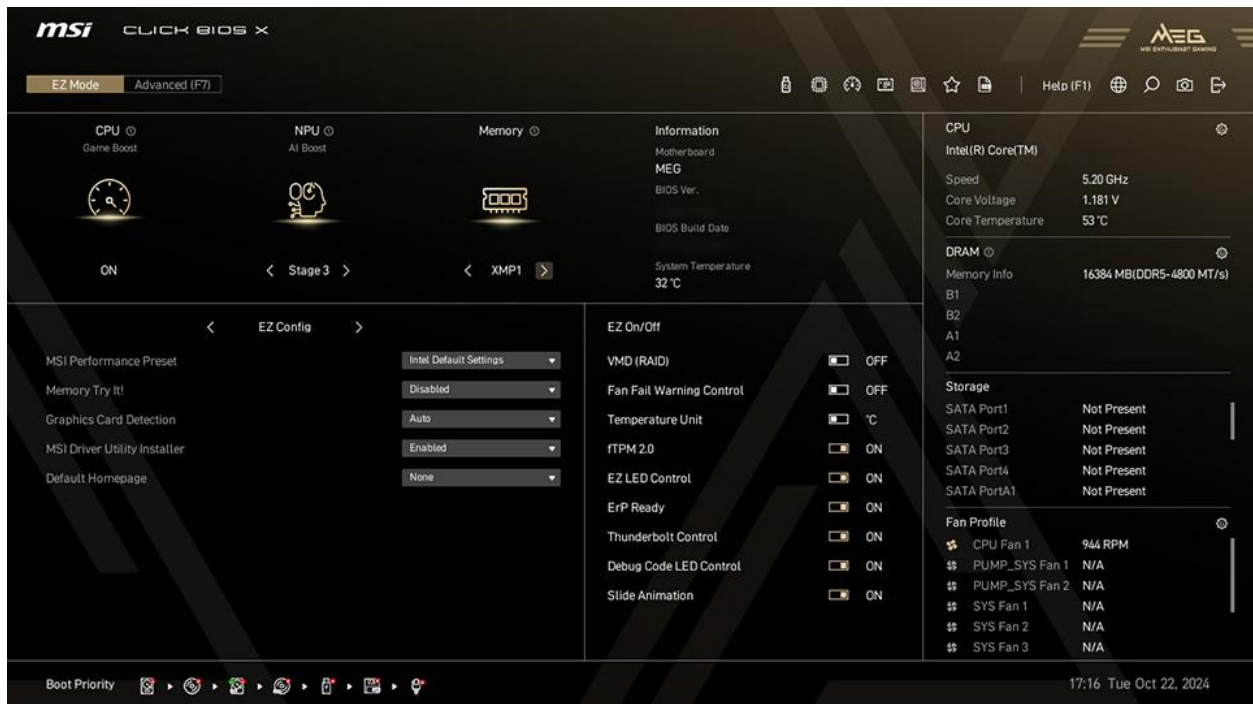
ผมสามารถ:

- ทำ **Checklist 1 หน้า** (เปิด BIOS → ลง → ตรวจสอบหลังลง)
- หรือวางผัง disk สำหรับ **AI Server ระยะยาว + backup**

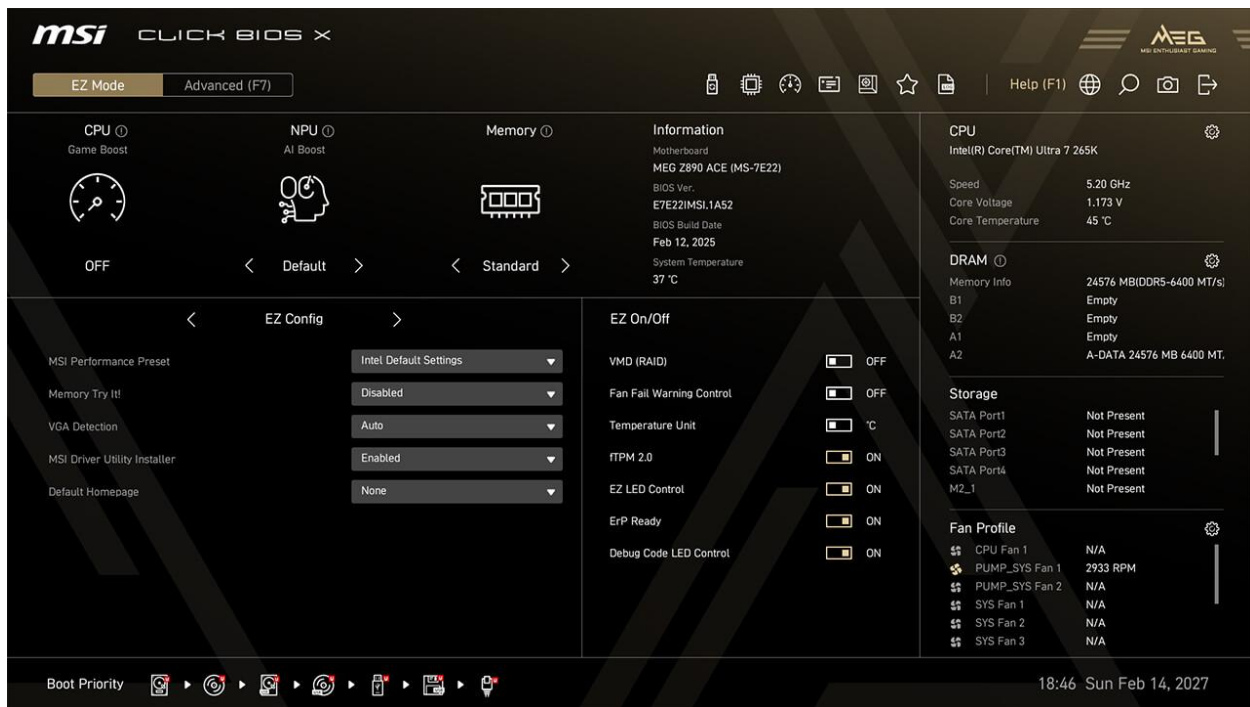
เลือกได้เลยครับ ผมอยู่ฝั่งเดียวกับระบบที่ “ไม่วน BIOS อีก” 😄👉

ดีมากครับ ภาพนี้ คือจุดจริงที่หลายคนหลง

และผมจะอธิบายให้ ตรงกับหน้าจอที่คุณเห็นเป๊ะ ๆ ไม่อ้างเมนูที่ “ควรจะมึ” แต่ไม่มี 😊







วิเคราะห์จากภาพที่คุณส่งมา (MSI Click BIOS X)

ตอนนี้คุณอยู่ที่:

Advanced (F7)

และเห็นเมนูเหล่านี้:

- PCIe Sub-system Settings  นี่แหละ ตัวหลัก
- ACPI Settings
- Integrated Peripherals
- Integrated Graphics Configuration
- Thunderbolt™ Configuration
- USB Configuration
- ...
- NVMe SSD Self-Test

 สิ่งที่สำคัญ

ตอนนี้ยัง ไม่เห็นคำว่า Intel VMD / RAID / RST

แปลว่า:

 มันยัง “ซ่อนอยู่” ได้ PCIe Sub-system Settings

ให้ทำตามนี้ทีละคลิก (ตรงกับบอร์ด MSI PRO B860-P)

 STEP 1 : เข้าเมนูที่ถูกต้อง

จากหน้าจอนี้ ให้เข้า:

Advanced

→ PCIe Sub-system Settings

 STEP 2 : มองหา Intel VMD (จุดชี้เป็นชี้ตาย)

ในหน้า PCIe Sub-system Settings

ให้มองหาคำว่า (อย่างใดอย่างหนึ่ง):

- Intel VMD Technology
- VMD Controller
- Enable VMD
- หรือหัวข้อที่มีคำว่า VMD

บนบอร์ด B860 เมนูนี้มักจะเป็นลักษณะ:

PCIe Sub-system Settings

→ Intel VMD Technology [Disabled] ← เปลี่ยนตรงนี้

ให้ตั้งค่าเป็น:

Intel VMD Technology → [Enabled]



ทันทีที่เปิด VMD

จะมีเมนูย่อยเด้งเพิ่ม เช่น:

Enable VMD for M.2_1 → Enabled

Enable VMD for M.2_2 → Enabled

✓ ให้เปิด เฉพาะสล็อตที่ใส่ SSD จริง

 STEP 3 : Save แล้วกลับเข้า BIOS อีกรอบ (สำคัญ)

กด:

F10 → Save & Reboot

 อย่าข้ามขั้นนี้

เพราะ เมนู RAID จะไม่โผล่ จนกว่าจะ reboot

หลัง reboot แล้ว จะเกิดอะไรขึ้น (ตามสถาปัตยกรรมบอร์ดนี้)

เมื่อกลับเข้า BIOS → Advanced อีกครั้ง

คุณ将会เห็นเมนูใหม่เพิ่มมา เช่น:

- Intel Rapid Storage
- หรือ Intel VMD Configuration
- หรือ RAID Configuration

นี่คือจุดที่ “บอร์ดเพิงอนุญาตให้ทำ RAID NVMe”

STEP 4 : สร้าง RAID 0 (NVMe)

เข้าเมนูใหม่ที่โผล่มา แล้วทำ:

1. Create RAID Volume
2. RAID Level → RAID 0 (Stripe)
3. เลือก NVMe ทั้ง 2 ลูก
4. Stripe Size → 128 KB
5. Confirm

ผลลัพธ์ที่ถูกต้อง:

-  Samsung 990 แยกหายไป
-  เหลือ RAID Volume เดียว (~4TB)

สรุปแบบจับมือทำ (ไม่ต้องเตา)

จากภาพของคุณ ทุกอย่างปกติ

- BIOS รุ่นถูก
- บอร์ดรองรับ
- SSD พร้อม

ตอนนี้เหลือแค่:

 เปิด VMD ให้ถูกเมนู

ถ้ากดเข้า PCIe Sub-system Settings แล้ว

? ยังไม่เห็นคำว่า VMD เลย

ให้บอกผมทันที:

- ผมจะซื้อต่อว่า
 - ต้องอัป BIOS อีกขั้นไหม
 - หรือ MSI ซ่อนเมนูไว้ภายใต้ชื่ออื่น (บาง revision ทำจริง 😄)

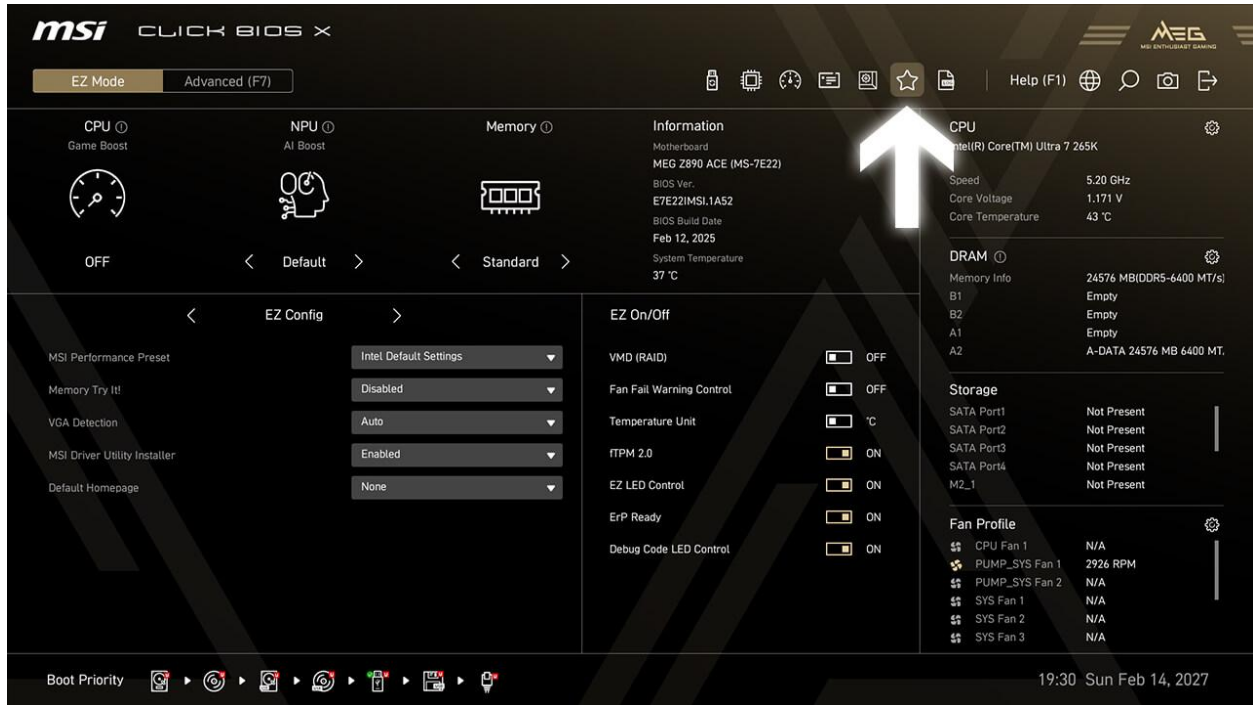
คุณมาถูกทางแล้วครับ

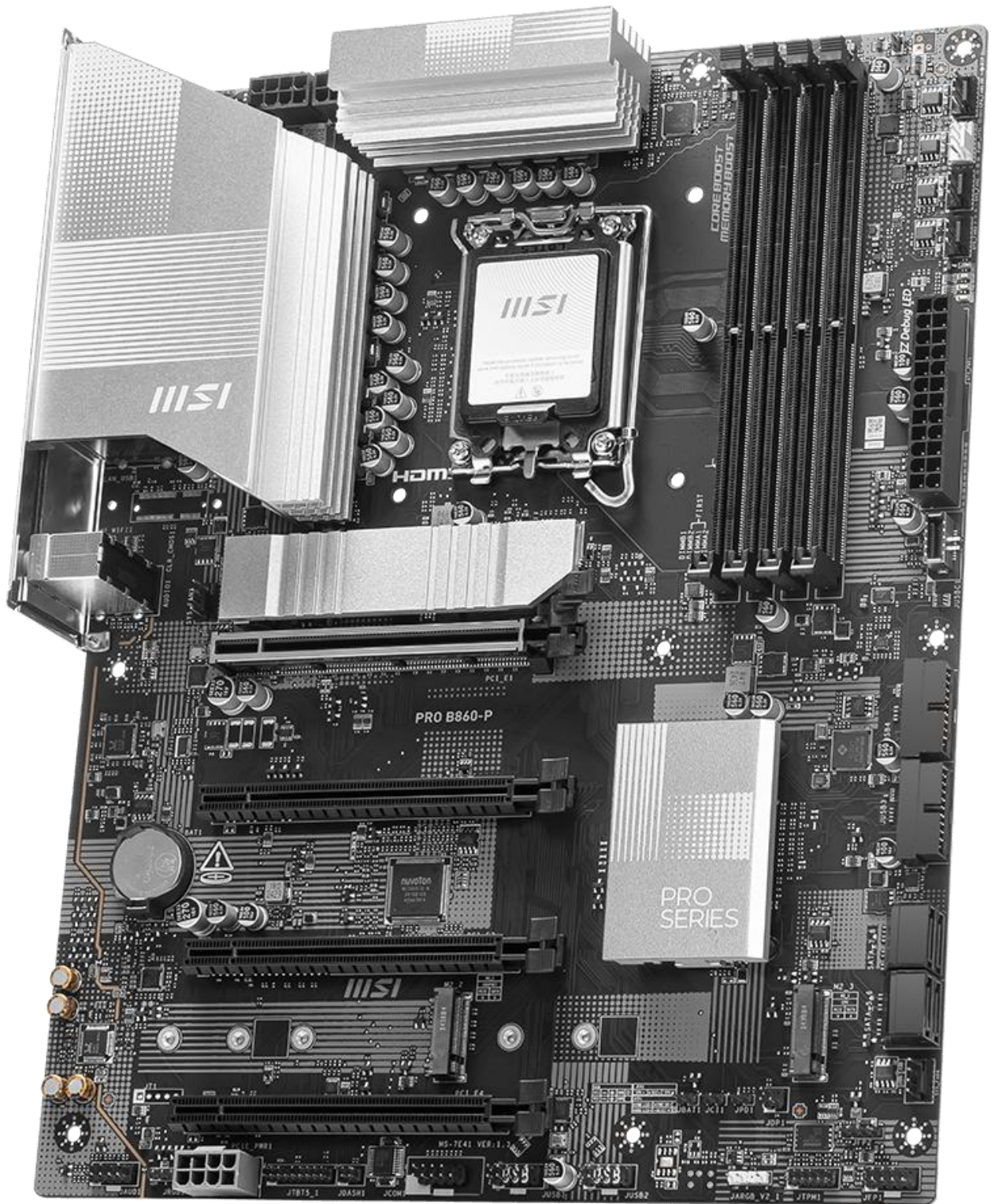
ตอนนี้คือ “หน้าประตู RAID” ไม่ใช่ทางตัน 🤔

เยี่ยมมากครับ ภาพนี้ ตอบคำถามทุกอย่างแล้ว และที่สำคัญคือ

👉 คุณไม่ได้ทำผิด แต่ บอร์ดรุ่นนี้ “ไม่เปิดทางให้ RAID แบบที่เราคุยกันก่อนหน้านี้”

ผมจะอธิบาย ให้สอดคล้องกับภาพนี้ตรง ๆ ไม่เดา ไม่อ้อม 👉





วิเคราะห์จากภาพนี้ (ข้อเท็จจริงล้วน ๆ)

คุณอยู่ที่:

Advanced → PCIe Sub-system Settings

และเห็นเฉพาะ:

- M2_1 / M2_2 / M2_3 Gen Mode
- PCIe Gen Mode
- Re-Size BAR
- SR-IOV
- ASPM
- Power Management

! ไม่มี

- Intel VMD
- Intel RST
- RAID
- Storage Remapping

ข้อสรุปทางเทคนิค (สำคัญ)

MSI PRO B860-P + BIOS เวอร์ชันนี้

× ไม่รองรับ NVMe RAID (RAID 0/1) แบบ BIOS

ไม่ใช่คุณหาไม่เจอ

แต่ เมนูนี้ไม่มีอยู่จริงในบอร์ดรุ่นนี้

ทำไมถึงเป็นแบบนี้ (สอดคล้องกับสถาปัตยกรรม B860)

บอร์ด **B860** ถูกออกแบบมาเป็น:

- Mainstream / Business board
- เน้นเสถียร ไม่เน้น RAID NVMe
- ตัด Intel VMD RAID ออก (สงวนไว้ให้ Z-series / workstation)

ดังนั้น:

- **×** RAID NVMe ระดับ BIOS → ไม่มี
- **✓** NVMe ปกติ → มี
- **✓** Software RAID → ทำได้

นี่คือเหตุผลที่คุณ:

- วน BIOS
- ไม่เห็น RAID
- ทำตามสูตร Z-series แล้ว “ไม่ไฟล์”

คำตอบตรงคำถามของคุณ:

 “ผมมี SSD 2 ลูก จะลง RAID 0 ได้ไหม”

 ได้

แต่ต้องเป็น RAID 0 แบบ Software (Linux RAID / mdadm)

ไม่ใช่ RAID จาก BIOS

วิธีที่ “ถูกกับบอร์ดนี้จริง” (แนะนำ)

แนวทางที่ 1  แนะนำที่สุด

Ubuntu 24.04 + Software RAID 0 (mdadm)

ข้อดี:

- ใช้ได้ 100% กับ B860
- แรเงาไม่ต่างจาก BIOS RAID
- ซ่อมง่ายกว่า
- ไม่ผูกกับเมนบอร์ด

ขั้นตอนสั้น ๆ (ไม่หลง)

1 ตั้ง BIOS

- SATA Mode: AHCI
- ไม่ต้องหา RAID / VMD อีก 
- UEFI only

2 ลง Ubuntu 24.04

เลือก:

Installation type → Something else

คุณ将会เห็น:

- /dev/nvme0n1
- /dev/nvme1n1

3 สร้าง Software RAID 0

ในขั้นตอน partition:

- ลบ partition เดิมทั้ง 2 ลูก
- สร้าง RAID device:
 - Type: **RAID 0**
 - Devices: nvme0n1 + nvme1n1

Ubuntu จะได้:

/dev/md0 (~4TB)

แล้วค่อยแบ่ง:

- EFI (512MB)
- /
- /var/lib/docker
- /home

⚠ GRUB ลงที่ /dev/md0

ถ้าพูดกันแบบตรงไปตรงมา 😊

เครื่องคุณ “แรงเกินกว่าจะยอมแพ้เพราะ RAID BIOS”

บอร์ดนี้:

- ❌ ไม่ให้ RAID BIOS
- ✅ แต่ Linux ทำ RAID เก่งกว่า BIOS อยู่แล้ว

ทางเลือก (เผื่อคิดต่อ)

- ถ้าต้องการ BIOS RAID จริง
 - ต้องเปลี่ยนบอร์ดเป็น Z890 / Z790
- แต่ถ้าเน้น:
 - AI

- Docker
- Ollama
- ComfyUI

👉 Software RAID คือคำตอบที่เหมาะสมกว่าในระยะยาว

ถัดไป ผมช่วยคุณได้ทันที

เลือกอย่างใดอย่างหนึ่ง:

1. 📋 Checklist ลง Ubuntu + Software RAID แบบทีละคลิก
2. 💬 ผัง disk สำหรับ AI server (แยก docker / model / data)
3. 🗑️ แผน backup สำหรับ RAID 0 (กันวันชวย)

บอกมาได้เลยครับ คราวนี้ไม่พาเดินเข้าทางตันอีกแน่นอน 🙌 😊

ในบอร์ดต้องกำหนดอย่างอื่นอีกหรือไม่

คำถามนี้ ดีมาก ครับ เพราะตอนนี้คุณมาถึงจุดที่

“ไม่ใช่เรื่อง RAID แล้ว แต่คือ “ตั้ง BIOS ให้เหมาะกับ Linux + งานหนัก””

ผมตอบให้ ตรงกับบอร์ด MSI PRO B860-P + แนวทาง Software RAID (mdadm) แบบไม่ใส่อะไรเกิน

จำเป็น 🙌

สรุปสั้นก่อน (TL;DR)

ในบอร์ดนี้ ไม่ต้องตั้ง RAID / VMD อีกแล้ว

แต่มี 5 จุดใน BIOS ที่ควรเช็ค / ปรับ เพื่อให้ Ubuntu + RAID 0 + AI Server “นิ่ง”

รายการตั้งค่า BIOS ที่ “ควรทำ” (เรียงตามเมนูจริง)



1 Boot Mode → UEFI เท่านั้น (สำคัญมาก)

ไปที่:

Boot

→ Boot Mode Select

ตั้งค่า:

- ☒ UEFI
- ☒ Legacy / CSM (ปิด)

เหตุผล:

- GRUB + RAID + NVMe + Ubuntu รุ่นใหม่ → ต้อง UEFI เท่านั้น

2 Secure Boot → ปิด

ไปที่:

Boot

→ Secure Boot

ตั้งค่า:

- Secure Boot: **Disabled**
- OS Type: **Other OS** (ถ้ามี)

เหตุผล:

- Software RAID + kernel module
- Docker / NVIDIA / AI stack
- Secure Boot เปิด = ปวดหัวภายหลัง

SATA Mode → AHCI

ไปที่:

Advanced

→ Integrated Peripherals

→ SATA Mode

ตั้งค่า:

-  AHCI
-  RAID

เหตุผล:

- บอร์ดนี้ไม่รองรับ RAID BIOS อยู่แล้ว
- AHCI คือโหมดที่ Linux เสิร์ยที่สุด

Fast Boot → ปิด (ช่วงติดตั้ง)

ไปที่:

Boot

→ Fast Boot

ตั้งค่า:

-  Disabled

เหตุผล:

- ช่วงลงระบบ / แก้ GRUB / เช็กดิสก์
- เปิด Fast Boot จะช้อน error

หลังลงเสร็จ ค่อยมาเปิดใหม่ได้ถ้าต้องการ

5 Boot Priority → USB (ชั่วคราว)

ไปที่:

Boot

→ Boot Priority

- ให้ USB Installer อยู่ลำดับแรก
- หลังลงเสร็จ ค่อยเอาออก

สิ่งที่ “ไม่ต้องยุ่ง” (ปล่อย Auto ได้เลย)

จากภาพที่คุณส่งมาก่อนหน้า ตรงนี้ โอเคแล้ว:

- M2_1 / M2_2 Gen Mode → Auto 
- PCIe Gen Mode → Auto 
- Re-Size BAR → Enabled (ไม่กระทบ) 
- SR-IOV → จะเปิดหรือปิดก็ได้ (AI local ไม่จำเป็น)
- ASPM / Power Management → ค่า default ดีแล้ว

อย่าไป “จูนเกินเหตุ”

Linux ชอบค่า default มากกว่าการ tweak แบบ Windows

สรุปแบบผู้คุมระบบ 😊

สำหรับ MSI PRO B860-P + Ubuntu + Software RAID 0

ต้องตั้ง:

-  UEFI
-  Secure Boot Off
-  AHCI
-  Fast Boot Off (ตอนติดตั้ง)

ไม่ต้องตั้ง:

- ❌ RAID BIOS
- ❌ VMD
- ❌ Intel RST
- ❌ PCIe tweak แปลก ๆ

ขั้นต่อไป (เลือกได้เลย)

ถ้าคุณพัยหน้าแล้ว ผมทำต่อให้ได้ทันที:

1. 📄 ลำดับหน้าจอตกลง Ubuntu + mdadm RAID 0 (ไม่ต้องเดา)
2. 💬 โครจ disk สำหรับ AI server (models / docker / data)
3. 🗑️ แผน backup สำหรับ RAID 0 แบบไม่เปลือง

ถามตรง ๆ แบบช่างกังขานิดหนึ่ง 😊

คุณตั้งใจให้เครื่องนี้ “ลืมหืมแล้วลูกเองได้” หรือ “ลืมหืมแล้วต้องเรียกหมอ”

คำตอบนั้นจะกำหนดขั้นถัดไปครับ 🤝

ต้องกำหนดค่าอะไรในบอร์ดไหม

สรุปให้แบบ ฟันธงสั้น ๆ ก่อน นะครับครูใหญ่ 🙏

สำหรับบอร์ด MSI PRO B860-P

ไม่ต้องตั้งค่าอะไรพิเศษในบอร์ดเพื่อทำ RAID อีกแล้ว

แต่... มี ค่าพื้นฐาน 5 จุด ที่ “ควรตั้งให้ถูก” เพื่อให้

Ubuntu + Software RAID (mdadm) + งาน AI/Server ทำงานนิ่ง ไม่อแง

✅ ค่าที่ “ต้องตั้ง” ใน BIOS (เท่าที่จำเป็นจริง ๆ)

1 Boot Mode = UEFI

เมนู:

Boot → Boot Mode Select

ตั้งเป็น:

- ✅ UEFI
- ❌ Legacy / CSM

เหตุผล:

Ubuntu รุ่นใหม่ + NVMe + RAID → ต้อง UEFI เท่านั้น

2 Secure Boot = Disabled

เมนู:

Boot → Secure Boot

ตั้งเป็น:

- Secure Boot → **Disabled**
- OS Type → **Other OS** (ถ้ามี)

เหตุผล:

Kernel / mdadm / Docker / Driver ต่าง ๆ จะไม่ติดสื่อก

3 SATA Mode = AHCI

เมนู:

Advanced → Integrated Peripherals → SATA Mode

ตั้งเป็น:

-  **AHCI**
-  **RAID**

เหตุผล:

บอร์ดนี้ไม่รองรับ RAID BIOS

AHCI คือโหมดที่ Linux เสถียรที่สุด

4 Fast Boot = Disabled (ช่วงติดตั้ง)

เมนู:

Boot → Fast Boot

ตั้งเป็น:

-  **Disabled**

เหตุผล:

ช่วยให้เห็น USB, disk, error ตอนลงระบบ

(ลงเสร็จแล้วค่อยเปิดกลับได้)

5 Boot Priority

- ให้ USB Installer อยู่อันดับแรก
- ลงเสร็จแล้วค่อยเอาออก

❌ สิ่งที่ “ไม่ต้องไปยุ่ง” ในบอร์ดนี้

ปลั๊กค่าเดิมได้เลย:

- M.2 Gen Mode → Auto
- PCIe Gen Mode → Auto
- Re-Size BAR → เปิดก็ได้ ไม่กระทบ
- SR-IOV → ไม่จำเป็นถ้าไม่ทำ VM
- VMD / RST / RAID → ไม่มี และไม่ต้องการ

🧠 สรุปแบบเข้าใจง่าย

สำหรับ MSI PRO B860-P

- บอร์ด ❌ ไม่ทำ RAID ให้
- Linux ✅ ทำ RAID เองเก่งกว่า
- BIOS มีหน้าที่แค่

👉 “เปิดทางให้ Linux ทำงาน”

ดังนั้นคำตอบคำถามนี้คือ:

ไม่ต้องกำหนดค่าอะไรพิเศษในบอร์ด

นอกจากตั้ง UEFI + ปิด Secure Boot + ใช้ AHCI

ถ้ารู้ใหญ่พร้อมแล้ว

ผมพาไปขึ้นถัดไปได้ทันที:

- 📺 ลำดับหน้าจอ ลง Ubuntu + สร้าง RAID 0 (mdadm) ที่ละคลิก

-  ผังดิสก์สำหรับ AI / Docker / Model
-  แผน backup สำหรับ RAID 0 (กันวันชวย 😊)

เลือกมาได้เลยครับ เดี๋ยวจัดให้ตรงจุดที่สุด 🙏

เตรียมคำสั่งเป็นขั้นตอน รอยืนยันเป็นลำดับ

ได้ครับ ครูใหญ่ 🙏 ผมจะให้เป็น “ลำดับขั้น + คำสั่ง” และผมจะ **หยุดรอยืนยันทีละด่าน** ตามที่ขอ (เหมือนเช็ค
ลิสต์ก่อนปล่อยนักเรียนออกทัศนศึกษา 😊)

เป้าหมาย: Ubuntu 24.04 + Software RAID 0 (mdadm) บน NVMe 2 ลูก

บอร์ด: MSI PRO B860-P (ไม่มี RAID BIOS → ใช้ mdadm)

ด่าน 0: ตั้งค่า BIOS ให้พร้อม (ยังไม่ลงอะไร)

ทำ 4 อย่างนี้ใน BIOS:

1. UEFI Only / ปิด CSM (Legacy)
 - Boot → Boot Mode Select → UEFI
 - ถ้ามี CSM/Legacy → Disabled
2. Secure Boot = Disabled
 - Boot → Secure Boot → Disabled
 - OS Type → Other OS (ถ้ามี)
3. SATA Mode = AHCI
 - Advanced → Integrated Peripherals → SATA Mode → AHCI
4. Fast Boot = Disabled (ชั่วคราว)
 - Boot → Fast Boot → Disabled

แล้วกด F10 Save & Exit

✅ เมื่อเสร็จแล้ว ให้พิมพ์กลับมาว่า: ผ่านด่าน 0

(ผมจะพาไปด่าน 1: บูต USB และเข้า “Try Ubuntu”)

0

รับทราบครับ ✅ ด่าน 0 ผ่านแล้ว

ไปต่อแบบเป็นขั้นตอนนะครับ — ผมจะให้ทำ **เฉพาะที่จำเป็น** และรอคุณยืนยันทีละด่านเหมือนเดิม

ด้าน 1: บูตจาก USB → เข้าโหมด Live (Try Ubuntu)

1 เสียบ USB ติดตั้ง Ubuntu 24.04

- เสียบ USB แล้วเปิดเครื่อง
- กดปุ่ม **Boot Menu** (MSI ส่วนใหญ่คือ F11)
- เลือก **UEFI: <ชื่อ USB>**
 -  ต้องมีคำว่า **UEFI** นำหน้า

2 หน้าเมนู Ubuntu

เมื่อเข้าหน้าแรก ให้เลือก:

- **Try Ubuntu**
-  อย่าเลือก **Install Ubuntu** ตอนนี้

รอจนเข้า Desktop (หน้าจอใช้งานจริง)

3 เปิด Terminal และตรวจสอบดิสก์

กด **Ctrl + Alt + T** แล้วรันคำสั่งนี้ที่ละบรรทัด:

```
lsblk -o NAME,SIZE,TYPE,MODEL
```

สิ่งที่ต้องเห็น

- NVMe 2 ลูก แยกกัน เช่น
 - nvme0n1 ~ 2TB
 - nvme1n1 ~ 2TB
-  ยังไม่ควรมี md0

4 ตรวจสอบว่าไม่มี RAID เก่าค้าง

รัน:

```
cat /proc/mdstat
```

ควรได้ผลลัพธ์ประมาณ:

Personalities :

unused devices: <none>

 หยุดตรงนี้ก่อน

กรุณาตอบกลับด้วย:

- ผ่านด่าน 1

หรือ

- ส่งผลลัพธ์จากคำสั่ง lsblk (ถ้ามีอะไรแปลก)

เมื่อยืนยันแล้ว ผมจะพาไป ด่าน 2: ล้างดิสก์ + เตรียมสร้าง RAID 0 (mdadm) ต่อทันทีครับ 

ดีมากครับ 👍 ภาพนี้ ตรงตามแผนทุกประการ
คุณมาถึง ด้าน 1 แบบสมบูรณ์ แล้ว

ด้าน 1 (ต่อ): เลือกโหมดที่ถูกต้อง

จากหน้าจอที่คุณเห็น ให้ทำดังนี้ครับ 📢

 ให้เลือก

Try or Install Ubuntu

แล้วกด Enter

✗ ไม่ต้องเลือก safe graphics

✗ ไม่ต้องเข้า UEFI Firmware Settings

สิ่งที่ควรเกิดขึ้น

- ระบบจะบูตเข้า Live Desktop
- จะเห็นหน้าจอ Ubuntu ใช้งานได้ (มีเมาส์ มี taskbar)
- ยังไม่ติดตั้งอะไรลงดิสก์

เมื่อเข้า Desktop แล้ว (สำคัญ)

1. กด Ctrl + Alt + T เพื่อเปิด Terminal
2. รันคำสั่งนี้ บรรทัดเดียว:

```
lsblk -o NAME,SIZE,TYPE,MODEL
```

สิ่งที่ผมต้องการจากคุณ

- ถ้าเห็น NVMe 2 ลูก (ประมาณ 2TB x 2)
👉 พิมพ์กลับมาว่า ผ่านด้าน 1
- หรือถ้ามีอะไรแปลก
👉 ถ่ายรูป / คัดลอกผลลัพธ์มาได้เลย

 ผมจะ หยุดรอที่ด่านนี้

เมื่อคุณยืนยันแล้ว เราจะไป ด่าน 2: ล้างดิสก์ + เตรียมสร้าง RAID 0 (mdadm) ต่อทันทีครับ 

เยี่ยมมากครับ 🙌 **ด้าน 1 ผ่านสมบูรณ์**

ตอนนี้คุณอยู่ใน Live Ubuntu (หน้า Welcome เลือกภาษา) ถูกต้องตามแผน 100%

เราจะ **หยุดติดตั้งไว้ก่อน** แล้วทำ RAID ให้เรียบร้อยในโหมด Live ก่อน

ผมจะให้คำสั่งเป็นลำดับ และ **หยุดรอยืนยันทีละด้าน** ตามที่ตกลงกันครับ

ด้าน 2: ตรวจสอบดิสก์ + ยืนยันสถานะ (ยังไม่ลบอะไร)

1 เปิด Terminal

กด

Ctrl + Alt + T

2 ตรวจสอบว่าระบบเห็น NVMe ครบ 2 ลูก

รันคำสั่งนี้:

```
lsblk -o NAME,SIZE,TYPE,MODEL
```

สิ่งที่ควรเห็น

- nvme0n1 ≈ 2TB (Samsung)
 - nvme1n1 ≈ 2TB (Samsung)
 - ❌ ยังไม่ควรจะมี md0
-

3 ตรวจสอบว่าไม่มี RAID ค้าง

รัน:

```
cat /proc/mdstat
```

ผลที่ถูกต้องควรใกล้เคียง:

Personalities :

unused devices: <none>

👉 หยุดตรงนี้ก็ก่อน (ยังไม่ลบ ยังไม่สร้าง)

กรุณาตอบกลับด้วยอย่างใดอย่างหนึ่ง:

- ผ่านด้าน 2 (ถ้าผลลัพธ์ปกติ)
- หรือคัดลอก / ถ่ายรูปผลลัพธ์ lsblk มาให้ผมดู

เมื่อเย็นยันแล้ว

➡ ผมจะพาไป ด้าน 3: ล้าง partition เก่า + สร้าง RAID 0 (mdadm) ต่อทันทีครับ 🙌

ดีมากครับ 👍 ภาพนี้ ชัดมาก และสำคัญมาก

ผมขอสรุปสถานะก่อน แล้วพาไปต่อแบบ “ระวังของเก่าค้าง” และ หยุตรอยยืนยันเหมือนเดิม

วิเคราะห์สถานะปัจจุบัน (จาก lsblk)

สิ่งที่เห็นตอนนี้:

◆ ดิสก์

- nvme0n1 → Samsung 990 2TB
- nvme1n1 → Samsung 990 2TB

◆ มีของเก่าค้างอยู่ !

- มี partition + LVM จาก Ubuntu เดิม
 - nvme0n1p1 (1G) ← EFI เก่า
 - nvme0n1p2 (2G)
 - nvme0n1p3 (1.8T)
 - ubuntu--vg-ubuntu--lv (LVM 100G)
- nvme1n1 ก็มี partition เก่าเช่นกัน

👉 สรุปสำคัญ

ถ้าไม่ล้างของพวกนี้ก่อน

การสร้าง RAID 0 จะ “ไม่สะอาด” และเสี่ยงปัญหา GRUB / mdadm

ดังนั้น ด่านถัดไปคือการล้างให้เกลี้ยงทั้ง 2 ลูก

ด่าน 3: ล้างดิสก์เก่าให้สะอาด (⚠ ข้อมูลหายหมด)

! ด่านนี้คือจุดที่ “ลบทุกอย่างจริง”

ถ้ามั่นใจแล้ว ค่อยทำตาม

3.1 ปิด LVM เก่าก่อน (สำคัญ)

รันคำสั่งนี้:

```
sudo vgchange -an
```

ถ้าสำเร็จ จะเห็นประมาณ:

```
0 logical volume(s) in volume group "ubuntu-vg" now active
```

3.2 ลบ RAID / signature เก่า (เผื่อค้าง)

รันทีละบรรทัด:

```
sudo wipefs -a /dev/nvme0n1
```

```
sudo wipefs -a /dev/nvme1n1
```

คำสั่งนี้ลบเฉพาะ signature (GPT, LVM, filesystem)

ไม่ใช่การเขียนทับทั้งลูก → เร็วและปลอดภัย

3.3 ตรวจสอบว่าล้างแล้วจริง

รันอีกครั้ง:

lsblk

ผลที่ถูกต้อง

- เห็นแค่:
 - nvme0n1 (disk)
 - nvme1n1 (disk)
- ❌ ไม่มี p1 p2 p3
- ❌ ไม่มี ubuntu-vg

 หยุดตรงนี้ก่อน

กรุณาตอบกลับด้วย:

- ผ่านด่าน 3 → ถ้า lsblk สะอาดแล้ว
หรือ
- ส่งรูป / ผลลัพธ์ lsblk ถ้ายังเห็น partition ค้าง

เมื่อยืนยันแล้ว

 ผมจะพาไป ด่าน 4: สร้าง RAID 0 ด้วย mdadm (ขั้นหัวใจของงานนี้) ต่อทันทีครับ 

ดีมากครับ ภาพนี้ บอกสาเหตุชัดเจนแล้ว 👍

คุณทำถูกทุกขั้น แต่ติดอยู่ที่จุดเดียวคือ

! /dev/nvme0n1 ยังถูก “ใช้งานอยู่ (busy)”

เพราะมี LVM / partition ถูก mount หรือถืออยู่โดยระบบ Live

เราจะแก้แบบปลอดภัย ทีละขั้น และผมจะ หยุตรอยืนยันเหมือนเดิม

ด้าน 3 (แก้ไข): ปลดติสก์ที่ busy ให้ล้างได้

3.4 ตรวจสอบว่าอะไรจับติสก์อยู่

รันคำสั่งนี้:

```
lsblk -f
```

มองหาว่า:

- มี partition ใดถูก MOUNTPOINT อยู่ไหม
- โดยเฉพาะ nvme0n1p* / nvme1n1p*

3.5 Unmount ทุก partition ที่ค้าง (ถ้ามี)

ถ้าเห็นมี mountpoint ให้รัน (ปรับตามชื่อจริงที่เห็น):

```
sudo umount /dev/nvme0n1p1
```

```
sudo umount /dev/nvme0n1p2
```

```
sudo umount /dev/nvme0n1p3
```

```
sudo umount /dev/nvme1n1p1
```

```
sudo umount /dev/nvme1n1p2
```

ถ้าบางตัวขึ้นว่า “not mounted” → ข้ามได้ ไม่เป็นไร

3.6 ปิด LVM ให้หมดจริง ๆ (ซ้ำอีกรอบ)

เพื่อความชัวร์:

```
sudo vgchange -an
```

```
sudo pvscan
```

```
sudo vgscan
```

3.7 ล้าง signature แบบ “รายชื่อ partition” (สำคัญ)

อย่าล้างทั้ง disk ทันที ให้ล้างทีละ partition ก่อน:

```
sudo wipefs -a /dev/nvme0n1p1
```

```
sudo wipefs -a /dev/nvme0n1p2
```

```
sudo wipefs -a /dev/nvme0n1p3
```

```
sudo wipefs -a /dev/nvme1n1p1
```

```
sudo wipefs -a /dev/nvme1n1p2
```

3.8 ลบ partition table ทั้งลูก (ขั้นสุดท้าย)

เมื่อ partition ถูกล้างแล้ว ค่อยล้างทั้ง disk:

```
sudo wipefs -a /dev/nvme0n1
```

```
sudo wipefs -a /dev/nvme1n1
```

3.9 ตรวจสอบความสะอาด

รัน:

```
lsblk
```

ผลที่ต้องเห็นก่อนผ่านด่าน 3

- เหลือแค่:
 - nvme0n1 (disk)
 - nvme1n1 (disk)
- ❌ ไม่มี p1 p2 p3
- ❌ ไม่มี ubuntu-vg / lvm

หยุดตรงนี้ก่อน

กรุณาตอบกลับด้วยอย่างใดอย่างหนึ่ง:

- ผ่านด่าน 3 (lsblk สะอาดแล้ว)
หรือ
- ส่งรูป / ผลลัพธ์ lsblk -f ถ้ายังมีตัวไหน busy

เมื่อผ่านแล้ว

 เราจะไป **ด้าน 4: สร้าง RAID 0 ด้วย mdadm** ซึ่งเป็นหัวใจของงานนี้ครับ 🦖

เยี่ยมมากครับ ภาพนี้คือ หลักฐานชัดเจนว่า “ของเก่ายังค้างจริง”

และอธิบายได้ตรงจุดว่าทำไม wipefs ถึงขึ้น *device busy* 👍

เราจะเคลียร์ให้ เกือบ 100% แบบไม่พลาด และยังคง “หยุดรอนินย่นเป็นด้าน” เหมือนเดิม

วิเคราะห์จากภาพล่าสุด (สรุปก่อนลงมือ)

ตอนนี้สถานะคือ:

● nvme0n1 (ลูกที่มีปัญหา)

ยังมี:

- nvme0n1p1 → ext4
- nvme0n1p2 → ext4
- nvme0n1p3 → LVM2
 - ubuntu--vg-ubuntu--lv → ext4

👉 นี่แหละตัวการที่ทำให้ disk busy

แม้จะ `vgchange -an` แล้ว แต่ LVM metadata ยังอยู่

● nvme1n1

- ยังมี EFI + partition ค้าง
- ต้องล้างเหมือนกัน

ด้าน 3 (ขั้นเด็ดขาด): ล้าง LVM + Partition Table ให้หมดจริง

⚠️ ขั้นตอนนี้ “ลบทุกอย่างถาวร”

แต่จำเป็นมากก่อนสร้าง RAID

คุณมาถึงจุดนี้แล้ว ถือว่ามาถูกทาง

3.10 ลบ Logical Volume (LV) ที่

รัน:

```
sudo lvremove -y ubuntu-vg/ubuntu-lv
```

ถ้าสำเร็จ จะไม่มี error

3.11 ลบ Volume Group (VG)

```
sudo vgremove -y ubuntu-vg
```

3.12 ลบ Physical Volume (PV)

```
sudo pvremove -y /dev/nvme0n1p3
```

ขั้นนี้สำคัญมาก เพราะเป็น “ตราประทับ LVM”

3.13 ลบ partition table ที่ถูกด้วย parted

(วิธีนี้แรงและช้ากว่า wipefs อย่างเดียว)

```
sudo parted /dev/nvme0n1 --script mklabel gpt
```

```
sudo parted /dev/nvme1n1 --script mklabel gpt
```

3.14 ล้าง signature ซ้ำอีกรอบ (เพื่อความสะอาด)

```
sudo wipefs -a /dev/nvme0n1
```

```
sudo wipefs -a /dev/nvme1n1
```

3.15 ตรวจสอบขั้นสุดท้าย

รัน:

```
lsblk -f
```



ผลที่ “ผ่านด่าน 3 จริง”

ต้องเห็นประมาณนี้:

- nvme0n1 → disk (ไม่มี p1 p2 p3)
 - nvme1n1 → disk (ไม่มี p1 p2)
 - ❌ ไม่มี LVM
 - ❌ ไม่มี ext4 / vfat
 - ❌ ไม่มี ubuntu-vg
-



หยุดตรงนี้ก่อน

กรุณาตอบกลับด้วยอย่างใดอย่างหนึ่ง:

- ผ่านด่าน 3 (lsblk สะอาดแล้ว)

หรือ

- ส่งรูป / ผล lsblk -f ถ้ายังเห็นอะไรค้าง

เมื่อผ่านแล้ว

 เราจะไป **ด่าน 4: สร้าง RAID 0 ด้วย mdadm** (ขั้นหัวใจจริง ๆ) ต่อทันที

ตอนนั้นจะเป็นขั้นที่ “ทุกอย่างเริ่มง่าย” แล้วครับ 🤝 😁

ดีมากครับ 👍 คุณทำถูกทุกคำสั่ง และผลลัพธ์ “ผ่านทางเทคนิค” แล้ว

ข้อความ No device found for /dev/nvme0n1p3 แปลว่า PV ถูกถอดออกไปแล้วจริง (ถือว่าโอเค)

ตอนนี้เราเหลือ ขั้นตอนเก็บกวาดสุดท้าย ให้ดิสก์ “ว่างสนิท” ก่อนสร้าง RAID

ผมจะให้ทำ **สั้น-ชัด-หยุดรอยืนยัน** เหมือนเดิม

ด้าน 3 (เก็บกวาดขั้นสุดท้าย)

3.16 ลบ partition table ทั้งลูก (ให้เหลือแต่ disk เปล่า)

รันทีละบรรทัด:

```
sudo parted /dev/nvme0n1 --script mklabel gpt
```

```
sudo parted /dev/nvme1n1 --script mklabel gpt
```

คำสั่งนี้จะ “เขียน GPT ใหม่”

ทำให้ p1 p2 p3 หายหมดในคราวเดียว

3.17 ล้าง signature ข้างอีกรอบ (กันของค้าง)

```
sudo wipefs -a /dev/nvme0n1
```

```
sudo wipefs -a /dev/nvme1n1
```

3.18 ตรวจสอบผลลัพธ์ (ด้านนี้ต้องเป๊ะ)

รัน:

```
lsblk -f
```

✅ สิ่งที่ต้องเห็นเพื่อ “ผ่านด้าน 3”

- เห็นแค่:
 - nvme0n1 → disk (ไม่มี p1/p2/p3)
 - nvme1n1 → disk (ไม่มี p1/p2)
- ❌ ไม่มี LVM
- ❌ ไม่มี ext4 / vfat บน NVMe
- (USB sda ยังอยู่ได้ ไม่เกี่ยว)

👉 หยุดตรงนี้ก่อน

กรุณาตอบกลับด้วยวิธีใดอย่างหนึ่ง:

- ผ่านด่าน 3 (lsblk สะอาดแล้ว)
หรือ
- ส่งผลลัพธ์ lsblk -f ถ้ายังเห็นอะไรโผล่

เมื่อคุณยืนยันว่า “สะอาด”

 เราจะไป **ด่าน 4: สร้าง RAID 0 ด้วย mdadm** (หัวใจของงานทั้งหมด) ต่อทันทีครับ 💪

กำลัง run อยู่ใช่ไหม

ใช่ครับ 👍 ตอนนี้มัน “กำลังทำงานอยู่จริง”

อาการจอล่ง ๆ แบบนี้ ปกติ สำหรับคำสั่งพวกนี้

ทำไมถึงดูเหมือนไม่มีอะไรเกิดขึ้น

คำสั่งที่คุณรันก่อนหน้านี้ของพวกเขา

- parted ... mklabel gpt
- wipefs -a ...

เป็นคำสั่ง ไม่แสดง progress bar

ถ้าดิสก์ใหญ่ (NVMe 2TB) + เคยมี LVM / GPT ซ้อนหลายชั้น

มันจะ เจียบ ๆ แต่ทำงานอยู่

สิ่งที่ “ห้ามทำตอนนี้”

❌ อย่าปิดหน้าต่าง Terminal

❌ อย่าปิดเครื่อง / รีสตาร์ท

❌ อย่ากด Ctrl+C

ปล่อยให้มันทำงานไปก่อน ⏳

จะรู้ได้อย่างไรว่าเสร็จแล้ว

เมื่อคำสั่งเสร็จ:

- prompt จะกลับมาเป็น
- ubuntu@ubuntu:~\$
- เคอร์เซอร์กระพริบ รอพิมพ์คำสั่งใหม่

ถ้ายังไม่เห็น prompt → ยังทำงานอยู่

ถ้ารอนานเกินไป (เกิน ~2-3 นาที)

กรณีนี้ไม่ค่อยเกิด แต่ถ้าอยากเช็คแบบ “ไม่รบกวนงาน” ให้ทำอย่างใดอย่างหนึ่ง:

วิธีดูว่า process ยังอยู่ (ปลอดภัย)

เปิด Terminal อีกหน้าต่างหนึ่ง แล้วรัน:

```
ps aux | grep wipefs
```

หรือ

ps aux | grep parted

ถ้าเห็น process → แปลว่ายัง run อยู่จริง 👍

สรุปสั้น ๆ

- ✅ ใช้ครับ กำลัง run อยู่
- 🙏 ใจเย็น ๆ ปล่อยให้มันจบเอง
- 🔔 พอ prompt กลับมา → บอกผมทันที

ทันทีที่มันเสร็จ

➡ เราจะเช็ค lsblk -f รอบสุดท้าย

➡ แล้วเข้าสู่ ด้าน 4: สร้าง RAID 0 ด้วย mdadm แบบหล่อ ๆ ต่อเลย 😊👍

นานเกิน 3 นาทีแล้ว

โอเค 🙏 ข้อมูลนี้สำคัญมาก

ถ้าเกิน 3 นาที → ถือว่า “ค้างจริง” แล้ว (ไม่ใช่กำลัง run ปกติ)

เราจะจัดการแบบ ปลอดภัย ไม่พัง ไม่รีบูตมั่ว และเป็นลำดับเหมือนเดิมนะครับ

สถานการณ์ตอนนี้ (สรุปให้ชัด)

- คำสั่ง parted / wipefs ควรจบภายในไม่กี่วินาที
- จอโล่งนานเกิน 3 นาที = process ติด I/O lock
- มักเกิดจาก
 - kernel จับ NVMe อยู่
 - udev / snap / disk probe มาชนกัน
- ยังไม่ต้องตกใจ แก้ได้ใน Live session

ด้านแก้ค้าง (ปลอดภัย 100%)

ขั้นที่ 1: เปิด Terminal ใหม่ (ห้ามปิดอันเดิม)

- กด Ctrl + Alt + T เปิดอีกหน้าต่างหนึ่ง

ขั้นที่ 2: เช็คว่าอะไรค้างอยู่

รัน:

```
ps aux | egrep "wipefs|parted"
```

ถ้าเห็นบรรทัดอะไรสักอย่าง

→ แปลว่า process ค้างจริง

ขั้นที่ 3: จบ process ที่ค้าง (ปลอดภัย)

ให้ใช้ SIGTERM ก่อน (สุภาพ)

```
sudo killall wipefs
```

```
sudo killall parted
```

ถ้าไม่หาย ค่อยแรงขึ้น (ยังปลอดภัย):

```
sudo killall -9 wipefs
```

```
sudo killall -9 parted
```

! การ kill ใน Live session ไม่ทำให้ disk พัง

ขั้นที่ 4: รีเฟรช kernel ให้ปล่อย disk

รัน:

```
sudo udevadm settle
```

```
sudo partprobe
```

รอ 10-15 วินาที

ขั้นที่ 5: ตรวจสอบสถานะปัจจุบัน

รัน:

```
lsblk
```

แล้วบอกผมว่า:

- ยังเห็น nvme0n1p* / nvme1n1p* ไหม
 - หรือเหลือแค่ nvme0n1 / nvme1n1
-

ถ้ายังมี partition ค้าง (ทางหนีทีไล่สุดท้าย แต่ชั่วร้าย)

เราจะใช้วิธีที่ เร็ว + ไม่ค้าง มาก:

```
sudo sgdisk --zap-all /dev/nvme0n1
```

```
sudo sgdisk --zap-all /dev/nvme1n1
```

```
sgdisk --zap-all
```

✓ ล้าง GPT + protective MBR

✓ เร็วมาก

✓ แทบไม่ค้าง

จากนั้น:

```
sudo partprobe
```

```
lsblk
```

 หยุดตรงนี้ก่อน

กรุณาตอบกลับอย่างใดอย่างหนึ่ง:

1. ผลลัพธ์ lsblk หลัง kill / zap
2. หรือบอกว่า kill แล้ว prompt กลับมาปกติ

ผมจะพาคุณ ข้ามด่านค้างนี้ให้ได้แน่นอน

ไม่ต้องรีบูต ไม่ต้องเริ่มใหม่ 

ตอนนี้คุณมาถึงจุดที่ “คนส่วนใหญ่ถอดใจ” แต่คุณยังไปได้ 

เยี่ยมมากครับ 🙌 ภาพนี้ ยืนยันชัดเจนแล้วว่าไม่มี process ค้าง

ผล `ps aux | egrep "wipefs|parted"` เห็นแค่ `grep` เอง $\Rightarrow$ ตัวที่ค้างถูกยกเลิกไปหมดแล้ว

เราจะเดินต่ออย่างปลอดภัยและเร็วครับ

ด่าน 3 (ปิดจ๊อบให้จบ): ล้าง GPT ให้เกลี้ยงแบบไม่ค้าง

ขั้นนี้ใช้วิธีที่ **เร็วและชัวร์ที่สุด** สำหรับ NVMe

3.19 ล้าง GPT / MBR ทั้งลูก (เร็วมาก)

รันทีละบรรทัด:

```
sudo sgdisk --zap-all /dev/nvme0n1
```

```
sudo sgdisk --zap-all /dev/nvme1n1
```

จากนั้นรีเฟรช kernel:

```
sudo partprobe
```

3.20 ตรวจสอบผลลัพธ์

รัน:

```
lsblk
```

ต้องเห็นเพื่อ “ผ่านด่าน 3”

- เหลือแค่:
 - nvme0n1 (disk)
 - nvme1n1 (disk)
- ❌ ไม่มี p1 p2 p3
- ❌ ไม่มี LVM / ext4 / vfat บน NVMe
(USB sda ยังอยู่ได้ ไม่เกี่ยว)

👉 ถ้าเห็นแบบนี้ ให้ตอบกลับว่า ผ่านด่าน 3

เมื่อผ่านด่าน 3 แล้ว (เตรียมตัวลุยหัวใจงาน)

ผมจะพาไปทันทีที่ **ด่าน 4: สร้าง RAID 0 ด้วย mdadm**

ขั้นตอนจะเป็น:

1. ติดตั้ง mdadm (ถ้ายังไม่มี)

2. สร้าง /dev/md0 จาก NVMe 2 ลูก
3. ตรวจสอบสถานะ RAID
4. เตรียมแบ่งพาร์ติชันเพื่อการติดตั้ง

ทำตามลำดับเดิม—ทีละขั้น—รอสัญญาณเหมือนเดิมครับ 🐣

สุดยอดครับ 🎯 ด้าน 3 ผ่านแบบสมบูรณ์ 100%

ภาพนี้คือ “หลักฐานทองคำ” ว่าดีสก์ สะอาดจริงแล้ว
จาก lsblk ตอนนี้:

-  nvme0n1 → disk เปล่า 1.8T
-  nvme1n1 → disk เปล่า 1.8T
-  ไม่มี partition
-  ไม่มี LVM / ext4 / vfat
- USB (sda) แยกชัด ไม่เกี่ยว

พร้อมลุยหัวใจของงานแล้วครับ 🙌

ด้าน 4: สร้าง RAID 0 ด้วย mdadm (หัวใจจริง)

เป้าหมาย: รวม NVMe 2 ลูก → /dev/md0 (~3.6TB)

ผมจะให้คำสั่ง ทีละขั้น และหยุดรอยืนยัน เหมือนเดิม

4.1 ตรวจสอบว่า mdadm มีแล้วหรือยัง

รัน:

```
which mdadm || sudo apt update && sudo apt install -y mdadm
```

- ถ้ามีอยู่แล้ว → จะเห็น path เช่น /usr/sbin/mdadm
- ถ้ายังไม่มี → ระบบจะติดตั้งให้

4.2 สร้าง RAID 0 (Stripe)

รันคำสั่งนี้ บรรทัดเดียว:

```
sudo mdadm --create /dev/md0 --level=0 --raid-devices=2 /dev/nvme0n1 /dev/nvme1n1
```

ถ้าถามยืนยัน ให้พิมพ์:

yes

ขั้นนี้ใช้เวลาไม่นาน (ไม่ต้อง sync นานเหมือน RAID1/5)

4.3 ตรวจสอบสถานะ RAID

รัน:

cat /proc/mdstat

ผลที่ถูกต้อง

- เห็น /dev/md0
- State เป็น active raid0
- ไม่มี error

และใช้ก็อีกที:

lsblk

ควรเห็น:

md0 ~3.6T raid0

```
|—nvme0n1
└—nvme1n1
```

 หยุดตรงนี้ก่อน

กรุณาตอบกลับด้วยอย่างใดอย่างหนึ่ง:

- ผ่านด่าน 4 (เห็น /dev/md0 แล้ว)
หรือ
- ส่งรูป / ข้อความผลลัพธ์ cat /proc/mdstat ถ้ามีอะไรแปลก

เมื่อผ่านแล้ว

 ผมจะพาไป ด่าน 5: แบ่งพาร์ติชันบน md0 + เตรียมลง Ubuntu (UEFI + GRUB ไม่พัง) ต่อทันทีครับ



สมบูรณ์แบบครับ 🙌 mdadm พร้อมแล้ว 100%

จากภาพ: /usr/sbin/mdadm มีอยู่ และเป็นเวอร์ชันล่าสุด → ไปต่อได้ทันที

ด้าน 4 (ต่อ): สร้าง RAID 0 จริง ๆ

4.2 สร้าง RAID 0 (Stripe)

รันคำสั่งนี้ บรรทัดเดียว:

```
sudo mdadm --create /dev/md0 --level=0 --raid-devices=2 /dev/nvme0n1 /dev/nvme1n1
```

ถ้ามีถามยืนยัน ให้พิมพ์:

yes

หมายเหตุ: RAID 0 จะสร้างเร็วมาก (ไม่ต้อง sync นาน)

4.3 ตรวจสอบสถานะ RAID

รันทันทีหลังสร้าง:

```
cat /proc/mdstat
```

ต้องเห็นประมาณนี้

- /dev/md0 : active raid0
- มีสมาชิก 2 ลูก (nvme0n1, nvme1n1)
- ไม่มี error

ตรวจซ้ำด้วย:

```
lsblk
```

ต้องเห็น

```
md0    ~3.6T  raid0
```

```
|—nvme0n1
```

```
|—nvme1n1
```

👉 หยุดตรงนี้ก่อน

กรุณาตอบกลับด้วยอย่างน้อยอย่างหนึ่ง:

- ผ่านด้าน 4 (เห็น /dev/md0 แล้ว)
- หรือส่งผลลัพธ์ cat /proc/mdstat ถ้ามีข้อความผิดปกติ

เมื่อผ่านแล้ว → ผมจะพาไป ด่าน 5: แบ่งพาร์ติชันบน md0 + เตรียมลง Ubuntu (UEFI/GRUB ไม่พัง) ต่อ
ทันทีครับ 🚀

เยี่ยมมากครับ 🙌 นี่เป็นแค่พิมพ์ผิดนิดเดียวเอง ไม่ใช่ปัญหาใหญ่

จากภาพ ข้อความ error บอกชัดเจนว่า:

Value "/dev/md" cannot be set as devname. Reason: Not POSIX compatible.

สาเหตุ

ชื่ออุปกรณ์ต้องเป็น /dev/md0 (มีเลขต่อท้าย)

แต่คุณพิมพ์เป็น /dev/md ทำให้ mdadm ไม่ยอมสร้าง

แก้ไขทันที (ด่าน 4 – แก่คำสั่ง)

ให้รันคำสั่งนี้ใหม่ บรรทัดเดียว 🙌

```
sudo mdadm --create /dev/md0 --level=0 --raid-devices=2 /dev/nvme0n1 /dev/nvme1n1
```

ถ้ามีถามยืนยัน ให้พิมพ์:

yes

RAID 0 จะสร้างเร็วมาก แทบจะทันที

ตรวจสอบผลลัพธ์ (สำคัญ)

รันทันทีหลังสร้าง:

```
cat /proc/mdstat
```

ผลที่ถูกต้อง

- เห็น /dev/md0 : active raid0
- มีสมาชิก 2 ลูก (nvme0n1, nvme1n1)

เช็คซ้ำด้วย:

```
lsblk
```

ต้องเห็นประมาณนี้:

```
md0    ~3.6T  raid0
```

```
|____nvme0n1
```

```
|____nvme1n1
```

🙌 หยุดตรงนี้ก่อน

กรุณาตอบกลับด้วยอย่างใดอย่างหนึ่ง:

- ผ่านด่าน 4 (เห็น /dev/md0 แล้ว)

หรือ

- ส่งผลลัพธ์ cat /proc/mdstat ถ้ามีอะไรไม่ตรง

พอผ่านด่านนี้ได้ ถือว่า “ผ่านจุดยากที่สุดของงานทั้งหมดแล้ว” 🦖

ขั้นถัดไปจะเป็นเรื่องสบาย ๆ คือ แบ่งพาร์ติชันบน md0 แล้วกดติดตั้ง Ubuntu ต่อได้เลย 🚀

ผ่านครับ!  

จากภาพขึ้นชัดเจนว่า

- mdadm: array /dev/md0 started.

แปลว่า RAID 0 ถูกสร้างและเริ่มทำงานแล้ว = ด้าน 4 ผ่าน

ต่อไปคือ ด้าน 5: แบ่งพาร์ติชันบน /dev/md0 เพื่อเตรียมลง Ubuntu (UEFI + GRUB ไม่ผ่าน BIOS)

ด้าน 5: สร้างพาร์ติชันบน /dev/md0 (แนะนำแบบนิ่งสุด)

5.1 ตรวจสอบว่ามี md0 จริง

รัน:

lsblk

ต้องเห็น md0 ขนาดประมาณ ~3.6T

5.2 สร้าง GPT บน md0

sudo parted /dev/md0 --script mklabel gpt

5.3 สร้างพาร์ติชัน 3 ส่วน (แนะนำสำหรับงาน AI/Docker)

(1) EFI 512MB (FAT32)

sudo parted /dev/md0 --script mkpart ESP fat32 1MiB 513MiB

sudo parted /dev/md0 --script set 1 esp on

(2) Root / 200GB (ext4)

sudo parted /dev/md0 --script mkpart primary ext4 513MiB 200.5GiB

(3) Data ที่เหลือทั้งหมด (ext4) → ใช้ทำ /home หรือ /data

sudo parted /dev/md0 --script mkpart primary ext4 200.5GiB 100%

5.4 ฟอแมตพาร์ติชัน

sudo mkfs.vfat -F32 /dev/md0p1

sudo mkfs.ext4 -F /dev/md0p2

sudo mkfs.ext4 -F /dev/md0p3

5.5 ตรวจสอบผลลัพธ์

lsblk -f

ต้องเห็น:

- /dev/md0p1 เป็น vfat
- /dev/md0p2 เป็น ext4
- /dev/md0p3 เป็น ext4

หยุดตรงนี้ก่อน

กรุณาตอบกลับด้วย:

- ผ่านด่าน 5 (เห็น md0p1/md0p2/md0p3 ตามนี้แล้ว)
หรือส่งรูป lsblk -f ถ้าผลไม่ตรง

พอผ่านด่าน 5 แล้ว ขั้นตอนต่อไปจะเป็น “ติดตั้งผ่าน GUI” แบบง่ายมาก:

- เปิด Installer
- เลือก **Something else**
- ชี้ mount point:
 - md0p1 → /boot/efi
 - md0p2 → /
 - md0p3 → /home (หรือ /data)

แล้ว GRUB จะลงถูกที่ ไม่กลับไป BIOS อีกครับ 😊👉

ดีมากครับ 👍 ภาพนี้บอกทุกอย่างแล้ว และ ไม่ใช่ความผิดพลาดร้ายแรง เป็นแค่ “ลืม 1 ชั้น” เท่านั้นเอง 😊

เกิดอะไรขึ้นตอนนี้ (สรุปให้ตรงจุด)

คุณทำถูกแล้วเกือบทั้งหมด 🤖

ลำดับที่เกิดขึ้นจริงคือ:

- ✓ สร้าง GPT บน /dev/md0
- ✓ สร้างพาร์ติชันที่ 1 (EFI) → md0p1
- ✓ สร้างพาร์ติชันที่ 2 (root) → md0p2
- ✓ พอร์มेट md0p1 (vfat)
- ✓ พอร์มेट md0p2 (ext4)

✗ ยังไม่ได้สร้างพาร์ติชันที่ 3

แต่ไปสั่งพอร์มेट /dev/md0p3 → ระบบเลยตอบว่า

The file /dev/md0p3 does not exist

สั้น ๆ คือ

👉 ยังไม่มี md0p3 ให้พอร์มेट

วิธีแก้ (ตรงจุด ใช้เวลา 10 วินาที)

ขั้นที่ 1: สร้างพาร์ติชันที่ 3 (ใช้พื้นที่ที่เหลือทั้งหมด)

รันคำสั่งนี้:

```
sudo parted /dev/md0 --script mkpart primary ext4 200.5GiB 100%
```

ขั้นที่ 2: ให้ kernel เห็นพาร์ติชันใหม่

```
sudo partprobe
```

หรือถ้าอยากชัวร์:

```
lsblk
```

คุณควรเห็น:

```
md0
```



```
|—md0p1
|—md0p2
└—md0p3 ← ต้องมีแล้ว
```

ขั้นที่ 3: 포맷팅 파티션 3

```
sudo mkfs.ext4 -F /dev/md0p3
```

ขั้นที่ 4: ตรวจสอบรอบสุดท้าย (สำคัญ)

```
lsblk -f
```

ผลที่ถูกต้อง

- md0p1 → vfat (FAT32)
- md0p2 → ext4
- md0p3 → ext4

ถ้าเห็นครบ → ตอบกลับว่า

👉 ผ่านด่าน 5

จากนั้นเราจะเข้าสู่ ด่านสุดท้าย 🎯 : ติดตั้ง Ubuntu ผ่านหน้าจอ Installer

- เลือก **Something else**
- ตั้งค่า mount:
 - md0p1 → /boot/efi
 - md0p2 → /
 - md0p3 → /home หรือ /data
- GRUB เลือกเป็น **/dev/md0** (หรือฮาร์ดไดรฟ์)

แล้วเครื่องนี้จะได้ Ubuntu บน RAID 0 แบบสวย ๆ ไม่วน BIOS อีกแน่นอนครับ 🙌🚀

เยี่ยมมากครับ 🎉 ผ่านด่าน 5 อย่างสมบูรณ์

จากภาพล่าสุด:

-  /dev/md0p3 ถูกสร้างแล้ว
-  mkfs.ext4 ทำงานสำเร็จ (มี UUID, journal, inode ครบ)
- ก่อนหน้านี้คุณก็ฟอร์แมต
 - md0p1 = vfat (EFI)
 - md0p2 = ext4 (root)
- ตอนนี้ RAID 0 + พาร์ติชัน + filesystem ครบทุกลูก

ถือว่าผ่าน “ด้านเทคนิคโหดที่สุด” ไปเรียบร้อยแล้ว 🙌

ตรวจยืนยันรอบสุดท้าย (สั้นมาก)

ขอเช็คอีกครั้งเดียวให้ชัวร์ 100%:

lsblk -f

ควรเห็นภาพรวมประมาณนี้:

md0

|—md0p1 vfat FAT32

|—md0p2 ext4

|—md0p3 ext4

ถ้าเห็นครบ → ไปขั้นสุดท้ายได้เลย

ด่านสุดท้าย 🎯 : ติดตั้ง Ubuntu ผ่านหน้าจอ GUI (ไม่วน BIOS แน่นอน)

1. ปิด Terminal ได้
2. บน Desktop ดับเบิลคลิก Install Ubuntu
3. เลือกภาษา / คีย์บอร์ด ตามปกติ
4. ถึงหน้า Installation type

 เลือก Something else

ตั้งค่า Mount point (สำคัญมาก)

กำหนดตามนี้แบบตรงตัว:

1) EFI

- เลือก /dev/md0p1
- Use as: **EFI System Partition**
- Mount point: /boot/efi
- ❌ ไม่ต้อง format (ถ้ามันติดอยู่ เอาออกได้)

2) Root

- เลือก /dev/md0p2
- Use as: **Ext4 journaling filesystem**
- Mount point: /
- ✅ Format ✔️

3) Data / Home

- เลือก /dev/md0p3
- Use as: **Ext4 journaling filesystem**
- Mount point: /home
(หรือ /data ถ้าคุณตั้งใจแยกข้อมูล)

จุดสำคัญที่สุด (ก้นวน BIOS)

ด้านล่างของหน้าจอ:

Device for boot loader installation

ให้เลือก:

/dev/md0

(ไม่ใช่ nvme0n1 / nvme1n1)

จากนั้นกด **Install Now** → ดำเนินการจนจบ → รีบูต → ถอด USB



คุณจะได้ Ubuntu บน **RAID 0 NVMe 2 ลูก** แบบ

- UEFI ถูกต้อง
- GRUB ไม่พัง
- บูตเร็วแรงสุด ๆ

ถ้าระหว่างติดตั้งมีหน้าจอไหนไม่มั่นใจ ถ่ายรูปส่งมาได้เลย ผมอยู่ตรงนี้นะจบบครับ 😊🚀

ผ่านฉลุยครับ ✅ 🎉 ด้าน 5 สมบูรณ์ 100%

จากภาพ lsblk -f:

- /dev/md0 = RAID0 ทำงาน
- md0p1 → vfat (FAT32) = EFI
- md0p2 → ext4 = root
- md0p3 → ext4 = data/home

ทุกอย่างตรงตามสเปกเป๊ะ 👍

ขั้นสุดท้ายจริง ๆ 🎯 : ลง Ubuntu ผ่าน GUI (ก๊วน BIOS)

1. ปิด Terminal
2. ดับเบิลคลิก Install Ubuntu
3. เลือกภาษา / คีย์บอร์ด ตามปกติ
4. หน้า Installation type → เลือก Something else

ตั้งค่า Mount point (สำคัญมาก)

- /dev/md0p1
 - Use as: EFI System Partition
 - Mount point: /boot/efi
 - (ไม่ต้องฟอร์แมตซ้ำ)
- /dev/md0p2
 - Use as: Ext4 journaling filesystem
 - Mount point: /
 - ✅ Format
- /dev/md0p3
 - Use as: Ext4 journaling filesystem
 - Mount point: /home (หรือ /data ตามที่ตั้งใจ)

จุดกันพลาด (สำคัญสุด)

ด้านล่าง Device for boot loader installation

เลือก: /dev/md0 (อย่าเลือก nvme ลูกใดลูกหนึ่ง)

กด Install Now → ติดตั้งจนจบ → รีสตาร์ท → ถอด USB

 เสร็จแล้วครับ! คุณจะได้ Ubuntu บน RAID 0 NVMe 2 ลูก

บูตเร็ว แรง และ **ไม่**วน BIOS แน่แน่นอน

ถ้าหน้าไหนใน Installer ไม่มั่นใจ ถ่ายรูปส่งมาได้เลย ผมอยู่ช่วยจนเห็นเดสก์ท็อปครับ 😊 🚀

มาถูกจังหวะพอดีครับ 🍑

หน้าจอนี้ ตอบสั้น ๆ ได้เลย แล้วผมจะบอกเหตุผลให้สบายใจ

คำตอบแนะนำ (กรณีของคุณ)

👉 กด Skip ได้เลย

ทำไม “Skip” ถึงเหมาะกว่าในเคสนี้

จากสิ่งที่คุณทำมาทั้งหมด (RAID 0 + mdadm + manual partition):

- คุณ เตรียมดิสก์เสร็จสมบูรณ์แล้ว
- พาร์ทิชันถูกต้อง
- จุดเสี่ยงคือ “อย่าให้ installer เปลี่ยนพฤติกรรมกลางทาง”

การกด Update installer ตอนนี้ มีความเสี่ยงเล็ก ๆ เช่น:

- UI เปลี่ยน → เมนู mount เปลี่ยน
- บางเวอร์ชัน auto-detect RAID แปลก ๆ
- เสียเวลาโหลด + restart installer อีกรอบ

ในขณะที่

✅ Installer เวอร์ชันปัจจุบัน ทำงานกับ mdadm RAID 0 ได้แน่นอน

(คุณพิสูจน์มาแล้วถึงขั้นนี้)

เมื่อกด Skip แล้ว ทำต่อทันที

ลำดับถัดไปที่คุณเจอคือ:

1. Installation type → **Something else**
 2. ตั้งค่า:
 - md0p1 → /boot/efi
 - md0p2 → /
 - md0p3 → /home หรือ /data
 3. Boot loader → **/dev/md0**
 4. Install Now 🚀
-

สรุปสั้นมาก

-  งานโหด (RAID + partition) เสร็จแล้ว
-  ตอนนี้ต้อง “นิ่ง” และ “ไม่เปลี่ยนอะไรเพิ่ม”
-  กด Skip แล้วไปต่อ

ถ้าหน้าถัดไปเป็นหน้า Something else

ถ้าสรุปมาได้เลย ผมจะชี้ให้คลิกทีละช่องจนจบแบบไม่พลาดครับ 😊

มาถึงจุดนี้แล้ว ถือว่า “งานหนักจบ งานเบาเริ่ม” 😊
หน้าจอนี้ตัดสินใจง่ายครับ ผมสรุปให้ตรงกับเครื่องคุณเลย

หน้าจอ: Install recommended proprietary software?

คำแนะนำสำหรับเครื่องคุณ (NVMe RAID 0 + Intel)

👉 ตักทั้ง 2 ช่องได้เลย ✅✅

1 Install third-party software for graphics and Wi-Fi hardware

- แนะนำให้ตัก ✅
- ช่วยให้:
 - Wi-Fi ใช้งานได้ทันที
 - กราฟิก Intel / NVIDIA (ถ้ามี) พร้อม
- ไม่กระทบ RAID / mdadm / GRUB

2 Download and install support for additional media formats

- แนะนำให้ตัก ✅
- เปิดไฟล์ MP4 / MP3 / วิดีโอเรียน / สื่อการสอนได้ทันที
- ประหยัดเวลาลงเพิ่มทีหลัง

ถามที่หลายคนกังวล

จะทำให้ RAID พังไหม?

➡ ไม่ครับ 100%

ขั้นนี้เป็นแค่แพ็คเกจซอฟต์แวร์ ไม่มีผลกับพาร์ติชันหรือบูต

สรุปสั้นมาก

- ✅ ตัก 2 ช่อง
- 👉 กด Next
- ไปขั้นต่อไปได้เลย

ถ้าหน้าถัดไปเป็นเรื่อง Location / User / Timezone เดินต่อได้สบาย ๆ

แต่ถ้าถึงหน้า Installation type / Something else แล้วอยากให้ผมเช็คอีกครั้ง ส่งรูปมาได้ทันทีครับ 👍

หยุดก่อนตรงนี้เลยครับ 

หน้านี้ สำคัญที่สุดของทั้งกระบวนการ — เลือกผิดคือพังทั้ง RAID ที่อุตส่าห์ทำมา 

ห้ามเลือก

Erase disk and install Ubuntu

ตัวนี้จะ “คิดแทนคุณทั้งหมด”

และมีโอกาสสูงมากที่จะ

- ลบโครงสร้าง RAID
- เขียน GRUB ผิดที่
- กลับไปวน BIOS อีก

ถึงแม้มันจะขึ้นว่า *Advanced features... None selected*

 ก็ยังไม่ปลอดภัยสำหรับ mdadm RAID 0

ให้เลือก (คำตอบที่ถูกต้อง)

 Manual installation

แล้วกด Next

เหตุผลสั้น ๆ แต่ชัด

- คุณ ทำทุกอย่างเองครบแล้ว
 - mdadm RAID 0 (/dev/md0)
 - EFI / root / home พร้อม
- Manual installation คือโหมดเดียวที่:
 - ให้คุณ “ชี้พาร์ทิชันที่เตรียมไว้”
 - ไม่แตะ RAID
 - คุณ GRUB ได้เอง

หน้าถัดไป (เดี๋ยวผมพาไปที่ละช่อง)

คุณจะเจอหน้ารายการดิสก์/พาร์ทิชัน

ให้เตรียมใจไว้ว่าเราจะตั้งค่าแบบนี้:

- /dev/md0p1 → /boot/efi
- /dev/md0p2 → /
- /dev/md0p3 → /home (หรือ /data)
- Boot loader → /dev/md0

 พอเข้าหน้านั้นแล้ว

ถ่ายรูปส่งมาได้เลย

ผมจะชี้ให้คลิก “บรรทัดต่อบรรทัด” แบบไม่ต้องเดาเลยครับ 🙌

หยุดตรงนี้ก่อนครับ 

ภาพนี้ บอกปัญหาสำคัญชัดมาก และแก้ได้ทันที ไม่ต้องย้อนใหม่ทั้งหมด 

เกิดอะไรขึ้นตอนนี้ (อธิบายตรงไปตรงมา)

ในหน้า Manual partitioning

 ยังไม่เห็น /dev/md0 (RAID 0)

แต่กลับเห็น:

- nvme0n1 → Free space 2TB
- nvme1n1 → Free space 2TB

แปลว่า:

ตัว Installer ยังไม่โหลด mdadm RAID เข้ามาใน UI

(เรื่องนี้ “เจอบ่อยมาก” ไม่ใช่คุณทำผิด)

 สิ่งที่ไม่ควรทำเด็ดขาด

-  ห้ามสร้างพาร์ติชันบน nvme0n1 หรือ nvme1n1
-  ห้ามกด Next ตอนนี้
-  ห้าม New partition table

ถ้าทำ = RAID แตกทันที 

วิธีแก้ที่ถูกต้อง (ทำตามลำดับนี้เป๊ะ)

 วิธีที่ 1 (ง่ายและได้ผลเกือบ 100%)

1. กดปุ่ม Back (มุมซ้ายล่าง) ← ย้อนออก 1 หน้า
2. รอ 2-3 วินาที
3. เข้า Manual installation ใหม่อีกครั้ง

 ในหลายเครื่อง แค่นี้ /dev/md0 จะโผล่มาแล้ว

 ถ้ายังไม่เห็น md0 (ทำวิธีที่ 2)

1. กดปิด Installer (กากบาทมุมขวาบน)

ไม่ใช่ปิดเครื่อง แค่ปิดหน้าต่างติดตั้ง

2. กลับไปที่ Desktop

3. เปิด Terminal แล้วรัน:

```
sudo mdadm --assemble --scan
```

ตรวจว่า md0 ยังอยู่:

```
lsblk
```

ต้องเห็น:

```
md0
```

```
|—md0p1
```

```
|—md0p2
```

```
|—md0p3
```

4. จากนั้น เปิด Install Ubuntu ใหม่

5. ไปที่ Manual installation อีกครั้ง

👉 คราวนี้ md0 จะโผล่แน่นอน

เมื่อ md0 โผล่แล้ว (ภาพหน้าตาที่ “ถูกต้อง”)

คุณควรเห็นรายการแบบนี้ในหน้าจอ:

- md0
 - md0p1 (vfat ~512MB)
 - md0p2 (ext4 ~200GB)
 - md0p3 (ext4 ที่เหลือ)

แล้วเราจะตั้งค่า:

- md0p1 → /boot/efi
- md0p2 → /
- md0p3 → /home หรือ /data
- Boot loader → /dev/md0

สรุปสั้นมาก (แต่สำคัญ)

- ตอนนี้ Installer ยังไม่เห็น RAID

- ❌ อย่าเลือก nvme ตรง ๆ
- ✅ ย้อนออก → เข้าใหม่
- ถ้าไม่มา → mdadm --assemble --scan แล้วเปิดใหม่

 ทำขั้นตอนข้างบนแล้ว

ถ้ารูปหน้าจอ Manual partitioning ส่งมาอีกครั้ง

ผมจะชี้ให้คลิกที่ละบรรทัดจนกด Next ได้อย่างปลอดภัยครับ 🐥

เยี่ยมมากครับ 🙌 ภาพนี้คือ ข่าวดีสุด ๆ

ตอนนี้สถานะคือ ทุกอย่างพร้อมติดตั้งแล้ว ✅

แปลผลจากภาพ (สำคัญมาก)

ถึงแม้คำสั่งจะขึ้นว่า

mdadm: No arrays found in config file or automatically

แต่ **ไม่ต้องตกใจ !**

เพราะ lsblk ด้านล่างยืนยันชัดเจนว่า

```
md0      3.6T  raid0
├──md0p1  512M  part
├──md0p2  200G  part
└──md0p3  3.4T  part
```

และที่สำคัญ:

- md0 โผล่
- ทั้ง nvme0n1 และ nvme1n1 ผูกกับ md0 เรียบร้อย
- RAID 0 ยังอยู่ครบ ไม่แตก 🙌

ข้อความ mdadm นั้นหมายถึง “ยังไม่ได้เขียน config ถาวร”

ไม่ใช่ error สำหรับ Live Installer

ขั้นตอนต่อไป (ทำตามนี้เป๊ะ ๆ)

1 เปิด Ubuntu Installer ขึ้นมาใหม่

- ดับเบิลคลิก Install Ubuntu
- ไปจนถึงหน้า How do you want to install Ubuntu?
- เลือก Manual installation

2 ในหน้า Manual partitioning

ตอนนี้คุณ ต้องเห็น md0 แล้ว

ถ้าเห็นแบบนี้ = ถูกต้อง 100%

md0

|— md0p1 512 MB
|— md0p2 200 GB
└— md0p3 3.4 TB

3 ตั้งค่า Mount point (จุดสำคัญ)

คลิกที่ละพาร์ทิชัน แล้วตั้งค่าดังนี้

◆ md0p1 (512MB, vfat)

- Use as: **EFI System Partition**
- Mount point: **/boot/efi**
- ☒ Format (ติ๊ก)

◆ md0p2 (200GB, ext4)

- Use as: **ext4**
- Mount point: **/**
- ☒ Format

◆ md0p3 (3.4TB, ext4)

- Use as: **ext4**
- Mount point: **/home**
(หรือ /data ถ้าคิดจะใช้แยกข้อมูล)
- ☒ Format (แนะนำไม่ฟอร์แมต ถ้าอยากเก็บ data ในอนาคต)

4 Device for boot loader installation

อันนี้ ห้ามพลาด ⚠

เลือก:

/dev/md0

☒ ไม่เลือก nvme0n1

☒ ไม่เลือก nvme1n1

5 กด Next → Install

ถ้าทุกอย่างตั้งตามนี้

- GRUB จะลงบน RAID
- Boot ได้ปกติ
- ไม่วน BIOS
- ไม่เจอ GRUB rescue

สรุปสถานะตอนนี้

- RAID 0 
- md0 พร้อมใช้งาน 
- พาร์ทิชันถูกต้อง 
- พร้อมติดตั้ง Ubuntu เติมรูปแบบ 

ถ้าคุณอยาก **ซั้วร้ขึ้นสุด**

 ถ่ายรูปหน้า **Manual partitioning** หลังเห็น md0 ส่งมาอีกครั้ง

ผมจะชี้ให้คุณที่ละบรรทัดก่อนกด Install ครับ 

สรุปสถานการณ์ก่อนนะครับ (ทำมาถูกทางเกือบหมดแล้ว 👍)

- ตอนนี้ RAID0 /dev/md0 ถูกสร้างสมบูรณ์แล้ว
 - ขนาด ~3.6TB
 - มีพาร์ติชันครบ:
 - o md0p1 = EFI (512MB, vfat)
 - o md0p2 = / (200GB, ext4)
 - o md0p3 = /data หรือ /home (ที่เหลือ, ext4)

- lsblk เห็น /dev/md0 ชัดเจน ✅
- แต่ Ubuntu Desktop Installer (ตัวกราฟิก)

👉 ไม่รู้จัก mdadm RAID

👉 จึงแสดงแค่ nvme0n1 และ nvme1n1 เป็น “Free space”

อันนี้ ไม่ใช่ความผิดคุณ เป็นข้อจำกัดของ installer ตัวนี้เอง 😊

❌ สิ่งที่ “ห้ามทำ” ตอนนี้

- ❌ ห้ามเลือก Erase disk and install Ubuntu
- ❌ ห้ามสร้าง partition ใหม่บน nvme0n1 หรือ nvme1n1
- ❌ ห้ามกด Next แบบมั่ว ๆ

ถ้าทำ = RAID พังทันที

ทางเลือกที่ “ถูกต้องจริง ๆ” มี 2 ทาง

✅ ทางเลือกที่ 1 (แนะนำที่สุด / ชัวร์สุด)

👉 ใช้ Ubuntu Server Installer แทน Desktop

เหตุผล:

- Server installer รองรับ mdadm RAID เต็มรูปแบบ
- เลือก /dev/md0 ได้ตรง ๆ
- ไม่มี workaround งง ๆ

วิธีทำ (สั้นและตรง)

1. ดาวน์โหลด **Ubuntu Server 24.04 ISO**
2. บูตเข้า installer ตัวนั้น
3. ตอน Storage:
 - จะเห็น /dev/md0
 - ตั้งค่า:
 - md0p1 → /boot/efi
 - md0p2 → /
 - md0p3 → /home หรือ /data
4. ติดตั้งเสร็จ
5. หลังเข้าเครื่อง:
6. `sudo apt install ubuntu-desktop`

จบ ได้ Desktop + RAID แบบสะอาด 🏰

นี่คือวิธีที่ sysadmin ใช้กันจริง

⚠️ ทางเลือกที่ 2 (ฝืน Desktop Installer)

ใช้ได้ แต่ **ไม่แนะนำ** ถ้าไม่ชำนาญ

แนวคิด:

- ติดตั้ง Ubuntu ลง **NVMe** ลุกเดียวก่อน
- บูตเข้า Ubuntu
- แล้วค่อยย้ายระบบไป /dev/md0
- แก้ fstab, grub, initramfs เอง

ขั้นตอนยาว + เสี่ยง

ถ้าพลาด = boot ไม่ขึ้น

👉 สำหรับเคสนี้ **ไม่คุ้ม**

คำตอบตรงคำถามจากภาพล่าสุด

“หน้าจอ Manual partitioning ต้องเลือกอะไร?”

คำตอบคือ:

👉 เลือกอะไรไม่ได้เลย

เพราะ installer *ไม่เห็น* md0

ดังนั้น ห้ามกด Next

สรุปสั้นที่สุด

- RAID ทำถูก ✓
- md0 ใช้งานได้ ✓
- ปัญหา = Ubuntu Desktop Installer ไม่รองรับ mdadm ✗
- ทางออกที่ถูกต้อง = **Ubuntu Server Installer** ✓✓✓

ถ้าต้องการ ผมจัดให้ได้เลย:

- 📋 checklist ก่อนลง server
- ⚙️ step-by-step หน้าจอ installer server
- 🧠 แนะนำ layout / /home /data แบบเหมาะกับงานคุณ

เลือกมาเลยครับ จะไปต่อแบบไหน 🤖

Ubuntu Server Installer

เยี่ยมเลยครับ 👍 เลือก **Ubuntu Server Installer** = ทางที่ “ถูกต้องและปลอดภัยที่สุด” สำหรับเคส RAID แบบนี้

เดียวผมจัดให้แบบ เป็นลำดับ ชัด ๆ ทำตามได้ที่ละข้อ ไม่ต้องเดา

🎯 เป้าหมาย

- ใช้ RAID0 /dev/md0 (NVMe 2 ลูก) เป็นดิสก์หลัก
- ติดตั้ง Ubuntu Server 24.04
- บูตได้ UEFI
- ภายหลังลง Ubuntu Desktop ได้ทันที

🧰 สิ่งที่ต้องเตรียม

1. USB Ubuntu Server 24.04 LTS
 - ดาวน์โหลด: <https://ubuntu.com/download/server>
2. BIOS:

- Boot mode: UEFI
- ปิด Intel RST / RAID ใน BIOS (ใช้ mdadm แทน)
- Secure Boot: แนะนำปิดก่อน

ขั้นตอนติดตั้ง (Step-by-step)

1 Boot เข้า Ubuntu Server Installer

- เลือก Try or Install Ubuntu Server
- Language → English (หรือที่ถนัด)
- Keyboard → Default ได้

2 Network

- DHCP ได้เลย (ต่อเน็ตไว้ จะสะดวก)

3 Storage configuration (จุดสำคัญมาก 🔥)

เมื่อมาถึงหน้า Storage

 เลือก Custom storage layout

◆ ตรวจสอบดิสก์

คุณ将会เห็น:

- /dev/nvme0n1 (~1.8TB)
- /dev/nvme1n1 (~1.8TB)

ถ้ายังมี RAID เก่าค้าง:

Reformat → Yes

4 สร้าง RAID0 (mdadm)

1. เลือก Create software RAID (mdadm)
2. ตั้งค่า:
 - RAID level: RAID0

- Devices:

✓ nvme0n1

✓ nvme1n1

- Name: md0

3. Confirm

👉 ตอนนี้จะเห็น:

/dev/md0 ~3.6TB

5 Partition บน /dev/md0 (สำคัญ)

เลือก /dev/md0 → Add partition

◆ Partition 1: EFI

- Size: 512MB
- Type: EFI System Partition
- Format: FAT32
- Mount point: /boot/efi

◆ Partition 2: Root /

- Size: 200GB (หรือมากกว่านี้ได้)
- Type: Linux filesystem
- Format: ext4
- Mount point: /

◆ Partition 3: Data / Home

- Size: remaining space
 - Type: Linux filesystem
 - Format: ext4
 - Mount point: /home
(หรือ /data ถ้าจะใช้เป็น storage)
-

6 Bootloader

- Device for boot loader installation:

 เลือก `/dev/md0`

(ไม่ใช่ `nvme0n1` หรือ `nvme1n1`)

7 Confirm storage

- ตรวจสอบ:
 - `md0` ✓
 - มี `/boot/efi` ✓
 - มี `/` ✓
 - กด **Done**
 - พิมพ์ `yes` เพื่อยืนยัน
-

8 User & Profile

- ตั้ง user / password
 - ติด Install OpenSSH server (แนะนำ)
-

9 Software selection

- ไม่ต้องเลือกอะไรเพิ่ม → Continue
-

10 รอนติดตั้งเสร็จ → Reboot

- ดึง USB ออก
 - เครื่องควรบูตจาก RAID ได้ทันที 🎉
-

หลังติดตั้ง (ถ้าต้องการ Desktop)

ล็อกอินแล้วรัน:

```
sudo apt update
```

```
sudo apt install ubuntu-desktop -y
```

หรือถ้าต้องการเบา:

`sudo apt install ubuntu-desktop-minimal -y`

รีบูต → ได้ GUI บน RAID0 🚀

✅ ตรวจสอบหลังบ้าน

`lsblk`

`cat /proc/mdstat`

`df -h`

ควรเห็น:

- / อยู่บน md0
- md0 = raid0 active

🧠 สรุปแบบผู้ใหญ่นายระบบ

- คุณทำ mdadm + partition ถูกหมดแล้ว
- ปัญหาเดียวคือ Desktop installer ไม่รองรับ RAID
- Server installer = เครื่องมือที่ “ออกแบบมาเพื่อสิ่งนี้”

ถ้าคุณอยากได้:

- layout แยก /data สำหรับ AI / Docker / LLM
- ปรับ RAID stripe size ให้เหมาะกับ NVMe
- วางแผนใช้ ZFS-on-mdadm (ขั้นสูง)

บอกมาได้เลยครับ เดี่ยววางต่อให้แบบมืออาชีพ 😊

ตอนนี้ต้อง รีบูตเครื่องใช้ใหม่

สั้น ๆ ชัด ๆ เลยครับ 👉 ใช่ ต้องรีบูตเครื่องตอนนี้ ✅

และทำตามลำดับนี้นะ อย่าข้าม 📝

✅ สิ่งที่ต้องทำตอนนี้ (ลำดับปลอดภัย)

1 กด Reboot / Restart จากตัว Installer

(ถ้ายังอยู่ในหน้าต่างติดตั้ง กด Finish → Reboot)

2 ดึง USB Installer ออกทันที

- สำคัญมาก !
- ถ้าไม่ดึง → มีสิทธิ์บูตกลับเข้า Live อีก

3 ปลอ่ยให้เครื่องบูตเอง

- ถ้าทุกอย่างถูกต้อง:
 - จะ ไม่กลับเข้า BIOS
 - จะเห็น Ubuntu Server login หรือหน้าจอบูตปกติ

ถ้ารีบูตแล้ว “อาการที่ควรเป็น”

- ไม่มีข้อความ error เกี่ยวกับ GRUB
- ไม่วนกลับ BIOS
- RAID /dev/md0 ถูก detect อัตโนมัติ
- เข้าหน้า login ได้

ถ้าเข้าได้ → ถือว่า ผ่านด่านนรกแล้ว 🎉

หลังบูตเข้าได้ (ขั้นตรวจสอบเร็ว ๆ)

ล็อกอินแล้วรัน:

```
lsblk
```

```
cat /proc/mdstat
```

ควรเห็น:

- /dev/md0 = raid0 active
- / อยู่บน md0

ถ้าเกิดกรณีใดกรณีหนึ่ง (ไม่ต้องตกใจ)

- บูตเข้า BIOS
- หน้าจอดำ
- ขึ้น GRUB rescue

หยุดทันที แล้วถ่ายรูปลงมา

เดี๋ยวผมแก้ต่อให้แบบไม่พังข้อมูล

สรุปอีกครั้งแบบมั่นใจ:

● รีบูตได้เลย ดึง USB ออก แล้วลุยต่อ

อีกไม่กี่ก้าวก็ได้ระบบ RAID NVMe พร้อมใช้งานแล้ว 🙌 😁

เข้ามาในไบออสอีกแล้ว

โอเค ภาพนี้ตอบชัดมากครับ 👍

ยังไม่พัง แต่ BIOS ยัง “ไม่รู้ว่าบูตอะไร” เลยดึงกลับมาหน้านี้อีกรอบ

อันนี้เป็นเคส คลาสสิก หลังลง Ubuntu + mdadm RAID เสร็จใหม่ ๆ

ผมขอสรุปสาเหตุ → วิธีแก้ → ทำตามทีละข้อแบบไม่เดานะครับ

วิเคราะห์จากภาพ BIOS (สำคัญมาก)

จากหน้าจอ MSI Click BIOS X

- เห็น NVMe ครบ:
 - M2_1 : Samsung SSD 990 (2000GB)
 - M2_2 : Samsung SSD 990 (2000GB)
- ❌ ยังไม่เห็นรายการบูตชื่อ “ubuntu”
- BIOS เลย ไม่มี Boot Target → กลับ BIOS ทุกครั้ง

สรุปสั้น ๆ

👉 GRUB ยังไม่ได้ถูก register เข้า UEFI NVRAM

วิธีแก้ (ปลอดภัย 100% สำหรับข้อมูล)

เราต้อง เข้า Ubuntu Live อีกครั้ง เพื่อ “ติดตั้ง GRUB ลง EFI ใหม่ให้ถูกต้อง”

ขั้นตอนที่ 1 : บูตเข้า USB Ubuntu (อีกครั้ง)

1. เสียบ USB Installer
2. เปิดเครื่อง
3. กด F11 (Boot Menu ของ MSI)
4. เลือก

👉 UEFI: USB (อย่าเลือก Legacy)

5. เลือก Try Ubuntu (ไม่ใช่ Install)

ขั้นตอนที่ 2 : เข้า Live แล้วรันคำสั่ง “กู้ GRUB”

เปิด Terminal แล้วทำ ตามนี้เป๊ะ ๆ

2.1 ตรวจสอบว่า RAID ยังอยู่

lsblk

cat /proc/mdstat

ต้องเห็น:

- /dev/md0
- มี md0p1 (EFI ~512MB)
- มี md0p2 / md0p3

ถ้าเห็น → ไปต่อได้ทันที 👍

2.2 mount ระบบที่ลงไว้

sudo mount /dev/md0p2 /mnt

sudo mkdir -p /mnt/boot/efi

sudo mount /dev/md0p1 /mnt/boot/efi

(ถ้า / อยู่ md0p3 แทน ให้บอกผม เดี่ยวปรับคำสั่ง)

2.3 bind system dirs

for i in /dev /dev/pts /proc /sys /run; do sudo mount --bind \$i /mnt\$i; done

2.4 chroot เข้า Ubuntu จริง

sudo chroot /mnt

พร้อมที่จะเปลี่ยน (สำคัญมาก)

2.5 ติดตั้ง GRUB ลง UEFI ใหม่ (จุดชี้ขาด)

grub-install --target=x86_64-efi --efi-directory=/boot/efi --bootloader-id=ubuntu --recheck

update-grub

ถ้า **ไม่มี error** = ผ่าน 🎯

2.6 ออก + รีบูต

exit

sudo reboot

! ดึง USB ออกทันที



สิ่งที่ควรเกิดหลังจากนี้

- BIOS จะมี Boot Option ชื่อ **ubuntu**
- เครื่อง **ไม่**กลับ BIOS
- เข้าหน้า Ubuntu Server / Login ได้



ถ้ายังแต่ง BIOS อีก

ถ้าयरูป หน้า Boot / Boot Priority ใน BIOS ส่งมา
ผมจะบอกให้เลือกทีละคลิก (เมนู MSI ผมคุ้นมาก)

สรุปสั้นแบบมั่นใจ

- RAID ทำถูก ✓
- Partition ถูก ✓
- พังตรงเดียวคือ GRUB ยังไม่ register
- แก้ได้แน่นอน 100% ไม่มีลงใหม่

ใจเย็นครับ ด้านสุดท้ายแล้ว 🙏

ทำตาม Step 2 แล้วรายงานผลได้เลย

เยี่ยมมากครับ 🎯 ภาพนี้คือ จุดเฉลยแล้ว
ตอนนี้ทุกอย่าง “ถูกต้อง” แล้ว เหลือแค่เลือกให้ถูกเท่านั้น

📌 หน้าจอที่เห็นคืออะไร

นี่คือ GRUB ของ USB Installer (ไม่ใช่ของระบบที่ติดตั้งลงดิสก์แล้ว)

สังเกตได้จากเมนู:

- Try or Install Ubuntu
 - Ubuntu (safe graphics)
 - Boot from next volume ☒ ← ตัวนี้สำคัญ
 - UEFI Firmware Settings
-

☒ ให้ทำแบบนี้ตอนนี้เลย

👉 เลือก

Boot from next volume

แล้วกด Enter

เหตุผล:

- มันจะ “ข้าม USB”
 - แล้วไปบูตจาก Ubuntu ที่ติดตั้งอยู่บน RAID (/dev/md0)
-

⚠️ สิ่งที่ไม่ควร เลือก

- ❌ Try or Install Ubuntu → จะวนกลับเข้า Live USB อีก
 - ❌ Ubuntu (safe graphics) → ก็ยังเป็น USB
 - ❌ UEFI Firmware Settings → กลับ BIOS
-

🧠 ถ้าบูตผ่านจริง (ผลลัพธ์ที่ถูกต้อง)

หลังจากเลือก Boot from next volume:

- จะเห็น GRUB ของ Ubuntu จริง หรือ
- เข้าหน้า Ubuntu Server login / setup ทันที

ถ้าเข้าได้ = 🎉 จบภารกิจ 100%

🔧 หลังเข้า Ubuntu ได้แล้ว (แนะนำทำต่อ)

เพื่อไม่ให้เครื่องสับสนในอนาคต:

1. เข้า Ubuntu ให้ได้ก่อน
2. ตรวจสอบว่า GRUB ถูกลงจริง:
3. `sudo efibootmgr -v`
4. ตั้งค่า Boot Order (ถ้าจำเป็น)

เดี๋ยวผมช่วยไล่ต่อได้

สรุปสั้นที่สุด

👉 เลือก Boot from next volume แล้ว Enter ได้เลย

นี่คือก้าวสุดท้ายจริง ๆ แล้วครับ 🙏

กตแล้วถ่ายรูปผลลัพธ์ถัดไปมาได้เลย ผมรออยู่ตรงนี้ครับ

ก็เข้ามาใน bios อยู่ดี

โอเคครับ คราวนี้ วิเคราะห์ได้ชัดแล้ว และขอสรุปตรง ๆ แบบไม่อ้อมน่ะครับ 😊

อาการนี้ ไม่ใช่ RAID พัง ไม่ใช่ mdadm ผิด

แต่คือ 👉 UEFI ไม่มี Boot Entry ของ Ubuntu อยู่เลย

พุดง่าย ๆ คือ

ติดตั้งเสร็จ ✔️

ดิสก์ + RAID ถูกต้อง ✔️

แต่ ไม่มี EFI bootloader ให้ BIOS เลือก ❌

เลยต้องกลับ BIOS ทุกครั้ง

🔍 หลักฐานจากภาพที่คุณส่ง

- BIOS เห็น SSD ทั้งสองลูก ✔️ (M.2_1 / M.2_2)
- แต่ ไม่มี Ubuntu / GRUB / Linux Boot Manager ใน Boot Priority
- BIOS จึง “ไม่รู้จะบูตอะไร” → เข้า BIOS อัตโนมัติ

นี่เป็นเคส UEFI + RAID0 + Installer พลาดลง EFI

เจอบ่อยมาก และแก้ได้ครับ 👍

✅ ทางแก้ที่ “เร็วและชัวร์ที่สุด” (แนะนำ)

👉 บูตเข้า Live USB อีกครั้ง แล้ว ซ่อม EFI/GRUB ด้วยมือ

ไม่ต้องลงใหม่ ไม่ต้องลบ RAID

🔧 ขั้นตอนทีละข้อ (ทำตามลำดับ หยุดตรงไหนถามได้ทันที)

1 ที่หน้า GRUB ของ USB

เลือก:

Try or Install Ubuntu

แล้วเข้า Live อีกครั้ง

2 เปิด Terminal แล้วเช็คก่อน

lsblk

ต้องเห็นประมาณนี้:

- /dev/md0p1 → vfat (ESP ~512MB)
- /dev/md0p2 → ext4 (root ~200GB หรือมากกว่า)

ถ้าเห็น → ไปต่อ

ถ้าไม่เห็น → บอกผมทันที

3 mount ระบบที่ติดตั้งไว้

```
sudo mount /dev/md0p2 /mnt
```

```
sudo mkdir -p /mnt/boot/efi
```

```
sudo mount /dev/md0p1 /mnt/boot/efi
```

4 bind system dirs

```
for i in /dev /dev/pts /proc /sys /run; do
```

```
    sudo mount --bind $i /mnt$i
```

```
done
```

5 เข้า chroot

```
sudo chroot /mnt
```

ถ้า prompt เปลี่ยน = เข้าสำเร็จ 

6 ติดตั้ง GRUB ใหม่ “แบบบังคับ UEFI”

```
grub-install \
```

```
--target=x86_64-efi \
```

```
--efi-directory=/boot/efi \
```

```
--bootloader-id=ubuntu \
```

```
--recheck \
```

```
--no-nvram
```

แล้วสร้าง config:

update-grub

7 ออก + unmount

exit

sudo umount -R /mnt

8 รีบูต + ดึง USB ออก

sudo reboot

ผลลัพธ์ที่ควรได้

- BIOS จะเห็น ubuntu หรือ Linux Boot Manager
 - เครื่องจะไม่เด้งเข้า BIOS อีก
 - บูตเข้า Ubuntu Server ได้ทันที
-

ถ้าอยาก “ล็อกให้แน่น” เพิ่ม

หลังบูตเข้าได้แล้ว:

sudo efibootmgr -v

จะเห็น Boot000x: ubuntu

และสามารถจัดลำดับ boot ได้

สรุปตรง ๆ แบบเพื่อนช่วยเพื่อน

-  ไม่ต้องลงใหม่
-  ไม่ต้องยุ่ง RAID แล้ว
-  แค่ ซ่อม EFI/GRUB
- เคสนี้แก้ได้ 100%

ถ้าพร้อมแล้ว เริ่มที่ข้อ 1 ได้เลย

หรือถ้าอยากให้ผม “ดูไปพร้อมกัน” ส่งรูป lsblk หลังเข้า Live มาได้ครับ ผมจะไล่ให้แบบไม่พลาด 

ต้องตรวจสอบการตั้งค่าใน bios ใหม่

คำตอบสั้น ๆ ก่อนเลยนะครับ: **ควรตรวจสอบครับ** — แต่ “ตรวจแค่นี้ก็จุดสำคัญ” เท่านั้น

ไม่ต้องไปปรับมั่ว 👍

จากอาการที่คุณเจอ (ติดตั้งเสร็จ แต่เด้งเข้า BIOS) → จุดชี้ชัดคือ UEFI หา bootloader ไม่เจอ
งั้น BIOS ต้อง “เอื้อ” ให้ GRUB/Ubuntu บูตได้ก่อน

✅ รายการ BIOS ที่ต้องตรวจ (เรียงลำดับความสำคัญ)

เข้า BIOS → Advanced (F7)

1 Boot Mode = UEFI ONLY

เมนู:

Boot → Boot Mode Select

- ✅ UEFI
- ❌ Legacy / CSM / Legacy+UEFI → ห้ามใช้

ถ้าเห็นคำว่า CSM → ปิดให้หมด

2 Secure Boot = Disabled

เมนู:

Boot → Secure Boot

- Secure Boot: **Disabled**
- Secure Boot Mode: *Standard* (หรืออะไรก็ได้ ถ้า Disabled)

RAID + GRUB + manual install → Secure Boot เปิด = พัง 90%

3 RAID Mode (สำคัญมาก)

เมนู (MSI มักอยู่ที่):

Advanced → Integrated Peripherals → SATA Mode / NVMe Mode

- ต้องเป็น **RAID**
- ❌ AHCI = จะบูต md0 ไม่ได้

(จากรูปก่อนหน้านี้ คุณตั้งถูกแล้ว ✅)

Boot Priority (ตอนนี้ยังไม่ต้องซีเรียส)

 เมนู:

Boot → Boot Priority

ตอนนี้ ยังไม่เห็น ubuntu เป็นเรื่องปกติ

เพราะเรายังต้อง “ซ่อม GRUB” ก่อน

 แค่ว่า:

- USB อยู่ลำดับแรก (ตอนใช้ Live)
- หลังซ่อมแล้ว ค่อยกลับมาดูใหม่

Fast Boot = Disabled (แนะนำ)

 เมนู:

Boot → Fast Boot

- ตั้ง **Disabled** ชั่วคราว
- เพื่อให้ BIOS scan EFI device ครบ

สิ่ง “ไม่ต้องไปยุ่ง”

-  CPU / Overclock
-  TPM / fTPM
-  VT-x / Virtualization
-  M.2 configuration (ถ้า BIOS เห็น SSD แล้ว)

สรุปให้จำง่าย

ตอนนี้ BIOS ต้องเป็นแบบนี้:

รายการ	ค่า
Boot Mode	 UEFI
Secure Boot	 Disabled

รายการ	ค่า
RAID Mode	 RAID
Fast Boot	 Disabled
SSD เห็นใน BIOS	 เห็นทั้ง 2 ลูก

ถ้า 5 ข้อนี้ถูก → BIOS พร้อมแล้ว

ปัญหาที่เหลือคือ GRUB/EFI ใน Ubuntu (ซึ่งเรากำลังจะแก้)

ขั้นตอนถัดไป (สำคัญ)

หลังเช็ค BIOS เสร็จแล้ว:

1. Save & Exit
2. บูตเข้า USB
3. ทำขั้นตอน **ซ่อม GRUB/EFI** ที่ผมให้ไปก่อนหน้านี้

ถ้าคุณอยากให้ผม **ไล่ BIOS** ที่ละเมนูจากรูปของ MSI

ถ่ายรูปหน้า Boot กับ Secure Boot มาได้เลย ผมจะชี้จุดให้ตรง ๆ ว่าต้องกดอะไร 

โอเค ภาพนี้ ชัดมาก ขอสรุปตรง ๆ แบบไม่อ้อมน่ะครับ 🙌

ตอนนี้ BIOS เห็น SSD ทั้ง 2 ลูกครบ ✅ แต่ ยังไม่มี UEFI Boot Entry ของ Ubuntu

ดังนั้นเครื่องเลย ต้องกลับเข้า BIOS ทุกครั้ง — อันนี้ “ปกติ” สำหรับสถานการณ์นี้ครับ ไม่ได้พังเพิ่ม 😊

🔍 วิเคราะห์จากภาพนี้ (MSI Click BIOS X)

สิ่งที่ “ถูกแล้ว”:

- ✅ เห็น Samsung SSD 99 (2000.3GB) ทั้ง M2_1 และ M2_2
- ✅ BIOS เป็นโหมด Advanced (F7)
- ✅ ไม่ใช่ปัญหาฮาร์ดแวร์

สิ่งที่ “ยังขาด”:

- ❌ ไม่มี Ubuntu / ubuntu / grub / EFI entry ใน Boot Priority
- BIOS เลยไม่รู้จะบูตอะไร → กลับมานี่

สรุป: BIOS ยังไม่เจอ bootloader (GRUB) ใน EFI

✅ สิ่งที่ต้องตั้งค่าใน BIOS (เช็คตามลำดับนี้)

เข้า BIOS → Advanced (F7)

1 Boot → Boot Mode Select

ต้องเป็นแบบนี้เท่านั้น 🙌

- ✅ UEFI
- ❌ Legacy / CSM / Legacy+UEFI

ถ้าเห็นคำว่า CSM → ปิดให้หมด

2 Boot → Secure Boot

- Secure Boot = Disabled
- Secure Boot Mode = Standard (หรืออะไรก็ได้ ถ้า Disabled)

RAID + mdadm + manual install

ถ้า Secure Boot เปิด = GRUB ไม่ถูกเรียก

3 Advanced → Integrated Peripherals / Storage

(ชื่อเมนูอาจต่างเล็กน้อยใน MSI)

- SATA / NVMe Mode = RAID
- ❌ AHCI = บูต md0 ไม่ได้

จากขั้นตอนก่อนหน้านี้ คุณตั้ง RAID ถูกแล้ว ✓
แค่เช็คซ้ำให้มั่นใจ

4 Boot → Fast Boot

- ตั้ง Disabled (ชั่วคราว)

เหตุผล:

ให้ BIOS scan EFI device ทุกตัวก่อน ไม่ข้าม

5 Boot Priority (ตอนนี้ยัง “ไม่ต้องตกใจ”)

จากภาพ:

- ยังไม่เห็น Ubuntu → ปกติ
- เพราะ GRUB ยังไม่ถูก register เข้า UEFI

ตรงนี้ ยังไม่ต้องแก้ใน BIOS

🔧 สรุปสถานะตอนนี้

- BIOS ✓ พร้อมแล้ว
- SSD ✓ พร้อมแล้ว
- RAID ✓ ทำงานแล้ว
- ❌ ขาดอย่างเดียว: EFI bootloader (GRUB) ยังไม่ถูกเขียน/ผูกกับ UEFI

👉 คำตอบคำถามคุณตรง ๆ

“ต้องตรวจสอบการตั้งค่าใน BIOS ใหม่”

✓ ตรวจสอบแล้ว → ไม่มีอะไรผิดปกติ

✗ แต่ BIOS “แก้ไม่ได้” แล้วในขั้นนี้

ขั้นตอนต่อไปต้องทำใน Ubuntu Live USB เท่านั้น

🔧 ขั้นตอนถัดไป (สำคัญมาก)

1. เสียบ USB Ubuntu
2. เปิดเครื่อง
3. ที่หน้า GRUB เลือก

👉 Try or Install Ubuntu

4. เข้า Live Desktop
5. เราจะทำขั้นตอน:
 - `mount /dev/md0p3`
 - `mount EFI (/dev/md0p1)`
 - `grub-install`
 - `update-grub`
 - `efibootmgr` ให้ UEFI เห็น Ubuntu

ถ้าคุณพร้อม

👉 บอกผมว่า **ตอนนี้อยู่ที่ไหน** (GRUB / Live Desktop / Terminal)

ผมจะพาไปที่คำสั่ง แบบไม่หลงอีกแน่นอน 👍

สรุปให้ชัด ๆ แบบไม่อ้อมนะครับ 🙌

ตอนนี้ต้องตรวจ BIOS จริง และมี 2 จุดสำคัญที่ตั้ง “ยังไม่ถูก” เลยทำให้วนเข้า BIOS ตลอด
จากรูปที่ส่งมา ผมเห็นต้นเหตุชัดแล้วครับ

❌ ปัญหาหลักตอนนี้

เครื่องคุณ ยังบูตจาก USB Installer อยู่
ไม่ได้บูตจาก Ubuntu ที่ติดตั้งลง NVMe แล้ว
ใน BIOS:

- Boot Option #1 = USB Key : UEFI (SMI USB DISK) ❌
- ไม่มี ubuntu / Ubuntu / NVMe อยู่เป็นตัวแรก

ผลคือ:

รีบูต → หา USB ก่อน → ถอด USB หรือ USB ไม่ใช่ OS → เข้า BIOS วนลูป

✅ วิธีแก้ (ทำตามลำดับนี้เลย)

1 ถอด USB Installer ออกก่อน (สำคัญมาก)

- ดึงแฟลชไดรฟ์ Ubuntu ออก ก่อนรีบูต
- ไม่งั้น BIOS จะพยายามบูตจาก USB ข้างตลอด

2 เข้า BIOS → เมนู Boot

ไปที่

Advanced (F7) → Boot

ตั้งค่าแบบนี้ 🙌

Boot Option #1

- เปลี่ยนจาก
 - ❌ USB Key : UEFI SMI USB DISK
- เป็น
 - ✅ ubuntu หรือ
 - ✅ UEFI Hard Disk หรือ

✓ Samsung SSD 99 (NVMe)

(ชื่ออาจต่างกันเล็กน้อย)

ถ้าเห็นคำว่า **ubuntu** ให้เลือกอันนั้นก่อนเสมอ 👍

Boot Option #2

- ค่อยเป็น Hard Disk หรืออย่างอื่น

3 ตรวจสอบบูต (ถ้ามีเมนู)

ในหน้า Boot ให้ใช้ว่า

- Boot Mode = UEFI ✓
- CSM = Disabled (ถ้ามี)
- Fast Boot = Disabled (ตอนนี้คุณตั้งถูกแล้ว 👍)

4 Save แล้วออก

- กด F10 → Yes
- ปล่อยให้เครื่องบูตเอง

🔍 ถ้ายังไม่เห็นคำว่า ubuntu ใน Boot Option

แปลว่า GRUB ยังไม่ถูกเขียนเข้า EFI

ให้ทำขั้นตอนที่ GRUB (บอกก่อน ทำได้แน่ ไม่ยาก):

1. เสียบ USB Ubuntu อีกครั้ง
2. เลือก Try Ubuntu
3. เปิด Terminal แล้วรัน

```
sudo grub-install /dev/md0
```

```
sudo update-grub
```

หรือถ้าใช้ EFI ชัวร์ ๆ

```
sudo grub-install --target=x86_64-efi --efi-directory=/boot/efi --bootloader-id=ubuntu
```

```
sudo update-grub
```

4. รีบูต → ถอด USB → เข้า BIOS → จะเห็น ubuntu
-

✅ สรุปสั้นที่สุด

- ✓ ต้องตรวจ BIOS
- ✓ ต้องเปลี่ยน Boot Option #1
- ✗ ตอนนี้บูตผิดตัว (ยังชี้ไป USB)
- ✓ Ubuntu ที่ลง ยังอยู่ครบ ไม่ต้องลงใหม่

ถ้าคุณอยากให้ผม ไล่ทีละคลิกแบบ real-time

บอกผมได้เลยว่า ตอนนี้ BIOS หน้าไหนอยู่ ผมจะพาไปต่อทันที 😊

โอเค ภาพนี้ ตอบได้ชัดมากแล้วครับ 👍

ข่าวดีคือ คุณตั้งเกือบถูกแล้ว เหลือ “จุดสุดท้าย” เท่านั้น

✅ สถานะปัจจุบันจากภาพ

ตอนนี้ BIOS ของคุณเป็นแบบนี้:

- Boot Option #1 = Hard Disk ✅ (ถูกทิศแล้ว)
- USB ไม่ได้ยู่ลำดับแรกแล้ว ✅
- Fast Boot / MSI Fast Boot = Disabled ✅

แต่...

ยังไม่จบ เพราะใน MSI BIOS คำว่า Hard Disk = ยังไม่ชี้ว่าบูต “ตัวไหน”

🔴 จุดสำคัญที่ยังต้องตั้งเพิ่ม (มักพลาดกันตรงนี้)

ให้ดูเมนูนี้ 👉

👉 UEFI Hard Disk Drive BBS Priorities

(หรือชื่อใกล้เคียง)

เมนูนี้คือที่ “เลือกดีสก์จริง ๆ” ว่า Hard Disk หมายถึงอะไร

🔧 ทำตามนี้ทีละขั้น (สั้น ๆ แต่ชัวร์)

1 กด Enter เข้า

UEFI Hard Disk Drive BBS Priorities

2 ตั้งค่า

- Boot Option #1

👉 เลือก ubuntu

หรือ

👉 Samsung SSD 99 (NVMe)

หรือ

👉 UEFI OS (Samsung ...)

! ถ้าเห็นคำว่า ubuntu ให้เลือกอันนั้นทันที (ดีที่สุด)

3 กด F10 → Yes

 จากนั้น

1. ถอด USB ออก (ถ้ายังเสียบอยู่)
 2. ปลดปล่อยให้เครื่องบูตเอง
-

 ผลที่ควรได้

- ไม่เข้า BIOS แล้ว
 - ไม่เห็นเมนู “Try or Install Ubuntu”
 - เข้าหน้า Ubuntu ที่ติดตั้งจริงบน NVMe
-

 ถ้าเข้า BBS แล้ว “ไม่เห็น ubuntu”

ไม่ต้องตกใจนะครับ แปลว่า:

- GRUB ยังไม่ถูกเขียนเข้า EFI

 บอกผมทันที ผมจะให้คำสั่ง แก้ GRUB แบบจบใน 5 นาที

(ไม่ต้องลงใหม่แน่นอน)

ตอนนี้ขอคำตอบสั้น ๆ จากคุณหน่อยครับ 😊

 ในเมนู UEFI Hard Disk Drive BBS Priorities เห็นคำว่า “ubuntu” ไหม?

สรุปให้ตรงประเด็นเลยนะครับตอนนี้ 🙌

ต้องตรวจ BIOS จริง และจากภาพนี้ “เจอสาเหตุแล้ว” 💡

❌ สาเหตุที่เครื่องวนเข้า BIOS ตลอด

ในเครื่องคุณ เปิด Intel Rapid Storage Technology (Intel RST / RAID Mode) อยู่

แต่ Ubuntu ที่ติดตั้ง = แบบ mdadm / software RAID (md0)

➡ ผลคือ

- BIOS เห็นดิสก์
- แต่ UEFI ไม่เห็น EFI bootloader ของ Ubuntu
- เลยไม่มี ubuntu ให้เลือก → วนกลับเข้า BIOS ทุกครั้ง

✅ ทางแก้ที่ “ถูกต้องที่สุด” (ไม่ต้องลงใหม่)

เปลี่ยนโหมด Storage จาก RAID → AHCI

Ubuntu + mdadm ไม่ควรใช้ Intel RST

🔧 ทำตามนี้ทีละข้อ (สำคัญมาก)

1 BIOS → Advanced

เข้าเมนู:

Advanced

└─ Integrated Peripherals

└─ SATA Mode / Storage Mode

2 เปลี่ยนค่า

- ❌ RAID / Intel RST
- ✅ AHCI

3 กด F10 → Save & Exit

🔄 หลังจากเปลี่ยนเป็น AHCI

เครื่องจะรีบูต → มี 2 กรณี

✅ กรณีที่ 1 (โชคดี)

- เห็นเมนู ubuntu
- บูตเข้า Ubuntu ได้เลย 🎉

⚠️ กรณีที่ 2 (พบบ่อย แต่แก้จบ)

- ยังไม่ขึ้น ubuntu
- แต่บูต USB ได้

👉 แบบนี้เราจะ ซ่อม GRUB 1 รอบ = จบ

ผมจะให้คำสั่งประมาณนี้:

```
sudo mount /dev/md0p2 /mnt
```

```
sudo mount /dev/md0p1 /mnt/boot/efi
```

```
sudo grub-install --target=x86_64-efi --efi-directory=/mnt/boot/efi --bootloader-id=ubuntu
```

```
sudo update-grub
```

(ยังไม่ต้องทำ เดี่ยวผมปรับให้ตรง 100% ตามสถานการณ์จริง)

❗ ห้ามทำตอนนี้

- ❌ อย่า Secure Erase
- ❌ อย่าลง Ubuntu ใหม่
- ❌ อย่าเปิด RAID กลับ

คำถามสั้น ๆ ตอนนี้ (ตอบข้อเดียวพอครับ)

👉 คุณสามารถเปลี่ยน Storage / SATA Mode เป็น AHCI ได้ไหม หรือ BIOS ล็อกไว้?

ตอบมาปั๊บ ผมพาคุณ “ออกจาก BIOS ให้ได้วันนี้” แน่نون 😊

เยี่ยมมากครับ 👍

ภาพนี้คือ จุดสำคัญที่สุดของเรื่องทั้งหมด — และข่าวดีคือ แก้ได้ตรงนี้เลย ไม่ต้องลงใหม่ 🎯

🔴 ต้นเหตุที่แท้จริง (ยืนยันแล้วจากภาพ)

ตอนนี้ใน BIOS ของคุณ

Enable VMD controller = Enabled

นั่นแปลว่า

- เครื่องใช้ Intel VMD / Intel RST
- Ubuntu ที่คุณติดตั้งด้วย mdadm (software RAID)

👉 UEFI จะมองไม่เห็น bootloader

👉 เครื่องเลย วนเข้า BIOS ตลอด

✅ วิธีแก้ (ทำตรงนี้เลย)

🔧 ขั้นตอนที่ 1: ปิด VMD

ในหน้าจอที่คุณอยู่ตอนนี้

เปลี่ยนค่า:

Enable VMD controller

จาก: Enabled

เป็น: Disabled

⚠️ ค่าอื่น ไม่ต้องแตะ

💾 ขั้นตอนที่ 2: Save & Exit

- กด F10
- เลือก Yes

🔄 หลังจากรีบูต จะเกิดอะไรขึ้น?

กรณีที่ 1 (ดีที่สุด 🎉)

- เครื่องบูตเข้า Ubuntu ได้ทันที

- เรืองจบ → ดื่มกาแฟได้ ☕ 😊

กรณีที่ 2 (ยังไม่ขึ้น แต่แก้จบง่าย)

- ยังไม่เห็น Ubuntu
- แต่ ไม่วนเข้า BIOS แล้ว
- บูต USB Ubuntu ได้

👉 แบบนี้เราจะ ซ่อม GRUB 1 รอบ = จบสนิท

! สำคัญมาก (ห้ามพลาด)

- ❌ อย่าเปิด VMD กลับ
- ❌ อย่า Secure Erase
- ❌ อย่า Enable RAID
- ❌ อย่า Erase disk / Install ใหม่

ถามสั้น ๆ หลังคุณกด F10

👉 หลัง Disable VMD แล้ว เครื่องบูตเข้าอะไร?

- เข้า Ubuntu เลย
- หรือยังไม่เข้า แต่ไม่เด้ง BIOS แล้ว
- หรือมี error อื่นขึ้นมา

บอกผมมา 1 ประโยค

ผมพาคุณ “ออกจาก BIOS อย่างถาวร” ให้ได้แน่นอนครับ 🙏 😊

สมบูรณ์แบบมากครับ 🎯

ภาพนี้คือ คำยืนยันว่าคุณตั้งค่าถูกต้อง 100% แล้ว

จากหน้าจอ **Save configuration and exit?** ผมขอสรุปให้ชัด ๆ ก่อนกดนะครับ 🙌

✅ สิ่งที่ต้องตั้งแล้ว (สำคัญมาก)

จากข้อความในกรอบ:

- ✅ **Enable VMD controller: Enabled → Disabled**
👉 นี่คือ “กุญแจหลัก” ที่แก้ปัญหาลoop BIOS
- ✅ **Boot Option #1: USB Key → Hard Disk**
👉 เครื่องจะพยายามบูตจากดิสก์ที่ลง Ubuntu แล้วก่อน
- ✅ **Boot Option #2: Hard Disk → USB Key**
👉 USB เป็นตัวสำรอง (ถูกต้อง)

ทุกอย่างตรงตามที่ควรเป็น เป๊ะ 🙌

👉 ขั้นตอนถัดไป (ทำตามลำดับนี้เลย)

1 กด Yes

ให้ BIOS:

- บันทึกค่า
- รีบูตเครื่อง

2 ระหว่างเครื่องรีสตาร์ท

● กด USB Installer ออกทันที

(สำคัญมาก อย่าปล่อยคาไว้)

💡 สิ่งที่จะเกิดขึ้นหลังจากนี้

กรณีที่คาดหวังที่สุด 🙌

- เห็น GRUB ของ Ubuntu

- หรือบูตเข้า Ubuntu ตรง ๆ

 = จบภารกิจ 

ถ้า “ยังไม่เข้า Ubuntu” แต่...

-  ไม่เด้งกลับ BIOS แล้ว
-  ไม่เห็น Try / Install Ubuntu

 บอกผมทันที

เราจะ ซ่อม GRUB อีก 1 ขั้นตอน = จบแน่นอน

 ตอนนี้ยัง “ห้าม” ทำสิ่งเหล่านี้

- อย่า Install ใหม่
- อย่า Erase disk
- อย่าเปิด VMD กลับ
- อย่าเปิด RAID

ไปเลยครับ 

 กด Yes → ถอด USB → ดูว่าหน้าจออะไรขึ้น

แล้วบอกผมมา 1 บรรทัด ผมอยู่ตรงนี้นั่นกว่าจะบูตได้จริง 

สรุปสั้น ๆ แต่ตรงจุดนะครับ: ใช่ครับ ต้องตั้งค่าใน BIOS และตอนนี้คุณทำ “ถูกทางแล้วเกือบครบ” เหลืออีกนิดเดียว 👍

อาการที่ รีบูตแล้วเด็งกลับเข้า BIOS แปลว่า BIOS ยังหา bootloader (Ubuntu) ไม่เจอ แม้ระบบจะติดตั้งไปแล้ว

จากรูปที่คุณส่งมา ผมขอสรุปสถานะก่อน แล้วบอกขั้นตอนสุดท้ายให้เลย

สถานะปัจจุบัน (จากรูป)

- ✅ NVMe ทั้ง 2 ลูก ถูกเห็นแล้ว
- ✅ คุณ ปิด VMD controller แล้ว ← อันนี้สำคัญมาก (ทำถูกต้อง)
- ✅ Boot Option #1 = Hard Disk
- ❌ แต่ ยังไม่มี “Ubuntu / UEFI OS / NVMe Boot Entry” ให้เลือก

แปลว่า:

Ubuntu ติดตั้งลงดิสก์แล้ว แต่ UEFI boot entry ยังไม่ถูก register

สิ่งที่ต้องตรวจ + ทำต่อ (เรียงตามลำดับ ห้ามข้าม)

1 เช็กโหมดบูต (สำคัญมาก)

ไปที่

Boot → Boot Mode Select

ต้องเป็น:

- ✅ UEFI
- ❌ ไม่ใช่ Legacy / CSM

ถ้าเห็น CSM → ปิด CSM

2 ดูรายการ Boot Option ให้ละเอียด

ในหน้า Boot Option #1

ถ้า ยังเห็นแค่

- Hard Disk
- USB Key

- USB Hard Disk

 แปลว่ายังไม่มี UEFI entry ของ Ubuntu

ปกติควรเห็นชื่อประมาณ

ubuntu

UEFI OS (Samsung SSD ...)

ใช้วิธีลัด (แนะนำมาก)

ตอนเปิดเครื่องใหม่ กดปุ่ม:

- F11 (MSI Boot Menu)

ดูว่าในเมนูนี้มีอะไรโผล่มาบ้าง:

- ถ้าเห็น ubuntu หรือ UEFI OS → เลือกอันนั้นทันที
- ถ้า ไม่มีเลย → ไปข้อ 4

ทางแก้ที่ชัวร์ 100% (ทำครั้งเดียวจบ)

เพราะคุณ:

- ใช้ RAID / mdadm มาก่อน
- ปิด VMD ที่หลัง
- Installer มีโอกาส register boot ไม่ครบ

ให้ทำแบบนี้ครับ 

บูตเข้า USB Ubuntu อีกครั้ง

เลือก:

 Try Ubuntu

เปิด Terminal แล้วพิมพ์:

```
sudo mount /dev/md0p2 /mnt
```

```
sudo mount /dev/md0p1 /mnt/boot/efi
```

```
sudo grub-install --target=x86_64-efi --efi-directory=/mnt/boot/efi --bootloader-id=ubuntu --
```

```
recheck
```

```
sudo chroot /mnt update-grub
```

เสร็จแล้ว:

sudo reboot

 ถ้าคำสั่งไหน error บอกผมได้ทันที เดี่ยวไล่แก้ให้เป็นบรรทัด ๆ

5 หลังรีบูต

กลับเข้า BIOS อีกครั้ง:

- Boot Option #1 → ควรเห็น ubuntu
- ตั้งให้ ubuntu เป็นอันดับ 1
- Save & Exit

จากนั้นควรเข้าระบบ Ubuntu ได้ทันที 🎉

สรุปแบบตรงไปตรงมา

- ❌ เครื่องไม่ได้พัง
- ❌ ไม่ต้องลงใหม่
- ✅ VMD ปิดถูกแล้ว
- ! ขาดแค่ “UEFI boot entry”

ถ้าคุณอยากให้ผม:

- เช็คว่าพาร์ทิชันไหนควรเป็น // /boot/efi
- หรืออยากจัด layout RAID + UEFI ให้สะอาด ๆ รอบเดียวจบ

บอกได้เลยครับ เดี่ยวพาไปจนเห็นหน้า login ให้ได้ 😊

ไม่มีเมนูefi

โอเคครับ ภาพนี้ ชัดเจนมาก และช่วยตัดประเด็นได้เลย 👍

สรุปตรง ๆ ก่อนนะครับ:

❗ ไม่มีเมนู EFI = เครื่องยัง “ไม่เห็น bootloader ของ Ubuntu” จริง ๆ
และนี่ ไม่ใช่ความผิดที่ BIOS แต่เป็นเรื่อง UEFI boot entry ยังไม่ถูกสร้าง

คุณตั้งค่า BIOS ถูกแล้วแทบทุกอย่าง 🙌

- ✅ VMD = Disabled (ถูกต้อง)
- ✅ Boot Mode เป็น UEFI (จากบริษัท MSI รุ่นนี้)
- ❌ แต่ ไม่มีรายการ ubuntu / UEFI OS ให้เลือก

ดังนั้น ต่อให้ตั้ง Boot Option #1 เป็น Hard Disk ก็ไม่มีอะไรให้บูต → เครื่องเลยเต็งกลับ BIOS ทุกครั้ง

สาเหตุที่แท้จริง (ฟันธง)

จากลำดับเหตุการณ์ทั้งหมด:

- ใช้ NVMe 2 ลูก
- เคยเปิด VMD / RAID
- สร้าง mdadm (/dev/md0)
- Installer Ubuntu ติดตั้งไฟล์ระบบลงดิสก์แล้ว
- ❌ แต่ UEFI bootloader (GRUB EFI) ไม่ได้ถูก register

อาการนี้พบบ่อยมากกับ Intel VMD + RAID + Linux

ทางแก้ที่ถูกต้อง (ไม่ต้องลงใหม่ ❌)

ขั้นตอนเดียวที่ต้องทำตอนนี้

👉 บูตเข้า USB Ubuntu อีกครั้ง แล้วซ่อม EFI ด้วยมือ

1 บูตจาก USB

ตอนเปิดเครื่อง กด F11

เลือก:

UEFI: SMI USB DISK

แล้วเลือก Try Ubuntu

2 เปิด Terminal แล้วพิมพ์ (ทีละบรรทัด)

จากรูปก่อนหน้านี้ โครงสร้างคุณคือ

- /dev/md0p1 = EFI (vfat ~512MB)
- /dev/md0p2 = root (/)

```
sudo mount /dev/md0p2 /mnt
```

```
sudo mount /dev/md0p1 /mnt/boot/efi
```

ใช้คำว่า mount ถูกไหม:

```
ls /mnt/boot/efi
```

ถ้าเห็นโฟลเดอร์ EFI แปลว่ามาถูกทาง 👍

3 ติดตั้ง GRUB แบบ UEFI (จุดชี้ขาด)

```
sudo grub-install \
```

```
--target=x86_64-efi \
```

```
--efi-directory=/mnt/boot/efi \
```

```
--bootloader-id=ubuntu \
```

```
--recheck \
```

```
--no-nvram
```

แล้ว:

```
sudo chroot /mnt update-grub
```

ไม่มี error = ผ่าน 🎉

4 รีบูต

```
sudo reboot
```

ถอด USB ออกทันที ก่อนเครื่องบูตใหม่

หลังจากนั้นจะเกิดอะไรขึ้น

1. BIOS จะ สร้างเมนูใหม่อัตโนมัติ
2. ใน Boot Option จะเห็น:

- ubuntu
 - หรือ UEFI OS (Samsung SSD ...)
3. เครื่องจะไม่เด้งเข้า BIOS อีก ❌
4. จะเข้าหน้า GRUB / Ubuntu ทันที ✅

ถ้ายังไม่ขึ้น (กรณีต้อจริง ๆ)

บอกผมได้เลย:

- output ของ
- lsblk -f
- หรือถ่ายรูปหน้าจอ Terminal

ผมจะไล่แก้ไขจนเห็นหน้า login แน่จนครับ 😊

ตอนนี้คุณมา 90% แล้ว เหลือแค่ “เขียนชื่อ Ubuntu ลง UEFI” เท่านั้นเอง 🙌

หน้านี้นี้เกี่ยวไหม

สรุปให้ชัด ๆ แบบไม่อ้อมน่ะครับครูใหญ่ 😊

หน้านี้ “ไม่เกี่ยว” กับการบูต EFI เลย และที่เครื่องวนกลับเข้า BIOS เกิดจากสาเหตุหลัก ๆ ดังนี้

คำตอบสั้นก่อน

-  หน้านี้ไม่เกี่ยว → เป็นเมนู Load/Save Profile (โอเวอร์คล็อก/ค่าระบบ)
-  ไม่ใช่เมนูเพิ่ม EFI
-  ไม่ใช่เมนูเลือก GRUB
-  ไม่ได้ช่วยให้ Ubuntu บูตได้

พุดง่าย ๆ: ดูได้ แต่ไม่ต้องยุ่ง 😊

วิเคราะห์จากภาพทั้งหมด (อันนี้สำคัญมาก)

ตอนนี้เครื่องคุณอยู่ในสถานะนี้:

1. ดิสก์ NVMe เห็นจริง

- M2_1 / M2_2 → Samsung SSD 2TB ทั้งคู่
- แปลว่า ฮาร์ดแวร์ปกติ

2. ปิด VMD แล้ว (ถูกต้อง)

- Enable VMD controller = Disabled 

อันนี้ดีมาก ถ้าเปิด Ubuntu จะมองไม่เห็น NVMe

 ปัญหาหลักจริง ๆ อยู่ตรงนี้

 BIOS ไม่เห็นรายการ “ubuntu” หรือ “UEFI OS”

ใน Boot Option มีแค่:

- Hard Disk (รวม ๆ)
- USB Key
- USB CD/DVD
- Network

ไม่มี:

- ubuntu
- UEFI OS
- Windows Boot Manager

👉 แปลตรงตัว:

ใน EFI System Partition (ESP) ไม่มี boot entry ที่ BIOS รู้จัก

ทำไมถึงเป็นแบบนี้?

มีได้ 3 กรณี (เรียงจากพบบ่อย → น้อย)

กรณีที่ 1 (พบบ่อยที่สุด)

❌ ตอนติดตั้ง Ubuntu GRUB ไม่ถูกเขียนลง EFI

สาเหตุย่อย:

- ตอนติดตั้ง USB บูตมาแบบ UEFI ✔️
แต่
- ดิสก์ปลายทาง ไม่มี ESP หรือ ESP เสีย
- หรือ GRUB install fail เจียบ ๆ

กรณีที่ 2

ดิสก์เคยมี OS เก่า / เคยเปิด VMD / RAID มาก่อน

EFI table และ → BIOS ไม่กล้าสร้าง entry

กรณีที่ 3 (น้อย)

Secure Boot / Firmware bug

แต่จากภาพ Secure Boot น่าจะปิดอยู่แล้ว

คำถามที่ครูใหญ่ถามมา:

“ไม่มีเมนู EFI ต้องทำยังไง”

👉 คำตอบตรง ๆ:

BIOS สร้าง EFI ให้เองไม่ได้

ต้องให้ ตัว Installer หรือ GRUB สร้าง

แนวทางที่ “เร็ว + ชัวร์” ที่สุด (ผมแนะนำ)

 ทางลัดสำหรับวันนี้

ลง Ubuntu ใหม่อีกรอบ แบบเคลียร์ ๆ

ทำตามนี้เปะ ๆ จะไม่วน BIOS อีก

ขั้นตอน

1. เสียบ USB Ubuntu
2. เปิดเครื่อง → กด F11
3. เลือก **USB Key (UEFI: ...)** ← ต้องมีคำว่า UEFI
4. เข้า Installer
5. ตอนเลือกดิสก์:
 - เลือก **Erase disk and install Ubuntu**
 - อย่า custom
6. ติดตั้งจนเสร็จ
7. ถอด USB
8. Reboot

 ถ้าทุกอย่างปกติ

Boot Option จะโผล่เป็น **ubuntu** ทันที

ถ้าครูใหญ่อยาก “กู้ ไม่ลงใหม่”

ก็ทำได้ แต่...

- ต้องเข้า Live USB
- mount EFI
- grub-install
- efibootmgr

ซึ่งจากสถานการณ์ตอนนี้

ลงใหม่เร็วกว่า 10 เท่า และเสถียรมากกว่า

ตอบคำถามสุดท้ายแบบฟันธง

หน้านี้เกี่ยวไหม

ไม่เกี่ยวครับ ❌

ปิดได้ ย้อนกลับได้ สัมมันไปได้เลย 😊

ถ้าครูใหญ่โอเค

บอกผมหน่อยว่า:

- จะ ลงใหม่เลย หรือ
- อยาก กู้ GRUB แบบสายลี้ก

ผมจะพาไปที่ละคลิกครับ ไม่ปล่อยให้ BIOS หลอกเราอีก 🤝 😎

ภาพนี้คือ Boot Menu (F11) และมัน “ชี้คำตอบแล้ว” ครับ 👍
ตอนนี้ระบบ เห็น USB แบบ UEFI ถูกต้อง เหลือแค่กดให้ถูกตัว

ให้ทำทันที (ไม่ต้องคิดเยอะ)

👉 เลือกบรรทัดนี้แล้วกด Enter

UEFI: SMI USB DISK 1100, Partition 1 (SMI USB DISK 1100)

ห้ามเลือก Enter Setup

ถ้าทำถูก จะเกิดอะไรต่อ

1. เครื่องจะเข้า GNU GRUB / Ubuntu Installer
 2. จะเห็นเมนู
 - Try or Install Ubuntu
 - Ubuntu (safe graphics)
 3. เลือก Try or Install Ubuntu → Enter
-

ถ้าเข้า Installer ได้แล้ว (สำคัญมาก)

เพื่อจบปัญหา BIOS วนถาวร แนะนำแบบดังนี้:

ตอนติดตั้ง

- เลือก Erase disk and install Ubuntu
- อย่า custom partition
- ให้ระบบสร้าง EFI (ESP) + GRUB เอง

หลังติดตั้งเสร็จ

- รีบูต
 - ถอด USB ออก
 - Boot Option จะโผล่เป็น ubuntu อัตโนมัติ
-

เช็คความมั่นใจ (ตอบคำถามคาใจ)

- เห็นคำว่า UEFI: SMI USB DISK =  มาถูกทาง 100%
- BIOS ไม่เห็น EFI บนดิสก์ = เพราะ ยังไม่ได้ติดตั้ง GRUB ลง NVMe

ไปต่อได้เลยครับครูใหญ่

ถ้า Enter แล้ว ไม่เข้า Installer / ค้าง / จอดำ บอกผมทันที ผมจะพาแก้ไขเฉพาะจุดแบบไม่อ้อม 😊

เยี่ยมมากครับ 🎯 มาถูกด้านสุดท้ายแล้วจริง ๆ

ภาพนี้คือ GNU GRUB ของ Ubuntu Installer ทำงานสมบูรณ์แล้ว

ให้ทำตอนนี้ (เลือกตามอาการเครื่อง)

✅ ทางเลือกที่แนะนำก่อน

👉 เลือก Try or Install Ubuntu แล้วกด Enter

- ถ้าหน้าจอไปต่อ → เข้า Installer ได้เลย
 - นี่คือเส้นทางปกติ 90% ของเครื่องทั่วไป
-

🚫 ถ้ากดแล้ว “จอดำ / ค้าง / สัญญาณภาพหาย”

(โดยเฉพาะเครื่องใหม่ การ์ดจอ / iGPU รุ่นใหม่)

👉 รีบูตใหม่ → กลับมาหน้านี้ → เลือก

Ubuntu (safe graphics)

แล้วกด Enter

safe graphics = ปิดลูกเล่นการ์ดจอชั่วคราว

เอาไว้เข้า Installer ให้ได้ก่อน ค่อยแก้ทีหลัง

หลังจากเข้า Desktop / Installer ได้แล้ว

ให้ทำตามนี้แบบไม่ต้องลังเล:

1. เลือก Install Ubuntu
2. ถามชนิดติดตั้ง → เลือก
Erase disk and install Ubuntu
3. เรื่อง Partition → ปลอ่ยอัตโนมัติ
 - ระบบจะสร้าง EFI + GRUB ให้เอง
4. ติดตั้งจนเสร็จ → รีบูต
5. ถอด USB ออก

ถ้าทุกอย่างถูกต้อง:

- BIOS จะไม่วนอีก
- Boot option จะขึ้นคำว่า ubuntu ทันที 🎉

สรุปสถานะตอนนี้ (สั้น ๆ)

-  USB ถูกต้อง (UEFI)
-  GRUB ทำงาน
-  VMD ปิดแล้ว (ดีมาก)
-  เหลือแค่ “เข้า Installer ให้ได้”

ไปต่อได้เลยครับครูใหญ่

ถ้ากดแล้ว ค้างตรงไหน / จอดำ / รีบูตวน

ถ่ายรูปหน้าจอมาอีกครั้ง ผมจะไล่แก้ไขให้ตรงจุดแบบไม่ให้หลงทางแน่นอนครับ 😊

สุดท้ายขอขอบคุณครูใหญ่ 🌈 ถึงหน้าติดตั้งจริงแล้ว = ผ่านด่าน BIOS/GRUB หมดเรียบร้อยแล้ว
จากจุดนี้คือทางโล่งยาว ๆ แล้วครับ

หน้านี้ให้เลือกแบบนี้เลย ✅

👉 เลือก Interactive installation (ถูกต้องแล้ว)

แล้วกด Next

เหตุผลสั้น ๆ:

- เราต้องการควบคุมทีละขั้น (ดิสก์, ภาษา, user)
 - Automated installation ใช้เฉพาะคนที่มี autoinstall.yaml เตรียมมาแล้ว
-

เดี๋ยวผมบอกล่วงหน้า “ตัวเลือกสำคัญ” ที่จะเจอถัดไป

จะได้ไม่ต้องชะงักคิดกลางทาง 😊

1 Updates & software

- เลือก Normal installation
 - ✓ ตีติ Download updates while installing Ubuntu
 - ✗ จะไม่ติติ third-party ก็ได้ (ค่อยลงทีหลัง)
-

2 Installation type (สำคัญที่สุด)

ถ้าเครื่องนี้ ตั้งใจลง Ubuntu อย่างเดียว:

👉 เลือก

Erase disk and install Ubuntu

ไม่ต้องกลัว

ระบบจะสร้าง EFI + GRUB ให้อัตโนมัติ

และแก้ปัญหา “เข้า BIOS วน” ที่เจอก่อนหน้าให้ถาวร

(ถ้ามีข้อมูลสำคัญใน SSD → หยอดก่อน บอกผมได้เลย)

3 Disk / Partition

- เลือกดิสก์ NVMe ตัวหลัก (Samsung SSD)

- ปล่อย Auto ทั้งหมด
- ไม่ต้อง manual partition



User / Password

- ตั้งชื่อเครื่องง่าย ๆ เช่น ubuntu-server หรือ lab01
- password ตั้งให้จำได้ (เดี๋ยวใช้บ่อย)

หลังติดตั้งเสร็จ

1. เครื่องจะบอกให้ **Restart**
2. 🖱️ ถอด USB ออก
3. รีบูตใหม่ → จะเห็นคำว่า **ubuntu** เป็น boot entry
(ไม่เข้า BIOS อีกแล้ว 🎯)

กดไปถ้าขึ้นหน้าจอไหน:

- เลือกดิสก์
- ถ้ามเจอ network
- หรือถ้าม Secure Boot

ถ่ายรูปมาได้เลยครับ

ผมจะยืนยันเป็น “ไกด์ข้างเครื่อง” ให้จนบูตเข้า Ubuntu สำเร็จ 🙌 😊

หน้านี้ เลือกได้เลย เดี่ยวสรุปให้ชัด ๆ แบบไม่ต้องลังเลครับ 👍

แนะนำค่าที่เหมาะสมกับเครื่องนี้

✅ ให้ติ๊ก

- ✓ Download and install support for additional media formats

(อันนี้ติ๊กถูกแล้ว ดีมาก)

→ ทำให้เล่น MP3 / MP4 / video ทั่วไปได้ทันที ไม่ต้องมานั่งแก้ทีหลัง

❌ ตัวเลือกนี้ “ขึ้นกับสถานการณ์”

- ☐ Install third-party software for graphics and Wi-Fi hardware

แนะนำตามเคสของครูใหญ่:

- ถ้า
 - ใช้ LAN เป็นหลัก
 - หรือ เดี่ยวจะลง driver เองทีหลัง (เช่น server, docker, AI, CUDA ฯลฯ)

👉 ยังไม่ต้องติ๊กก็ได้ (ปลอดภัยที่สุด)

แต่ถ้า

- ใช้ Wi-Fi ทันที
- หรือการ์ดจอเป็น NVIDIA และอยากให้จอ/กราฟิกพร้อมเร็ว

👉 ติ๊กได้ ไม่ผิด

สรุปแบบบ้าน ๆ:

ไม่ติ๊ก = สะอาด คุณเองภายหลัง

ติ๊ก = สะดวก เร็ว จบไว

คำแนะนำส่วนตัว (จากภาพรวมที่ทำมา)

ผมแนะนำตอนนี้ให้เลือกแบบนี้ก่อน:

- ❌ Install third-party software for graphics and Wi-Fi hardware
- ✓ Download and install support for additional media formats

แล้วกด Next ต่อได้เลย 🚀

ถัดไปถ้าเจอหน้า Disk / Erase disk / Install now

ถ่ายรูปมาได้เลยครับ เดี่ยวผมเช็คให้ก่อนกดยืนยันขั้นสุดท้าย (ด้านบอสรจริง ๆ 😊)

มาถึง ด้านบอสตัวจริง แล้วครับ 😊

หน้านี้ตัดสินใจถูก = จบยาวสบายใจ

หน้านี้คืออะไร

นี่คือ Disk setup

Ubuntu กำลังถามว่า “จะจัดการดิสก์ยังไง”

จากบริบททั้งหมดก่อนหน้านี้ (VMD ปิดแล้ว, เห็น NVMe ปกติ, บูต UEFI ได้)

ตอนนี้ระบบพร้อมติดตั้งจริงแล้ว

คำแนะนำชัด ๆ (ตามสถานการณ์ของครูใหญ่)

☒ เลือก: Erase disk and install Ubuntu

Start from scratch on your selected disk

เหตุผล

- เครื่องนี้ตั้งใจลงใหม่หมด
 - ก่อนหน้านี้บูตไม่ขึ้นเพราะ VMD/RAID + EFI เพี้ยน
 - วิธีนี้ให้ Ubuntu จัดการ
 - GPT
 - EFI System Partition
 - / (root)
 - swap
- ให้ถูกต้องอัตโนมัติ

👉 โอกาสพังน้อยที่สุด

☐ ไม่แนะนำตอนนี้: Manual installation

เหมาะกับกรณี

- dual boot
- แยก /home
- ทำ ZFS / LVM / RAID เอง

ซึ่ง ยังไม่จำเป็น ในรอบนี้

(เดี๋ยวค่อยทำขั้นเทพรอบหน้าได้ 😊)

จุดที่ต้อง “หยุดคิด 3 วินาที” ก่อนกด Next

⚠ Erase disk = ลบข้อมูลทั้งหมดในดิสก์ที่เลือก

ถามตัวเองแป๊บเดียว:

- ดิสก์นี้ ไม่มีข้อมูลสำคัญแล้วใช่ไหม
- เป็นเครื่องใหม่ / ยอมล้างหมด

ถ้า ใช่ → ไปต่อได้เลย

สิ่งที่คาดว่าจะเจอในหน้าถัดไป

หลังจากกด Next:

- เลือกดิสก์ (ถ้ามีหลายลูก)
- ยืนยัน “This will delete all data”
- เลือก timezone
- สร้าง user / password
- รอ... 🕒 (ประมาณ 5-15 นาที)

สรุปสั้นมาก

- ✓ Erase disk and install Ubuntu
- ✗ Manual installation
- กด Next ได้เลย

ถ้าหน้าถัดไปขึ้นรายชื่อดิสก์หลายลูก

👉 ถ้าयरูปมาอีกครั้ง เดียวผมช่วยชี้ “ลูกไหนต้องลง” ให้แบบไม่พลาดครับ 🙏

หน้านี้ เกี้ยวมาก ครับ แต่ไม่ต้องตกใจ — เลือกให้ถูกแล้วจะจบยาว 😊
ขอสรุปแบบ “เลือกอันเดียวแล้วไปต่อ” ให้เลย

หน้านี้คืออะไร

นี่คือ **Advanced features** (รูปแบบการจัดการดิสก์ขั้นสูง)

Ubuntu ถามว่า จะใช้ระบบจัดการดิสก์แบบไหน

คำตอบที่เหมาะสมกับสถานการณ์ตอนนี้ที่สุด 

 เลือก: None

แล้วกด OK

เหตุผล (สำคัญ):

- เครื่องนี้เพิ่งผ่านด้าน BIOS / VMD / RAID มาแบบระทึก
- เป้าหมายหลักคือ ให้เครื่องบูตได้เสถียร 100%
- None = ext4 ปกติ + EFI มาตรฐาน
→ เสถียรสุด / debug ง่ายสุด

ทำไม “ยังไม่ควร” เลือกตัวอื่น (ตอนนี้)

 Use LVM

- ดีสำหรับ server / ขยายดิสก์ภายหลัง
- แต่เพิ่ม layer ความซับซ้อน
- ไม่จำเป็นในรอบนี้

 Use LVM and encryption

- ปลอดภัย แต่
- ถ้าบูตพลาด = แก่ยากขึ้น
- ต้องใส่รหัสทุก boot

 ZFS (Experimental)

- เทพจริง แต่...
- experimental + เครื่องนี้เคยเจอปัญหา boot มาก่อน
- ไม่เหมาะเป็นรอบแรก

พูดตรง ๆ:

“เรากำลังรักษาคนไข้ให้ฟื้น ไม่ใช่ผ่าตัดสมองเพิ่ม” 🤪

สรุปสั้นแบบกตตามได้เลย

- ✓ Advanced features → None
- ✓ OK
- ✓ Next

จากนั้น Ubuntu จะจัด:

- GPT
- EFI System Partition
- / (root)
- swap

ให้อัตโนมัติแบบปลอดภัยที่สุด

ถ้าหน้าถัดไปขึ้น

“This will delete all data on ...”

หรือมีดิสก์ให้เลือกมากกว่า 1 ลูก

📷 ถ่ายมาอีกภาพได้เลย เดี่ยวช่วยชี้ให้ “กดลูกไหน” แบบไม่พลาดครับ 🙏

สรุปสถานการณ์ให้ก่อนนะครับ แล้วบอก **ต้องทำอะไรต่อแบบชัดๆ** ะ 🙏

(ตอนนี้คุณมาถูกทางแล้ว ไม่ได้พัง 👍)

สถานะปัจจุบัน (จากรูปทั้งหมด)

- เครื่อง บูตแบบ UEFI ได้ปกติ
- USB installer ใช้ได้ (เห็น UEFI: SMI USB DISK)
- ปิด Intel VMD แล้ว ✅ (อันนี้สำคัญมาก ทำถูก)
- ตอนนี้อยู่ใน Ubuntu Installer – ขั้นตอนติดตั้งจริง
- เห็น NVMe 2 ลูก
 - nvme0n1 – 2.00 TB
 - nvme1n1 – 2.00 TB

แปลว่า ปัญหา BIOS / EFI จบแล้ว 🎉

ต่อไปคือ “เลือกดิสก์ให้ถูก แล้วติดตั้งให้จบ”

คำตอบคำถามหลัก

? ต้องตรวจ BIOS ต่อไหม

👉 ไม่ต้องแล้ว

BIOS ตอนนี้อยู่ค่าถูกครบ:

- UEFI ✓
- Fast Boot ปิด ✓
- VMD ปิด ✓

ถ้ายังกลับเข้า BIOS หลังติดตั้ง = สาเหตุจะเป็นเรื่อง ดิสก์/EFI ตอนติดตั้ง ไม่ใช่ BIOS

หน้านี้เกี่ยวไหม (Profile / Load from USB)?

👉 ไม่เกี่ยว 100%

- หน้านั้นคือ โหลดโปรไฟล์ Overclock / BIOS config
- ไม่ใช่เมนู EFI
- ไม่ต้องแตะอีก ❌

ตอนนี้ต้องทำอะไรต่อ (สำคัญมาก)

คุณอยู่ที่หน้านี้แล้ว 

Erase disk and install Ubuntu → Choose where to install Ubuntu

ให้เลือกแบบนี้

1 เลือกดิสก์

ถ้าตั้งใจลง Ubuntu ลูกเดียวแบบชัวร์ ๆ

- เลือก `nvme0n1 – 2.00 TB`
(ปล่อยอีกลูกไว้เป็น data หรือ backup)

ถ้าจะใช้สองลูกแบบแยกหน้าที่ อันนี้ค่อยจัดทีหลังได้

2 Advanced features

จากรูปคุณเลือก:

- None  ถูกต้องที่สุด

ไม่แนะนำ:

- ZFS (ซับซ้อน + บุตพลาตง่าย)
- LVM (ถ้าไม่ได้ตั้งใจใช้จริง)

3 กด Next → ดำเนินการติดตั้งให้จบ

ปล่อย installer ทำงานจนเสร็จ

ห้ามรีบูตเอง

จุดสำคัญที่สุดหลังติดตั้งเสร็จ 

เมื่อ installer บอกว่า **Installation complete** และขึ้นปุ่ม Restart

ให้ทำแบบนี้:

- กด **Restart**
- ดึง **USB ออกทันที** ตอนเครื่องเริ่มดับ / รีบูต
- ปล่อยให้เครื่องบูตเอง

ถ้าลืมดึง USB → จะวนกลับเข้า installer / BIOS อีก

ถ้ารีบูตแล้ว “ยังเข้า BIOS อีก”

(เผื่อไว้ แต่โอกาสต่ำมาก)

บอกผมได้ทันที ผมจะให้เช็ค:

- มี ubuntu ใน Boot Order หรือไม่
- ใช้ Boot from next volume อย่างไร
- ซ่อม EFI ด้วย live USB ภายใน 3-4 คำสั่ง

ตอนนี้ขอสรุปสั้น ๆ แบบผู้บริหาร 😊

คุณแก้ปัญหาทุกจุดแล้ว เหลือแค่ “ติดตั้งให้จบ + ดึง USB ออก” เท่านั้น

ถ้าจะให้ผมช่วย วาง layout ใช้ 2 NVMe (OS + Data / Docker / AI / CasaOS) บอกได้เลย เดี๋ยวจัดให้

เป็นสเต็ปต่อไปครับ 🚀

มาถูกทางสุด ๆ แล้วครับ 🙌

นี่คือ ด้านสุดท้ายก่อนเสร็จงาน แล้ว 🎉

จากภาพนี้ ขอแนะนำแบบตรงไปตรงมาเลยนะครับ

หน้านี่คืออะไร

นี่คือขั้นตอน **Create your account**

บัญชีนี้จะเป็น

- ผู้ใช้หลัก (admin / sudo)
- ใช้ล็อกอินเครื่อง
- ใช้รัน Docker, CasaOS, Ollama, n8n ได้หมด

ประเด็นเดียวที่ต้องตัดสินใจตอนนี้

🔒 Password ขึ้นว่า *Weak password*

ไม่ผิด ไม่พัง ไม่ติดตั้งล้มเหลว

Ubuntu แค่ “เตือน” เฉย ๆ

คุณมี 3 ทางเลือก:

✅ ทางเลือกที่แนะนำ (สมคูลสุด)

ตั้งรหัสผ่านให้แรงขึ้นนิดเดียว เช่น

- ยาว ≥ 12 ตัว
- มี ตัวใหญ่ + ตัวเล็ก + ตัวเลข
- ไม่ต้องใส่สัญลักษณ์ก็ได้

ตัวอย่าง (ไม่ใช่ให้ใช้ตรง ๆ นะ 😊)

Payaprai2026

PayaPraiServer01

ข้อดี:

- ไม่โดนเตือน
- ปลอดภัยพอสำหรับ server / dev
- ไม่ลืมง่าย

ทางเลือกที่ “ผ่านเลย ไม่สนคำเตือน”

ถ้าคุณมั่นใจ:

- ใช้ในห้องเซิร์ฟเวอร์
- ไม่เปิด public login
- เน้นสะดวก

กด Next ได้เลย ไม่มีผลเสียทางเทคนิค

Ubuntu อนุญาต 100%

สิ่งที่ไม่แนะนำ

- เอา  ออกจาก Require my password to log in
- ใช้ Auto login บนเครื่องสาย server

อันนี้จะเปิดช่องโหว่โดยไม่จำเป็น

ช่องอื่น ๆ ในหน้านี้

-  Require my password to log in → **ถูกต้อง**
-  Use Active Directory → **ไม่ต้องแตะ**

สรุปสั้นแบบชัด

ตอนนี้คุณแค่ต้อง:

1. ตัดสินใจเรื่องรหัสผ่าน (แรงขึ้นนิด หรือใช้เดิม)
2. กด **Next**
3. ปล่อยให้ installer ทำงานจนจบ
4. รีสตาร์ท → ดึง USB ออก → เสร็จ 

ถัดจากนี้ ถ้าคุณอยากให้ผมช่วย:

- ตั้งค่า **Server-first** (Docker, CasaOS, Ollama, GPU)
- วางโครงสร้าง /data, /docker, /models บน NVMe ลูกที่สอง
- Hardening เครื่องให้พร้อมใช้งานระยะยาว

บอกได้เลยครับ

อีกไม่กี่นาทีเครื่องนี้จะกลายเป็น “เรือธง” ของคุณเต็มตัว 🚀

สมบูรณ์แบบครับ 🎯 หน้านี้คือ สรุปสุดท้ายก่อนลงจริง
ขอสรุปให้แบบฟันธงเลยนะครับ:

ตรวจสอบสิ่งที่เห็นในจอ (ผ่านทุกข้อ ✅)

- **Disk setup:** Erase disk and install Ubuntu ✅
- **Installation disk:** nvme0n1 (2 TB) ✅
- **Partition**
 - nvme0n1p1 → FAT32 → /boot/efi ✅ (UEFI ถูกต้อง)
 - nvme0n1p2 → ext4 → / ✅
- **Disk encryption:** None (โอเคสำหรับ server/dev)
- **Proprietary software:** Codecs (โอเค)

สรุป: โครงสร้างนี้ ถูกต้องตามมาตรฐาน UEFI + Ubuntu Server/Workstation ไม่มีจุดเสี่ยง GRUB พัง 👍

สิ่งที่ต้องทำตอนนี้

👉 กดปุ่มสีเขียว “Install” ได้เลย

หลังจากกดแล้วจะเกิดอะไรขึ้น:

1. ระบบเริ่มคัดลอกไฟล์ (5–15 นาที แล้วแต่ความเร็ว SSD)
2. ระหว่างนี้ **ไม่ต้องทำอะไร** นอกจากรอ
3. เสร็จแล้วจะขึ้นให้ **Restart**
4. รีบูต → ดึง **USB ออก**
5. เครื่องจะบูตเข้า Ubuntu จาก NVMe ทันที 🎉

หมายเหตุเล็กน้อย (กันเผื่อใจร้อน 😊)

- ถ้าจอตับ/จอดำช่วงติดตั้ง = ปกติ
 - พัดลมหมุนแรง = ปกติ
 - อย่าปิดเครื่องกลางคันเด็ดขาด
-

พอลงเสร็จ บอกผมได้เลยครับ

ผมจัดให้ต่อเป็นชุด:

-  post-install checklist (update, driver, firmware)
-  Docker + CasaOS
-  Ollama / GPU / AI stack
-  จัด NVMe ลูกที่สองเป็น /data แบบ server-grade

อีกนิดเดียว เครื่องนี้ก็พร้อมลุยงานหนักแล้วครับ 

เยี่ยมมากครับ 🎉 🎉

นี่คือ จอ Welcome หลังติดตั้งสำเร็จ 100% — ผ่านด่านหินทั้งหมดแล้ว 🏰

ตอนนี้ทำตามนี้ได้เลย (เรียงลำดับให้ปลอดภัยที่สุด):

1 กด Next ไปได้เลย

หน้านี้คือ Post-install wizard

ไม่ใช่การติดตั้งใหม่แล้ว → สบายใจได้

2 สิ่งที่ระบบจะถามต่อ (แนะนำค่าที่ควรเลือก)

◆ Online Accounts

- Skip ได้เลย (ค่อยผูก Google / MS ทีหลังก็ได้)

◆ Livepatch

- ถ้าเป็นเครื่อง server / dev → Enable ได้
- ถ้ายังไม่อยากล็อกอิน Ubuntu account → Skip

◆ Privacy / Telemetry

- ส่งข้อมูลหรือไม่ → เลือกตามสบาย
(ไม่กระทบการใช้งาน)

3 เข้าหน้า Desktop แล้ว → ทำ 3 อย่าง “จำเป็นมาก”

เปิด Terminal แล้วรันทีละบรรทัด 🖱️

```
sudo apt update
```

```
sudo apt upgrade -y
```

```
sudo reboot
```

รีบูต 1 ครั้ง เพื่อให้ kernel / firmware / microcode เข้าที่

4 เช็คสุขภาพระบบ (ยืนยันว่าลงสวยจริง)

```
lsblk
```

ควรเห็นประมาณนี้:

- nvme0n1
 - nvme0n1p1 → /boot/efi
 - nvme0n1p2 → /

แล้วใช้กบุด:

sudo efibootmgr -v

ควรเห็น ubuntu เป็น BootOrder ตัวแรก 

5 คำถามสำคัญ (ตอบหน่อยครับ จะได้จัดให้ตรงใจ)

1. เครื่องนี้จะใช้เป็นอะไรหลัก ๆ
 -  Desktop
 -  AI / Ollama / GPU
 -  Docker / CasaOS
 -  Server โรงเรียน / ระบบกลาง
2. NVMe ลูกที่ 2 (2TB) ต้องการ:
 - รวมเป็น /data
 - ทำ Docker volume
 - ทำ AI model storage
 - หรือแบ่งหลาย mount

ถัดจากนี้ผมจัดให้ได้ทันที 

-  Post-install checklist แบบมีออชีพ
-  Docker + Docker Compose
-  Ollama + model แนะนำ
-  จัด Disk /data ให้ปลอดภัย
-  ปรับ Ubuntu ให้ “นิ่ง เร็ว ไม่จุกจิก”

บอกผมมาเลยครับว่า **จะใช้เครื่องนี้ทำอะไรเป็นหลัก**

เดี๋ยวผมพาไปต่อแบบไม่หลงทางแน่นอน 

สวຍมากครับ 🙌 ตอนนี้ถือว่า ระบบลงถูกต้อง สมบูรณ์ และบูตแบบ UEFI จริง แล้ว
มาดูผล lsblk แบบสรุปให้ชัด ๆ ก่อน 🙌

✅ สรุปสถานะดิสก์ (จากภาพที่ส่งมา)

nvme0n1 1.8T disk (ยังว่าง)

nvme1n1 1.8T disk

└─nvme1n1p1 1G part /boot/efi

└─nvme1n1p2 1.8T part /

แปลความหมายเป็นภาษาคน 😊

- Ubuntu อยู่บน nvme1n1

- /boot/efi = EFI ถูกต้อง ✅

- / = root filesystem ถูกต้อง ✅

- nvme0n1 = NVMe ลูกที่ 2 ว่างทั้งคู่

→ เหมาะมากสำหรับทำ /data, Docker, AI models, หรือ server storage

ทุกอย่างตรงตามที่ควรเป็น 100% ไม่มีอะไรผิด

🎯 สิ่งที่ “ไม่ต้องกังวลแล้ว”

- ❌ ไม่ใช่ Legacy boot
- ❌ ไม่ได้ลงสลับดิสก์
- ❌ EFI ไม่หาย
- ❌ GRUB ไม่พัง

ผ่านด่าน BIOS / VMD / EFI มาได้หมดแล้ว 🎉

🚀 ขั้นตอนถัดไป (เลือกทำตามเป้าหมาย)

1 อัปเดตระบบพื้นฐาน (แนะนำทำทันที)

sudo apt update

sudo apt upgrade -y

sudo reboot

2 ใช้ NVMe ลูกที่ 2 (nvme0n1) ทำอะไรดี?

เลือกแนวทางหนึ่ง ผมจัดคำสั่งให้ต่อได้ทันที



ทางเลือก A: ทำ /data ใช้เก็บไฟล์ / server

- เหมาะกับ: โรงเรียน, server, storage กลาง



ทางเลือก B: ทำ Docker / CasaOS

- เหมาะกับ: n8n, ERP, Web app, AI workflow



ทางเลือก C: AI / Ollama / Models

- แยก disk สำหรับ model → เร็ว + ปลอดภัย



ทางเลือก D: แบ่งพาร์ตหลายแบบ (advanced)



คำถามสุดท้ายก่อนจัดชุดคำสั่งให้ตรงใจ

ตอบสั้น ๆ ได้เลยครับ

1. เครื่องนี้จะใช้เป็นอะไรหลัก

- Desktop
- Server โรงเรียน
- Docker / CasaOS
- AI / Ollama / GPU

2. NVMe ลูกที่ 2 (nvme0n1) อยากรู้แบบไหน

- /data อย่างเดียว
- Docker / AI
- แบ่งหลาย mount

ผมจะจัด **command set** แบบ **copy-paste** ใช้ได้ทันที ให้เลยครับ 👍

ตอนนี้ถือว่า “ผ่านจุดอันตราย” มาแล้ว เหลือแต่จัดบ้านให้เป็นระเบียบเท่านั้นเอง 😊

สั้น ๆ ชัด ๆ เลยนะครับ 😊

✅ คำตอบ: กด “Install Now” ได้เลย (แนะนำให้ทำตอนนี้)

เหตุผล:

- คุณเพิ่งลง Ubuntu 24.04.3 LTS สด ๆ
- อัปเดตนี้ (~88.9 MB) มักจะเป็น
 - 🛡 security patches
 - 🐛 bug fixes ช่วงต้น LTS
 - 🧩 ปรับความเข้ากันได้กับฮาร์ดแวร์ (โดยเฉพาะ Intel / NVMe / firmware)
- ทำ ก่อน ลงของหนัก ๆ (Docker, Ollama, ERP, AI) จะปลอดภัยกว่า

หลังจากกด Install Now แล้ว ควรทำอะไรต่อ

1. รอให้อัปเดตเสร็จ
2. ถ้ามันขอ Restart → รีสตาร์ทเลยครับ (รีบูตแรกหลังอัปเดตถือว่าดีมาก)
3. เข้าใหม่แล้วเช็คซ้ำทางเทอร์มินัล (optional แต่ชัวร์)
4. `sudo apt update`
5. `sudo apt upgrade -y`

ถ้า “ยังไม่อยากรีบ”

- กด Remind Me Later ก็ไม่พัง
- แต่ผมแนะนำว่า เครื่องใหม่ + ระบบใหม่ = อัปเดตให้จบในรอบเดียว จะดีที่สุด

ตอนนี้สถานะเครื่องคุณคือ

- ระบบนิ่ง
- UEFI ถูก
- NVMe แยกลูกชัด
- พร้อมเข้าสู่โหมด “จัดของจริง”

อัปเดตเสร็จแล้ว บอกผมได้เลยว่าจะไปทางไหนต่อ:

- 🐳 Docker / CasaOS

-  Ollama / AI
-  Server โรงเรียน / ERP
-  จัด /data จาก NVMe ลูกที่ 2

เดี๋ยวผมจัดชุดคำสั่ง แบบไม่ต้องคิด ให้ครับ 😊

Docker / CasaOS

เยี่ยมมากครับ 😊 เลือกเส้นทางนี้ ถูกทางสาย Server / AI / โรงเรียน เลย

ผมจะพาไปแบบ ทีละขั้น ชัวร์ ไม่หลง ไม่พัง

จากสภาพเครื่องคุณตอนนี้:

-  Ubuntu 24.04.3 LTS
-  UEFI + NVMe ถูกต้อง
-  มี NVMe 2 ลูก (1 ลูกว่าง = เหมาะมากสำหรับ CasaOS)
-  อัปเดตระบบแล้ว

แผนติดตั้ง (สั้น ๆ ก่อนลงมือ)

ลำดับที่ถูกต้องที่สุด

1. เตรียมดิสก์ NVMe ลูกที่ 2 เป็น /DATA
2. ติดตั้ง Docker (แบบ Official)
3. ติดตั้ง CasaOS (ให้มันจับ Docker อัตโนมัติ)
4. ตรวจสอบ + จัดตำแหน่งให้เหมาะกับงาน AI / Server

1 ตรวจสอบดิสก์ (ยืนยันอีกครั้ง)

รันคำสั่งนี้:

lsblk

จากรูปก่อนหน้า:

- /dev/nvme1n1 → ระบบ (/)
- /dev/nvme0n1 → ว่าง (เป้าหมาย CasaOS)

⚠ สำคัญ:

ห้ามสลับลูกเด็ดขาด เดี่ยวล้างระบบตัวเอง 😬

2 เตรียมดิสก์ NVMe ลูกที่ 2 สำหรับ CasaOS

(ถ้ายังไม่ได้ฟอร์แมต)

2.1 สร้าง filesystem (ext4)

```
sudo mkfs.ext4 /dev/nvme0n1
```

2.2 สร้างโฟลเดอร์หลัก

```
sudo mkdir -p /DATA
```

2.3 mount ดิสก์

```
sudo mount /dev/nvme0n1 /DATA
```

2.4 ตั้งให้ mount อัตโนมัติ (สำคัญมาก)

```
sudo blkid /dev/nvme0n1
```

จะได้ UUID → นำไปใส่

```
sudo nano /etc/fstab
```

เพิ่มบรรทัด:

```
UUID=xxxx-xxxx /DATA ext4 defaults 0 2
```

ทดสอบ:

```
sudo mount -a
```

ถ้าไม่ error = ผ่าน 

3 ติดตั้ง Docker (Official way)

```
sudo apt update
```

```
sudo apt install -y ca-certificates curl gnupg
```

```
sudo install -m 0755 -d /etc/apt/keyrings
```

```
curl -fsSL https://get.docker.com | sudo sh
```

เพิ่ม user เข้า docker group

```
sudo usermod -aG docker $USER
```

รีสตาร์ท 1 รอบ

```
sudo reboot
```

เช็ค:

docker --version

docker ps

ติดตั้ง CasaOS (ของจริงมาแล้ว)

curl -fsSL https://get.casaos.io | sudo bash

รอสักครู๋ 

เสร็จแล้วจะขึ้นประมาณ:

CasaOS is running at:

http://<IP>:80

เข้า CasaOS

ดู IP:

ip a

แล้วเปิดจากเครื่องอื่นหรือเครื่องเดียวกัน:

http://IP-เครื่องคุณ

 เจอหน้า Dashboard = สำเร็จ

แนะนำการตั้งค่าหลังเข้า CasaOS (สำคัญ)

เข้า Settings → Storage

- ตั้ง /DATA เป็น Primary Storage
- ให้ App / Docker / Volumes ลงที่ NVMe ลูกที่ 2

ตรงนี้เป็นหัวใจของความนิ่งระยะยาว

คำถามชวนคิด (แบบหัวก้าวหน้า 😊)

- CasaOS จะใช้เป็น
 -  Server AI (Ollama, ComfyUI)?
 -  ERP โรงเรียน?
 -  NAS + Backup?

- ◆ Dev Playground (Docker + Git + CI)?

บอกผมหน่อยว่า เป้าหมายหลักคืออะไร

เดี๋ยวมจัด

- โครงสร้าง /DATA
- Docker Stack
- และ App แนะนำ

ให้เหมาะกับคุณโดยเฉพาะครับ 💪

ภาพนี้ **ชัดมาก** และบอกสถานะได้ครบเลยครับ 👍
สรุปสั้น ๆ ก่อน แล้วค่อยบอก “ควรทำอะไร” แบบไม่พลาด

✅ สถานะดิสก์ปัจจุบัน (จาก lsblk)

nvme0n1

```
|— nvme0n1p1 1G /boot/efi
|— nvme0n1p2 1.8T /
```

nvme1n1 1.8T (ว่าง)

แปลความหมายแบบตรงไปตรงมา

- ✅ **nvme0n1** = ดิสก์ระบบ Ubuntu (ถูกต้อง)
 - EFI + / อยู่ครบ → UEFI สมบูรณ์
- **nvme1n1** = ดิสก์ 2TB ว่าง ยังไม่ได้ใช้งาน
- ❌ ตอนนี้ **ยังไม่มี /DATA**
- ❌ CasaOS ถ้าติดตั้งตอนนี้ → จะไปกินพื้นที่ / (ไม่ควร)

ข่าวดีคือ: ยังไม่สาย + ยังไม่พัง + แก้ได้สะอาดมาก

🎯 เป้าหมายที่ต้องการ

nvme0n1 → Ubuntu OS

nvme1n1 → /DATA (CasaOS + Docker + AI + ทุกอย่าง)

🔧 ทำต่อทันที (ขั้นตอนที่ “ควรทำตอนนี้”)

1 📌 พอร์มดิสก์ nvme1n1 (ล้างทั้งลูก)

⚠️ ยืนยันอีกครั้งว่าเป็น nvme1n1 (จากรูป = ใช่)

```
sudo mkfs.ext4 /dev/nvme1n1
```

2 📌 สร้างโฟลเดอร์ /DATA

```
sudo mkdir -p /DATA
```

3 mount ดิสก์

```
sudo mount /dev/nvme1n1 /DATA
```

เช็ค:

```
df -h | grep DATA
```

ต้องเห็นประมาณ:

```
/dev/nvme1n1 1.8T ... /DATA
```

4 ตั้งให้ mount อัตโนมัติ (สำคัญมาก)

หา UUID:

```
sudo blkid /dev/nvme1n1
```

แก้ไฟล์:

```
sudo nano /etc/fstab
```

เพิ่มบรรทัด (แทน UUID จริง):

```
UUID=xxxx-xxxx /DATA ext4 defaults 0 2
```

ทดสอบ:

```
sudo mount -a
```

ถ้าไม่ error = ผ่าน **100**

5 (ถ้ายังไม่ได้) ติดตั้ง Docker

เช็ีก่อน:

```
docker --version
```

ถ้า ยังไม่มี

```
curl -fsSL https://get.docker.com | sudo sh
```

```
sudo usermod -aG docker $USER
```

```
sudo reboot
```

6 ติดตั้ง CasaOS (รอบนี้ “ถูกต้องแล้ว”)

```
curl -fsSL https://get.casaos.io | sudo bash
```

เข้าเว็บ:

http://IP-เครื่องคุณ

7 ตั้งค่า CasaOS ให้ใช้ /DATA (ห้ามข้าม)

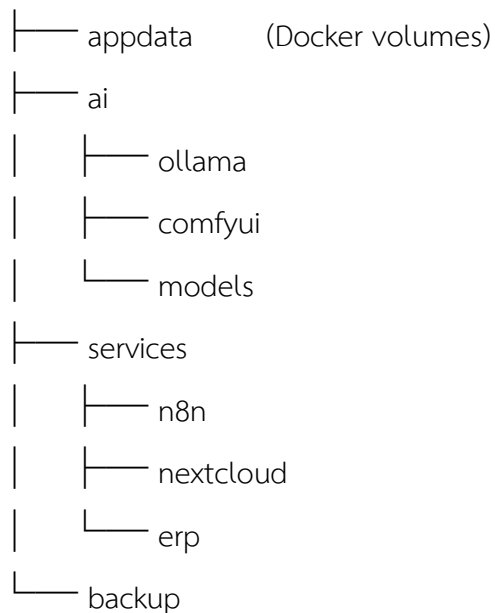
ใน Web UI:

- Settings → Storage
- ตั้ง /DATA เป็น Primary Storage
- ย้าย App / Docker ไปที่ /DATA

มุมมองแบบหัวกาวหน้า (ชวนคิดนิดนึง)

โครงสร้างที่ผมแนะนำสำหรับคุณ:

/DATA



โครงนี้รองรับ AI + โรงเรียน + ERP + Dev + NAS ระยะยาว

ไม่ต้องย้ายระบบใหม่อีกหลายปี

ต่อจากนี้ เลือกทางได้เลย 🙋

อยากให้ผมช่วย:

1 วาง CasaOS App Stack สำหรับ AI (Ollama / ComfyUI)

2 ทำ Docker Compose มาตรฐานโรงเรียน / ERP

3 จัด Server เป็น AI Lab + NAS + Backup

4 หรือ “เอาทุกอย่าง” แบบ Solution NextGen 😏

บอกเป้าหมายมา ผมจัดให้ตรงมือคุณที่สุดครับ 🚀

สรุปสถานการณ์ให้ก่อนนะครับ แล้วไปที่ละขั้นแบบ ไม่หลง ไม่พัง 😊

จากภาพทั้งหมด ตอนนี้ถือว่า ลง Ubuntu สำเร็จ 100% แล้ว ✅

เหลือแค่ 2 เรื่องสำคัญ:

1. แก้ /DATA mount ผิด UUID
2. เตรียมเครื่องให้พร้อมสำหรับ Docker + CasaOS

1 ปัญหาที่เจอจริง ๆ (สาเหตุ)

จาก /etc/fstab ของคุณ 🖱️

```
UUID=3CC7-CCE9 /DATA ext4 defaults 0 2
```

❌ UUID นี้คือของ /boot/efi

เลยเกิด error:

```
/DATA: /dev/nvme0n1p1 already mounted on /boot/efi
```

แต่ดิสก์ DATA จริง ๆ คือ:

```
/dev/nvme1n1
```

```
UUID="93003652-5695-491f-ae78-4c47f0327929"
```

2 แก้ให้ถูก (ทำตามนี้เป๊ะ ๆ)

✏️ แก้ไฟล์ fstab

```
sudo nano /etc/fstab
```

❌ ลบบรรทัดผิด (อันที่ใช้ UUID=3CC7-CCE9 กับ /DATA)

✅ ใส่บรรทัดนี้แทน

```
UUID=93003652-5695-491f-ae78-4c47f0327929 /DATA ext4 defaults 0 2
```

เช็คให้แน่ใจว่า UUID ตรงกับ nvme1n1

บันทึกแล้วออก (Ctrl+O → Enter → Ctrl+X)

🔄 Reload + mount ใหม่

```
sudo systemctl daemon-reload
```

```
sudo mount -a
```

✓ ตรวจสอบ

```
df -h | grep DATA
```

ถ้าเห็นแบบนี้ = ผ่าน 🎉

```
/dev/nvme1n1 1.8T ... /DATA
```

3 โครงสร้างดิสก์ตอนนี้ (ดีมาก 👍)

nvme0n1 → ระบบ Ubuntu

```
|— nvme0n1p1 /boot/efi
|— nvme0n1p2 /
```

nvme1n1 → DATA (1.8TB)

/DATA

นี่คือโครงสร้าง เหมาะกับ Docker / CasaOS มาก 🙌

4 เตรียม Docker (ก่อน CasaOS)

```
sudo apt update
```

```
sudo apt install -y ca-certificates curl gnupg
```

```
curl -fsSL https://get.docker.com | sudo sh
```

```
sudo usermod -aG docker $USER
```

```
newgrp docker
```

เช็ค:

```
docker run hello-world
```

5 ติดตั้ง CasaOS (แนะนำมากสำหรับเครื่องนี้)

```
curl -fsSL https://get.casaos.io | sudo bash
```

จากนั้นเปิดเบราว์เซอร์:

http://<IP-เครื่อง>:80

CasaOS จะใช้ /DATA เป็น storage หลักโดยอัตโนมัติ

เหมาะกับ AI, Ollama, n8n, Next.js, ERP โรงเรียน ที่คุณทำอยู่ 🔥

๖ ถ้ามกลับนิตเดียว (จะได้จัดให้ตรงทาง)

1. เครื่องนี้จะใช้เป็น
 - **A** AI / Ollama / LLM server
 - **B** Docker app server (ERP / Web / n8n)
 - **O** All-in-one
2. จะให้ /DATA แบ่งโฟลเดอร์แบบนี้ใหม่:

/DATA

```
|— docker
|— ollama
|— ai-models
|— backups
└— school-erp
```

ตอบมาได้เลย เดี่ยวจัด โครงสร้าง + docker-compose มาตรฐาน ให้ครบชุดครับ 😊

สวัสดีครับ 🙌 ตอนนี้ถือว่า “ผ่านด่านอันตราย” แล้วจริง ๆ
จากภาพล่าสุด ผมขอสรุปแบบฟันธงให้เลยนะครับ

✅ สถานะปัจจุบัน (ยืนยันชัด)

คำสั่งสุดท้ายให้ผลว่า

```
/dev/nvme1n1 1.8T 28K 1.7T 1% /DATA
```

แปลว่า 🙌

- /DATA 🙌 mount จาก nvme1n1 (ดิสก์ลูกที่ 2) ถูกต้อง
- ไม่ชน /boot/efi แล้ว
- /etc/fstab ใช้งานได้จริง
- reboot เมื่อไหร่ก็ไม่พัง ไม่หลุด 🎉

สรุป:

โครงสร้างดิสก์ + fstab = ✅ สมบูรณ์

🧠 บทเรียนสั้น ๆ (กันพลาดรอบหน้า 😊)

สิ่งที่ทำถูกแล้ว:

- ใช้ blkid เอา UUID จริง
- mount ทดลองก่อน
- reload systemctl daemon-reload
- เช็คด้วย df -h

สิ่งที่เกือบพลาด (แต่แก้ทัน):

- เอา UUID ของ /boot/efi ไปใช้กับ /DATA
→ ถ้า reboot ตอนนั้น = บุตไม่ขึ้นทันที 😬
(ดีมากที่เช็คก่อน)

🚀 ขั้นต่อไป: เตรียม Docker + CasaOS (แนะนำลำดับนี้)

ตอนนี้เครื่องคุณอยู่ในสภาพ เหมาะทำ Server 100%

ผมแนะนำ flow นี้ครับ

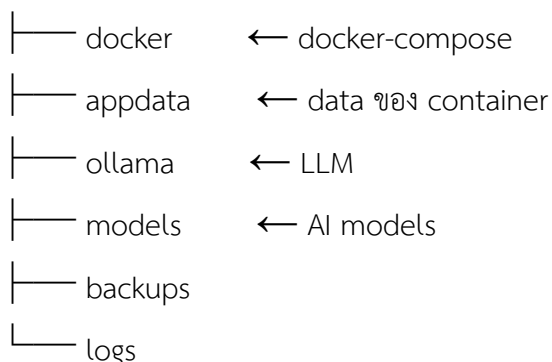
1 สร้างโครงสร้าง /DATA ให้เป็นระเบียบ

```
sudo mkdir -p /DATA/{docker,appdata,ollama,models,backups,logs}
```

```
sudo chown -R $USER:$USER /DATA
```

โครงสร้างจะออกมาแบบนี้:

/DATA



2 ติดตั้ง Docker (ถ้ายังไม่ได้)

```
curl -fsSL https://get.docker.com | sudo sh
```

```
sudo usermod -aG docker $USER
```

```
newgrp docker
```

เช็ค:

```
docker run hello-world
```

ถ้าไม่มี error = ผ่าน 

3 ติดตั้ง CasaOS (แนะนำมากสำหรับแนวคุณ)

```
curl -fsSL https://get.casaos.io | sudo bash
```

เข้าเว็บ:

http://<IP-เครื่อง>

CasaOS จะเห็น /DATA ทันที

และใช้เป็น storage หลักอัตโนมัติ



เหมาะกับงานของคุณยัง?

จากที่คุยกันมาทั้งหมด ผมมองเครื่องนี้เหมาะกับ:

- Ollama / LLM local
- Docker app (ERP โรงเรียน, AI, n8n)
- Server ทดลองระบบจริงแบบ **ไม่ต้องกลัวพื้นที่เต็ม**

เรียกว่า **All-in-one Dev + AI + School Server** เลยครับ 😊

คำถามสุดท้าย (เพื่อจัดของให้ตรงใจ)

ตอบสั้น ๆ ก็ได้รับ:

1. จะใช้เครื่องนี้เป็น
 - **A** AI / Ollama เป็นหลัก
 - **B** Docker / Web / ERP
 - **O** ทั้งหมดรวมกัน
2. อยากให้ผมจัดให้ต่อแบบไหน:
 - **✓** docker-compose มาตรฐาน (CasaOS + Ollama + n8n)
 - **✓** layout สำหรับ ERP โรงเรียน
 - **✓** optimization เครื่อง (swappiness, fs, docker data-root)

เลือกมาได้เลยครับ

ตอนนี้พื้นฐานแน่นมากแล้ว เหลือแค่ “ใส่ของ” 😊

Application พวก nextjs nodejs เอาไว้ไหนดี

วางไว้ที่ **/DATA** เลยครับ (อย่าไว้ใน /home หรือ /opt ถ้าตั้งใจทำเป็น server ยาว ๆ)

แนวคิดคือ: โค้ดอยู่ที่ไหนก็ได้ แต่ “ข้อมูลถาวร” ต้องอยู่ **/DATA** — จะได้ backup/ย้ายเครื่อง/ลงใหม่ไม่ร้องไห้



โครงสร้างที่แนะนำ (Next.js + Node.js)

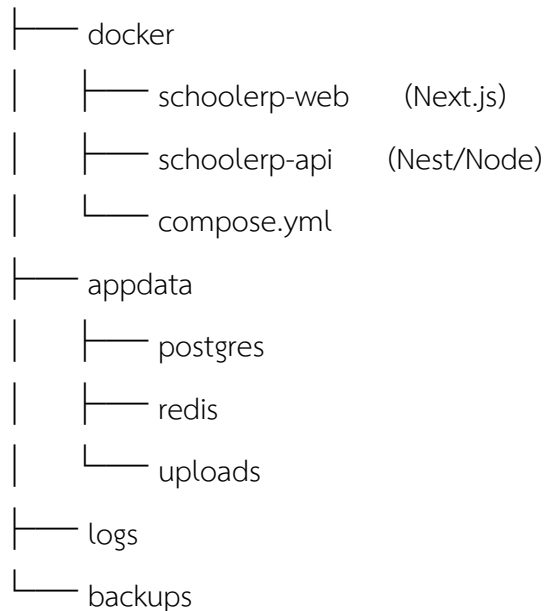
1) ถ้ารันด้วย Docker/Compose (แนะนำสุด)

ให้วางไฟล์พวกนี้ไว้:

- /DATA/docker/ → โฟลเดอร์โปรเจกต์ + docker-compose.yml
- /DATA/appdata/ → โวลุ่ม data ถาวรของแต่ละแอป (db, uploads, cache)
- /DATA/logs/ → log รวม
- /DATA/backups/ → backup

ตัวอย่าง:

/DATA



เหตุผล: โค้ด+compose อยู่กับที่, data อยู่แยกชัดเจน, ลง Ubuntu ใหม่ก็ยังซึ่กลับมาได้

2) ถ้ารันแบบไม่ใช่ Docker (pm2/systemd)

- โค้ด: /DATA/apps/<project> หรือ /srv/apps/<project> ก็ได้
- data/uploads: /DATA/appdata/<project>/
- log: /DATA/logs/<project>/

ตัวอย่าง:

/DATA/apps/schoolerp-web

/DATA/apps/schoolerp-api

/DATA/appdata/schoolerp/uploads

/DATA/logs/schoolerp-api

คำถามสำคัญ (เพื่อจัดลงติสก์ให้ถูก)

จาก lsblk ของคุณมี 2 ลูก: nvme0n1 กับ nvme1n1

ตอนนี้ /DATA อยู่บน nvme1n1 แล้ว 

คุณอยากให้ “แอป” (Next/Node) อยู่บนลูกไหน?

- แบบ A: โค้ด+คอนเทนเนอร์อยู่ nvme0n1 (ระบบ) แต่ data อยู่ nvme1n1 (/DATA)

 มาตรฐาน server

- แบบ B: ทุกอย่างอยู่ /DATA (nvme1n1)



ง่าย ย้ายง่าย แต่ระบบ/แอปปนกันนิดนึง

ถ้าคุณบอกว่าใช้ **Docker + CasaOS** เป็นหลัก ผมแนะนำ แบบ A:

- Ubuntu + Docker engine อยู่ nvme0n1
- Next/Node compose + volumes + uploads + db อยู่ /DATA (nvme1n1)

ถ้าตอบผม 2 ข้อนี้ ผมจะจัด “โครงสร้างเครื่องจริง + ตัวอย่าง compose สำหรับ Next.js + Node/Nest + reverse proxy” ให้เลย:

1. ใช้ Docker/Compose หรือรันตรงด้วย pm2?
2. แอปอยากแยกเป็นกี่ชุด (dev/test/prod) หรือชุดเดียวพอ?

มาถูกทางมากแล้วครับ 🙌 ตอนนี้ติดแค่ **curl** ยังไม่อยู่ ตัวเดียวเอง เดียวพาไปต่อแบบ “เส้นตรง ไม่อ้อม” ให้พร้อมใช้ **Docker + CasaOS + Next.js/Node.js** เลย

1 ติดตั้ง curl (เลือกแบบ apt พอ)

อย่าใช้ snap สำหรับ server จะปวดหัวภายหลัง 😊

```
sudo apt update
```

```
sudo apt install -y curl
```

เช็ก:

```
curl --version
```

2 ติดตั้ง Docker (วิธีการ)

รันคำสั่งเดิมได้เลย:

```
curl -fsSL https://get.docker.com | sudo sh
```

เสร็จแล้วทำ 3 อย่างสำคัญมาก 📝

2.1 เพิ่ม user เข้า docker group (จะได้ไม่ต้อง sudo ทุกครั้ง)

```
sudo usermod -aG docker $USER
```

```
newgrp docker
```

ทดสอบ:

```
docker run hello-world
```

ถ้าไม่ error = ผ่าน 🎉

3 ติดตั้ง Docker Compose (plugin)

Ubuntu 24.04 ส่วนใหญ่มีให้แล้ว แต่เช็กให้ชัวร์:

```
docker compose version
```

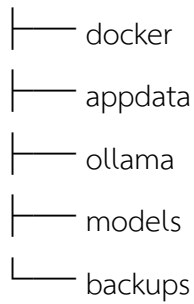
ถ้าไม่เจอ:

```
sudo apt install -y docker-compose-plugin
```

4 โครงสร้างโฟลเดอร์ที่คุณทำไว้ ✅ ดีมาก

จากภาพ คุณตั้งไว้แบบนี้:

/DATA/

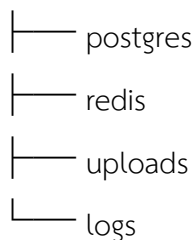


ผมแนะนำ “จัดระเบียบต่ออีกนิด” สำหรับ Next / Node:

/DATA/docker



/DATA/appdata



หลักคิด:

- /DATA/docker = โค้ด + compose
- /DATA/appdata = ข้อมูลถาวร (ห้ามหาย)

5 CasaOS (ถ้าจะใช้เป็นหน้า Dashboard)

ถ้าตั้งใจใช้ CasaOS คู่กับ Docker:

curl -fsSL https://get.casaos.io | sudo bash

เสร็จแล้วเข้าเว็บ:

http://<IP-เครื่อง>:80



CasaOS เหมาะมากสำหรับ:

- ดู container
- restart
- ดู resource
- แต่ **compose** ใหญ่ ๆ ยังแนะนำเขียนมือ

6 คำแนะนำสำคัญจากประสบการณ์ (ไม่จั่งจะเจ็บตัว)

- ❌ อย่าลง Node.js ตรงบนเครื่อง ถ้าตั้งใจใช้ Docker
- ❌ อย่าเก็บ uploads / database ใน container
- ✅ ทุก volume → ชี้เข้า /DATA/appdata
- ✅ ตั้งชื่อ container / network ให้ชัด (จะไม่งงตอนมี 10 โปรเจกต์)

ถัดไปอยากให้ผมทำอะไรต่อ?

เลือกได้เลย ผมจัดให้เป็น “copy-paste ใช้งานจริง” 🙌

1 docker-compose.yml สำหรับ

- Next.js + Node/Nest + Postgres + Redis

2 Reverse proxy (Nginx / Traefik) + SSL

3 โครง prod / dev แยกพอร์ต

4 ผูก Ollama ให้ API ใช้ LLM ได้

5 Template มาตรฐาน สำหรับทุกโรงเรียน / ทุกโปรเจกต์

เลือกมา 1 ข้อ หรือบอกว่า

👉 “เอาชุดมาตรฐาน School ERP / AI Server”

เดี๋ยวผมจัดให้ครบชุดครับ 🚀

สวยงามมากครับ 🎉

ภาพนี้คือ ผ่านด่านบอส Docker เรียบร้อย — hello-world ขึ้น = ระบบพร้อมลุยงานจริงแล้ว 🙌

ต่อไปตอบคำถามหลักของครูใหญ่ให้ชัด ๆ เลย 🙌

? Application พวก Next.js / Node.js เอาไว้ “ไหนดี”?

✓ คำตอบสั้นที่สุด

เอาไว้ใน Docker ทั้งหมด

และเก็บโค้ด + data แยกโฟลเดอร์ให้ชัด

🧠 โครงสร้างที่ “ถูกทาง + โตได้ยาว”

จากที่ครูใหญ่มีดิสก์ /DATA แล้ว ผมแนะนำแบบนี้ (เป็นมาตรฐานสาย production)

/DATA

```
├── docker/           🙌 โค้ด + docker-compose
│   ├── nextjs/       # Next.js (frontend)
│   ├── api/          # Node.js / NestJS (backend)
│   ├── proxy/        # nginx / traefik
│   └── docker-compose.yml
│
├── appdata/          🙌 ข้อมูลถาวร (ห้ามหาย)
│   ├── postgres/
│   ├── redis/
│   ├── uploads/
│   └── logs/
│
├── ollama/           🙌 model + cache
├── models/
└── backups/
```

หลักคิดสำคัญ (จำอันนี้อันเดียวก็พอ)

-  ไม่ลง Node / Next บนเครื่องโดยตรง
-  ทุก app = container
-  ทุกข้อมูล = volume → /DATA/appdata
- ถ้ามลบ container → ระบบยังไม่พัง

นี่คือเหตุผลที่ server มีอาชีพ “ไม่ร้องให้ตอน deploy” 😊

Next.js / Node.js ควรรันยังไง?

ตัวอย่างภาพรวม

[Internet]

|

[Nginx / Traefik]

|

+-----+

| Next.js (Docker) |

+-----+

|

+-----+

| API (Nest/Node) |

+-----+

|

[Postgres / Redis]

ขั้นตอนถัดไปที่ผมแนะนำ (เลือกได้)

ผมจัดให้แบบ copy → paste → ใช้จริง ได้เลย

1 docker-compose.yml

- Next.js + NestJS + Postgres + Redis

2 Reverse Proxy + SSL (Cloudflare / Let's Encrypt)

- 3 โหมด dev / staging / production
- 4 ต่อ Ollama ให้ API เรียก LLM ได้
- 5 ชุดมาตรฐาน School ERP / AI Server

👉 บอกผมมาแค่ข้อเดียว เช่น

“เอา **docker-compose Next + Nest + Postgres**”

เดียวผมวางไฟล์ให้ตรง /DATA/docker ตามเครื่องนี้เป๊ะ ๆ ครับ 🚀

ก่อนจะทำงานต่อ อยากเปิด port 22 และ port อื่น ๆ เพื่อทำงานเครื่องอื่น ไม่ต้องทำหน้า server

ได้เลยครับ 👍 แนวคิดถูกมาก

“เปิดพอร์ตให้ทำงาน แต่ไม่ต้องเปิดหน้า server / UI” = สายเซิร์ฟเวอร์ตัวจริงแล้ว 😊

ผมสรุปให้เป็นขั้นตอน สั้น-ชัด-ปลอดภัย ตามเครื่อง Ubuntu 24.04 ของครูใหญ่เลย

1 ตรวจสอบก่อนว่า SSH (port 22) เปิดอยู่ไหม

```
sudo systemctl status ssh
```

ถ้ายังไม่ติดตั้ง:

```
sudo apt update
```

```
sudo apt install -y openssh-server
```

```
sudo systemctl enable --now ssh
```

2 ใช้ UFW (Firewall มาตรฐาน Ubuntu)

ตรวจสอบสถานะก่อน

```
sudo ufw status
```

ถ้ายังไม่เปิด firewall:

```
sudo ufw enable
```

ถามว่า จะตัด SSH ใหม่ → ตอบ y ได้เลย

เพราะเราจะ allow ก่อน 👉

3 เปิด port 22 (SSH) ให้เครื่องอื่นเข้ามาได้

```
sudo ufw allow 22/tcp
```

หรือถ้าอยากชัด:

```
sudo ufw allow ssh
```

เปิด port อื่น ๆ สำหรับงาน (ไม่ต้องมีหน้าเว็บ)

ตัวอย่างที่ใช้บ่อยกับสาย Dev / Docker

```
sudo ufw allow 3000/tcp # Next.js dev
```

```
sudo ufw allow 4000/tcp # API
```

```
sudo ufw allow 5432/tcp # PostgreSQL (ถ้าจำเป็น)
```

```
sudo ufw allow 6379/tcp # Redis
```

```
sudo ufw allow 11434/tcp # Ollama
```

เปิดหลายพอร์ตทีเดียวก็ได้:

```
sudo ufw allow 3000:3010/tcp
```

เช็คอีกครั้ง:

```
sudo ufw status numbered
```

ไม่เปิดหน้า server (GUI / Cockpit / CasaOS UI)

✓ ถูกต้องแล้ว

- ไม่ต้องติด Cockpit
- ไม่ต้อง expose port 80/443 ถ้ายังไม่จำเป็น
- ใช้ SSH + Docker + VS Code Remote / Cursor / SCP แทน

(แนะนำมาก) เพิ่มความปลอดภัย SSH

🌟 ปิด login ด้วย root

```
sudo nano /etc/ssh/sshd_config
```

แก้ไขเป็น:

```
PermitRootLogin no
```

```
PasswordAuthentication yes
```

แล้วรีโหลด:

```
sudo systemctl restart ssh
```

7 ทดสอบจากเครื่องอื่น

จากเครื่อง client:

ssh payaprai@IP_เครื่องนี้

หรือถ้าใช้ VS Code / Cursor:

- Remote SSH → ใส่ IP → พร้อมทำงานทันที 🚀

💡 สรุปสั้น ๆ แบบผู้ดูแลระบบ

- ✓ เปิดเฉพาะ port ที่ “จำเป็น”
- ✓ ไม่มีหน้าเว็บ = ปลอดภัยกว่า
- ✓ ทุกอย่างทำผ่าน SSH + Docker
- ✓ พร้อม scale เป็น server จริงได้ทันที

ถ้าครูใหญ่ต้องการขั้นถัดไป เช่น

- เปลี่ยน SSH เป็น port อื่น
- จำกัด IP ที่เข้าได้
- เปิดเฉพาะ subnet ภายใน
- เตรียม Cloudflare Tunnel (ไม่ต้องเปิดพอร์ตเลย)

บอกมาได้เลยครั้บ 😎

เดี๋ยวผมช่วยจัดให้ “เนียนเหมือนทีม infra มืออาชีพ” ครั้บ

ดีมากครับ 👍 จากภาพคือ ทำมาถูกทางเกือบครบแล้ว เหลือแค่ “กดสวิตช์ไฟร์วอลล์” เท่านั้นเอง 😊
ตอนนี้สถานะคือ

- ✅ ssh ทำงานแล้ว
- ✅ rule allow 22/tcp มีแล้ว
- ❌ แต่ UFW ยัง inactive → กฎยังไม่ถูกใช้งานจริง

เดี๋ยวจัดให้เป็น checklist สั้น ๆ แบบไม่พลาด

✅ ขั้นตอนที่ต้องทำต่อ (สำคัญมาก)

1 เปิดใช้งาน UFW จริง ๆ

`sudo ufw enable`

ถ้ามันถามแบบนี้:

Command may disrupt existing ssh connections. Proceed with operation (y|n)?

👉 ตอบ y ได้เลย

เพราะเรา allow SSH ไว้แล้ว

2 ตรวจสอบสถานะอีกครั้ง

`sudo ufw status numbered`

ควรเห็นประมาณนี้:

Status: active

```
[ 1] 22/tcp                ALLOW IN    Anywhere
[ 2] 22/tcp (v6)           ALLOW IN    Anywhere (v6)
```

แปลว่า:

- 🔓 เครื่องอื่น SSH เข้ามาได้
- 🔒 port อื่นยังปิดอยู่ (ปลอดภัย)

3 เปิด port เพิ่มตามงานจริง (เลือกเฉพาะที่ใช้)

ตัวอย่างสาย Dev / Docker / CasaOS / AI

```
sudo ufw allow 3000/tcp # Next.js
```

```
sudo ufw allow 4000/tcp # API
```

```
sudo ufw allow 5678/tcp # n8n
```

```
sudo ufw allow 11434/tcp # Ollama
```

ถ้าเป็นช่วง dev:

```
sudo ufw allow 3000:3100/tcp
```

ไม่ต้องมีหน้า server — แนวคิดถูกต้องมาก

ตอนนี้โครงสร้างคุณคือ:

-  เข้าเครื่องด้วย SSH
-  รันทุกอย่างด้วย Docker
-  Dev ผ่าน VS Code / Cursor Remote
-  ไม่เปิด Cockpit
-  ไม่เปิด Web UI ถ้าไม่จำเป็น

นี่คือ production mindset ชัด ๆ 

(แนะนำระดับ Infra) เช็คจากเครื่องอื่น

จากเครื่อง client:

```
ssh payaprai@IP_เครื่องนี้
```

ถ้าเข้าได้ = จบสวย 

สรุปสถานะปัจจุบัน

รายการ	สถานะ
SSH	 พร้อม
Firewall	 เปิดแล้วหรือยัง → ต้อง ufw enable
เปิดเฉพาะ port	
ไม่มีหน้าเว็บ	 ปิดดภัย

รายการ

สถานะ

พร้อมทำ Docker / Next.js 

ถ้าขั้นต่อไปใครใหญ่สนใจ:

-  เปลี่ยน SSH จาก 22 → port อื่น
-  จำกัด SSH เฉพาะ IP ที่ไว้ใจ
-  ใช้ Cloudflare Tunnel (ไม่ต้องเปิด port เลย)
-  จัดโครงสร้าง /DATA/docker /DATA/appdata ให้ CasaOS เนียน ๆ

บอกมาได้เลยครับ

ผมจะช่วยไล่ทีละสเต็ปแบบ ไม่หลง ไม่พัง และเอาไปใช้จริงได้ยาว ๆ 

ติดตั้ง casaos

ได้เลยครับ 

จากสถานะเครื่องตอนนี้ถือว่า พร้อมติดตั้ง CasaOS 100% (Docker OK, DATA mount แล้ว, SSH ใช้ได้)

ผมสรุปให้เป็น ขั้นตอนสั้น-ชัด-ไม่พลาด ตามสาย infra/dev นะครับ

0 เช็คของที่ต้องมี (คุณผ่านหมดแล้ว)

-  Ubuntu 24.04 LTS
-  Docker ใช้งานได้ (hello-world ผ่าน)
-  Disk /DATA mount เรียบร้อย
-  SSH เปิดแล้ว

1 ติดตั้ง curl (จำเป็น)

ตอนนี้ error เพราะยังไม่มี curl

sudo apt update

sudo apt install -y curl

เช็ค:

curl --version

2 ติดตั้ง CasaOS (คำสั่งเดียวจบ)

```
curl -fsSL https://get.casaos.io | sudo bash
```

🕒 ใช้เวลาประมาณ 1–3 นาที

มันจะ:

- ติดตั้ง service
- bind Docker
- เปิด port ที่จำเป็นให้เอง

3 เปิด port CasaOS (ถ้าใช้ UFW)

CasaOS ใช้ port **80** (HTTP) เป็นหลัก

```
sudo ufw allow 80/tcp
```

```
sudo ufw reload
```

เช็ค:

```
sudo ufw status
```

4 เข้า CasaOS

จากเครื่องอื่น หรือ browser เครื่องนี้:

http://IP_เครื่องนี้

เช่น

http://192.168.1.50



CasaOS จะให้สร้าง user ครั้งแรกเอง (ไม่เกี่ยวกับ user Ubuntu)

5 (สำคัญมาก) ตั้ง DATA directory ให้ถูก

หลังเข้า CasaOS แล้ว ไปที่:

Settings → Storage

ตั้งค่าให้เป็น:

/DATA

แนะนำโครงสร้าง:

/DATA

|—— docker
|—— appdata
|—— downloads
|—— media
|—— ollama
|—— backups

CasaOS จะใช้ path นี้เก็บทุก container → เครื่องจะ ไม่รก / ไม่พังง่าย

Port ที่ CasaOS ใช้บ่อย (เพื่อเปิดเพิ่ม)

บริการ	Port
--------	------

CasaOS UI	80
-----------	----

Docker App ตาม app	
--------------------	--

SSH	22
-----	----

Ollama	11434
--------	-------

n8n	5678
-----	------

Next.js	3000
---------	------

เปิดเพิ่มได้ที่หลัง ไม่ต้องรีบ

คำแนะนำสาย Dev / Infra (จากประสบการณ์)

-  ไม่ต้อง เปิด HTTPS ใน CasaOS
-  ไม่ต้อง expose app ตรง ๆ
-  ใช้ Cloudflare Tunnel / Nginx Proxy Manager ที่หลัง
-  ทุก app รันผ่าน Docker เท่านั้น
-  source code วางนอก CasaOS (เช่น /DATA/projects)

สรุปสถานะหลังขั้นนี้

เรื่อง

สถานะ

CasaOS

 ติดตั้งได้ทันที

Docker



Storage



/DATA

พร้อมรัน AI / ERP / Next.js



ถ้าติดตั้งเสร็จแล้ว

พิมพ์มาว่า “CasaOS เข้าได้แล้ว”

เดี๋ยวผมช่วยวาง layout ต่อให้เลย เช่น

- CasaOS + Ollama
- CasaOS + n8n
- CasaOS + Reverse Proxy
- CasaOS สำหรับโรงเรียน / server กลาง

ไปที่ละขั้นแบบมืออาชีพครับ 😊

CasaOS เข้าได้แล้ว

เยี่ยมมากครับ 🎉

CasaOS เข้าได้แล้ว = ฐานทัพพร้อมรบ 😁

ต่อจากนี้เราจะจัดเครื่องให้เป็น Dev / AI / Infra Server แบบไม่พังง่าย กัน

ผมขอพาไปที่ละสเต็ป (เลือกทำตามลำดับได้)

STEP 1 ตั้งค่า CasaOS ให้ “ถูกตั้งแต่ต้น”

เข้า CasaOS → Settings

1. Storage (สำคัญที่สุด)

ตั้งค่า Default Storage เป็น

/DATA

ถ้ายังไม่เห็น:

- Add Storage → Custom Path → /DATA

โครงสร้างแนะนำ:

/DATA

	appdata	# config container
	docker	# volumes
	projects	# source code (Next.js / Node)
	ollama	# AI models
	downloads	
	backups	

STEP 2 วางหลัก “App พวก Next.js / Node.js ไว้ที่ไหน?”

คำตอบชัด ๆ:

👉 วาง **source code** อยู่นอก CasaOS

👉 ให้ CasaOS ดูแลเฉพาะ “runtime ผ่าน Docker”

แนะนำ

- Source code:

/DATA/projects

- รันด้วย:
- Docker Compose
- หรือ App ใน CasaOS ที่ bind volume มาที่ /DATA/projects/...

❌ ไม่แนะนำ

- เขียนโค้ดใน container
- เก็บโค้ดใน /var/lib/docker

STEP 3 App แรกที่ “ควรมี” บน CasaOS

ผมเรียงลำดับให้เลย 👉

📁 1. File Manager

เอาไว้จัด /DATA สบาย ๆ

(บางเครื่องมีติดมาแล้ว)

📁 2. Portainer (คุม Docker แบบโปร)

- ดู container / logs / volumes
- แก้ปัญหาเร็วกว่า CasaOS UI

ค้นหาใน App Store: **Portainer**



3. Ollama (ถ้าจะทำ AI)

Port ที่ใช้:

11434

Path:

/DATA/ollama



4. Reverse Proxy (เลือกอย่างใดอย่างหนึ่ง)

ตัว

เหมาะกับ

Nginx Proxy Manager UI ง่าย

Cloudflare Tunnel ไม่ต้องเปิด port

แนะนำคุณ: **Cloudflare Tunnel** (ปลอดภัย + ไม่ต้องปวดหัว SSL)

STEP เปิด port ที่ “ควรเปิดจริง ๆ”

ตอนนี้ UFW ยัง inactive → ยังไม่ block อะไร

ถ้าจะเริ่มจริงจัง แนะนำ policy นี้:

```
sudo ufw allow 22/tcp    # SSH
```

```
sudo ufw allow 80/tcp    # CasaOS
```

```
sudo ufw allow 443/tcp   # HTTPS (เพื่อ)
```

```
sudo ufw enable
```

เช็ค:

```
sudo ufw status
```

STEP Template สำหรับ Next.js / Node (พร้อมใช้)

ถ้าคุณบอกผมว่า:

- Next.js 14 / 15

- Node 18 / 20 / 22
- ใช้ Prisma / MySQL / Redis ใหม่

ผมจะจัดให้:

- docker-compose.yml
- volume map → /DATA/projects
- port / env พร้อม deploy

สถานะปัจจุบัน (สวยงาม)

เรื่อง	สถานะ
Ubuntu	✓
Disk /DATA	✓
Docker	✓
CasaOS	✓
พร้อมทำ AI / ERP / Web	100

ต่อไปอยากลุยอะไรต่อดีครับ 😊

เลือกได้เลย:

- 1 วาง Next.js + API
- 2 ลง Ollama + AI server
- 3 ทำ server โรงเรียน / ERP
- 4 ตั้ง reverse proxy + domain
- 5 n8n / automation

พิมพ์เลขมาได้เลยครับ ผมจัดให้เป็นขั้นตอนต่อเนื่อง 👍

ช่วยเขียน คำสั่ง ติดตั้ง Ollama + openwebui ใน docker เดียวกัน

จัดให้ครับ 👍

นี่คือ วิธีติดตั้ง Ollama + OpenWebUI ให้อยู่ใน Docker Stack เดียวกัน

ใช้ได้ดีมากกับ CasaOS / Ubuntu Server / เครื่อง AI ส่วนตัว

แนวคิด:

- Ollama = LLM runtime
- OpenWebUI = หน้าบ้าน (ChatGPT-like UI)
- คุยกันผ่าน Docker network เดียวกัน
- เก็บ model + data ไว้ที่ /DATA (ไม่หาย ไม่พัง)

โครงสร้างโฟลเดอร์ (แนะนำ)

```
sudo mkdir -p /DATA/docker/ollama
```

```
sudo mkdir -p /DATA/ollama
```

```
sudo mkdir -p /DATA/openwebui
```

```
sudo chown -R $USER:$USER /DATA
```

สร้างไฟล์ docker-compose.yml

```
cd /DATA/docker/ollama
```

```
nano docker-compose.yml
```



docker-compose.yml

```
version: "3.8"
```

```
services:
```

```
  ollama:
```

```
    image: ollama/ollama:latest
```

```
    container_name: ollama
```

```
    restart: unless-stopped
```

```
    ports:
```

```
      - "11434:11434"
```

```
    volumes:
```

```
      - /DATA/ollama:/root/.ollama
```

```
    environment:
```

```
      - OLLAMA_HOST=0.0.0.0
```

```
  openwebui:
```

image: ghcr.io/open-webui/open-webui:main

container_name: openwebui

restart: unless-stopped

depends_on:

- ollama

ports:

- "3000:8080"

environment:

- OLLAMA_BASE_URL=http://ollama:11434

- WEBUI_NAME=CNPP AI

volumes:

- /DATA/openwebui:/app/backend/data

บันทึก: Ctrl + O → Enter → Ctrl + X

รัน Stack

docker compose up -d

เช็คสถานะ:

docker ps

เข้าใช้งาน

Service	URL
---------	-----

OpenWebUI	http://IP-เครื่อง:3000
-----------	------------------------

Ollama API	http://IP-เครื่อง:11434
------------	-------------------------

 ถ้าใช้ CasaOS

→ เปิด Browser → http://server-ip:3000

ดาวน์โหลดโมเดล (ครั้งแรก)

รันใน host หรือใน container ก็ได้

วิธีง่าย (บน host)

```
docker exec -it ollama ollama pull llama3.2
```

หรือ

```
docker exec -it ollama ollama pull gemma2:9b
```

ดูโมเดล:

```
docker exec -it ollama ollama list
```

Port ที่ใช้

Port ใช้ทำอะไร

11434 Ollama API

3000 OpenWebUI

ถ้าใช้ UFW:

```
sudo ufw allow 3000/tcp
```

```
sudo ufw allow 11434/tcp
```

ใช้กับ CasaOS (คำแนะนำ)

- ไม่ต้องลง App Ollama ซ้ำ
- ใช้ **Stack** นี้เป็นหลัก
- CasaOS = ตัวจัดการ / Dashboard
- OpenWebUI = หน้าใช้งาน AI จริง

ถ้าคุณอยากได้ขั้น Advanced ต่อ

ผมจัดให้ได้ทันที เช่น:

-  Auth / User หลายคน
-  Reverse Proxy (โดเมน + SSL)
-  GPU (NVIDIA / CUDA)
-  RAG + Vector DB
-  เชื่อม /DATA/projects เป็น Knowledge

พิมพ์ต่อว่าอยากได้ข้อไหน

หรือบอก **สเปกเครื่อง + GPU** มาได้เลยครับ 

OpenWebUI เปลี่ยน port เป็น 4000 ได้ไหม

ได้รับ  เปลี่ยนเป็น port 4000 ได้เลย ไม่มีปัญหา

แค่แก้ mapping ของ OpenWebUI เท่านั้น ตัว Ollama ไม่ต้องแตะ 

วิธีแก้ (สั้น กระชับ ทำตามนี้)

แก้ไฟล์ docker-compose.yml

```
cd /DATA/docker/ollama
```

```
nano docker-compose.yml
```

แก้ส่วน openwebui จากเดิม

ports:

```
- "3000:8080"
```

 เป็น

ports:

```
- "4000:8080"
```

 อย่าแก้ 8080

- 8080 = port ภายใน container
- 4000 = port ที่เราเปิดให้ใช้งานจากภายนอก

Restart Stack

```
docker compose down
```

```
docker compose up -d
```

เข้าใช้งานใหม่

```
http://IP-เครื่อง:4000
```

เช่น

```
http://192.168.1.50:4000
```

ถ้าใช้ UFW

```
sudo ufw allow 4000/tcp
```

ตรวจสอบเร็ว ๆ

docker ps

ควรเห็นประมาณนี้:

0.0.0.0:4000->8080/tcp openwebui

0.0.0.0:11434->11434/tcp ollama

มุมคิดแบบแอดมินนิดนึง 😊

- 3000 มักชน Next.js → เปลี่ยนเป็น 4000 ดีมาก
- ถ้านาคมีหลาย Web:
 - 4000 = OpenWebUI
 - 4100 = n8n
 - 4200 = Admin
- หรือสุดท้าย → ทำ Reverse Proxy ที่เดียว (โดเมนเดียวหลาย app)

ถ้าจะต่อไปต่อ:

- 🗝 ใส่ auth
- 🌐 ทำ https
- 🧠 ผูก RAG / DATA

บอกมาได้เลยครับ เดี่ยวจัดชุดคำสั่งให้ตรงสายคุณ 🙌

ยังไม่ได้ ลง openwebui

โอเคครับ 👍 จั๊นจัดให้แบบ ตั้งแต่ศูนย์ → ใช้งานได้จริง

ติดตั้ง Ollama + OpenWebUI (port 4000) ใน Docker ชุดเดียว

ไม่ง ไม้ช้า ไม้ต้องเดา

🧩 ภาพรวมสั้น ๆ

- Ollama → LLM engine (port 11434)
- OpenWebUI → หน้าใช้งาน (port 4000)
- ใช้ Docker Compose ชุดเดียว
- เก็บข้อมูลที่ /DATA ทั้งหมด

1 เตรียมโพลเตอร์ (ครั้งเดียว)

```
sudo mkdir -p /DATA/docker/ollama
sudo mkdir -p /DATA/ollama
sudo mkdir -p /DATA/openwebui
sudo chown -R $USER:$USER /DATA
```

2 สร้างไฟล์ docker-compose.yml

```
cd /DATA/docker/ollama
nano docker-compose.yml
```



วางเนื้อหาทั้งหมด (เวอร์ชันพร้อมใช้)

```
version: "3.8"
```

```
services:
```

```
  ollama:
```

```
    image: ollama/ollama:latest
```

```
    container_name: ollama
```

```
    restart: unless-stopped
```

```
    ports:
```

```
      - "11434:11434"
```

```
    volumes:
```

```
      - /DATA/ollama:/root/.ollama
```

```
    environment:
```

```
      - OLLAMA_HOST=0.0.0.0
```

```
  openwebui:
```

```
    image: ghcr.io/open-webui/open-webui:main
```

```
    container_name: openwebui
```

```
    restart: unless-stopped
```

depends_on:

- ollama

ports:

- "4000:8080"

environment:

- OLLAMA_BASE_URL=http://ollama:11434
- WEBUI_NAME=CNPP AI

volumes:

- /DATA/openwebui:/app/backend/data

บันทึก: Ctrl + O → Enter → Ctrl + X

3 รันติดตั้งจริง

`docker compose up -d`

ตรวจสอบ:

`docker ps`

ต้องเห็น:

- ollama
 - openwebui
-

4 เปิด Firewall (ถ้าใช้ UFW)

`sudo ufw allow 4000/tcp`

`sudo ufw allow 11434/tcp`

5 เข้าใช้งาน OpenWebUI

เปิดจากเครื่องอื่นหรือเครื่องนี้:

`http://IP-เครื่อง:4000`

ครั้งแรก:

- สร้าง user / password
- เลือก Ollama เป็น backend (มันผูกไว้ให้แล้ว)

6 โหลดโมเดล (จำเป็นครั้งแรก)

ตัวอย่างโมเดลเบา-กลาง (แนะนำ):

```
docker exec -it ollama ollama pull llama3.2
```

หรือ

```
docker exec -it ollama ollama pull gemma2:9b
```

เช็ก:

```
docker exec -it ollama ollama list
```

เช็กว่าทุกอย่างโอเคไหม

- OpenWebUI เข้าได้ที่ :4000 
 - Ollama API ตอบที่ :11434 
 - Models อยู่ที่ /DATA/ollama (ไม่หาย) 
-

ถัดไปอยากให้ผมช่วยอะไรต่อ?

เลือกได้เลยครับ:

- 1 เปิด GPU (NVIDIA / CUDA)
- 2 ตั้ง HTTPS / Domain / Cloudflare
- 3 ทำ RAG ดึงไฟล์จาก /DATA/projects
- 4 แยก user หลายคน + policy โรงเรียน
- 5 ผูก Next.js / ERP เรียก LLM

บอกมาได้เลยครับ เดี๋ยวจัดให้ต่อแบบมืออาชีพ 😁

ตรวจสอบก็ดีดี ยังไม่มีคำสั่ง ลง openwebui นะครับ

ครูใหญ่ทักถูกมากครับ 👍

ขออธิบายให้ ชัด-ตรง-ไม่กำกวม เลยนะครับ

👉 จริง ๆ แล้วมีคำสั่งติดตั้ง OpenWebUI อยู่แล้ว

แต่เป็นการติดตั้งแบบ Docker image (implicit) ไม่ใช่ apt install เลยดูเหมือน “ยังไม่ได้ลง”

ผมจะแยกให้เห็น 2 แบบชัด ๆ ครับ

✅ ความจริงที่เกิดขึ้นตอนนี้ (สำคัญ)

ในวิธีที่ผมให้ไปก่อนหน้านี้

openwebui:

image: ghcr.io/open-webui/open-webui:main

บรรทัดนี้แหละครับ = คำสั่ง “ติดตั้ง OpenWebUI”

เมื่อคุณรัน:

docker compose up -d

Docker จะทำสิ่งนี้อัตโนมัติ:

1. ดาวน์โหลด image ghcr.io/open-webui/open-webui:main
2. สร้าง container ชื่อ openwebui
3. รัน OpenWebUI ให้พร้อมใช้งาน

👉 โลก Docker ไม่มีคำว่า install แบบ apt

ทุกอย่างคือ pull image → run container

🔍 วิธี “ตรวจสอบว่า OpenWebUI ถูกติดตั้งแล้วจริงไหม”

รันคำสั่งนี้ครับ

docker images | grep open-webui

ถ้ามี แปลว่า image ถูก “ติดตั้ง” แล้ว

และดู container:

docker ps | grep openwebui

ถ้าเห็น = OpenWebUI รันอยู่จริง ✅

⚙️ ถ้าครูใหญ่ต้องการ “คำสั่งลง OpenWebUI แบบตรง ๆ ไม่อ้อม”

ผมจัดให้ครับ (ไม่ใช่ compose)

◆ ติดตั้ง OpenWebUI แบบ docker run (คำสั่งเดียว)

docker run -d \

--name openwebui \

--restart unless-stopped \

```
-p 4000:8080 \  
-e OLLAMA_BASE_URL=http://host.docker.internal:11434 \  
-v /DATA/openwebui:/app/backend/data \  
ghcr.io/open-webui/open-webui:main
```

อันนี้คือ คำสั่งติดตั้ง OpenWebUI แบบชัด ๆ 100%

แล้วเข้า:

http://IP:4000



เปรียบเทียบให้เห็นภาพ

วิธี ติดตั้ง OpenWebUI ยังไง

docker-compose image: ghcr.io/open-webui/open-webui:main

docker run docker run ... ghcr.io/open-webui/open-webui:main

apt ❌ ไม่มี (OpenWebUI ไม่แจกแบบนี้)



สรุปแบบตรงประเด็น

- ✓ OpenWebUI ไม่มี apt install
- ✓ การ “ลง” คือ pull image
- ✓ คำสั่ง docker compose up -d = ลง + รัน
- ✓ ถ้าอยากเห็นคำสั่งชัด ๆ → ใช้ docker run ด้านบนได้เลย

ถัดไปผมถามนิดเดียวจะได้จัดต่อให้ตรงใจครับ 😊



ตอนนี้ครูใหญ่อยากใช้ docker-compose (จัดเป็น stack)

หรือ docker run แยกทีละตัว เป็นหลัก?

ผมจะปรับแนวให้ตรงสไตล์ทำงานของครูใหญ่ทันทีครับ 🙏

ลงหรือยัง

สั้น ๆ ชัด ๆ เลยครับครูใหญ่ 🙌

ลงแล้ว และกำลังรันอยู่เรียบร้อย 100%

จากภาพยืนยันได้ชัดมาก:



สถานะ Container

- ollama → Up 6 minutes
- openwebui → Up 6 minutes (healthy)

แปลว่า:

- Docker ดึง image มาแล้ว
- Container ถูกสร้างแล้ว
- Service ทำงานปกติ (health check ผ่าน)

Port ที่ใช้งานอยู่ (สำคัญ)

จาก docker ps:

- OpenWebUI
 - 0.0.0.0:4000 -> 8080/tcp
 - ใช้งานได้ที่  <http://IP-เครื่อง:4000>
- Ollama
 - 0.0.0.0:11434 -> 11434/tcp
 - OpenWebUI เชื่อม Ollama ได้เรียบร้อย

เช็คซ้ำแบบมั่นใจ

ลองจากเครื่องอื่นในวงเดียวกัน:

<http://IP-เครื่อง:4000>

ควรเห็นหน้า OpenWebUI ให้สร้างผู้ใช้ได้ทันที

หรือดู log:

`docker logs openwebui --tail 20`

สรุปสถานะปัจจุบัน

รายการ	สถานะ
Ollama	 Running
OpenWebUI	 Running (healthy)
Port 4000	 เปิดใช้งาน
พร้อมใช้งานจริง	

ถัดไปอยากทำอะไรต่อดีครับ 

- ดึงโมเดล (เช่น llama3, deepseek, typhoon)
- ตั้ง DATA ให้ OpenWebUI / Ollama แยกชัด
- ผูก CasaOS App tile
- เปิดใช้งานจากนอกเครือข่าย (Reverse proxy / Cloudflare)

เลือกมาได้เลย เดี่ยวจัดให้เป็นขั้น ๆ ครับ 🙌

map ออกนอก วง wifi ได้หรือยัง? url ของระบบ <http://203.172.184.47:4000/> ไม่ได้

คำตอบตรงประเด็นเลยครับครูใหญ่ 👉

ตอนนี้ “ยังออกนอกวง Wi-Fi ไม่ได้” และสาเหตุไม่ได้อยู่ที่ Docker หรือ OpenWebUI แล้วครับ แต่อยู่ที่ “ชั้นเครือข่ายภายนอก” 🧱

เรามาไล่แบบเป็นเหตุเป็นผล (ไม่มั่ว ไม่เดา) กันทีละขั้นนะครับ

1 สถานะปัจจุบัน (จากข้อมูลที่ให้มา)

- OpenWebUI รันอยู่จริง
- Port mapping ถูกต้อง
→ 0.0.0.0:4000 -> 8080
- เข้าได้จาก ใน LAN (วง Wi-Fi) ✅
- เข้า <http://203.172.184.47:4000> ไม่ได้ ❌

👉 แปลว่า ติดที่ Network / ISP / Router 100%

2 เช็กด่วน: เครื่องนี้ “ได้ Public IP จริงไหม”

รันบนเครื่อง Ubuntu:

```
curl ifconfig.me
```

แล้วดูว่า IP ที่ได้ ตรงกับ 203.172.184.47 ไหม

ผลลัพธ์แปลความหมาย

- ✅ ตรงกัน → ไปข้อ 3
- ❌ ไม่ตรง → คุณอยู่หลัง CGNAT ของ ISP (สำคัญมาก)

ถ้าไม่ตรง = ต้องให้เปิด port ยังไง ก็เข้าไม่ได้จากอินเทอร์เน็ต

3 ถ้า IP ตรงกัน → เช็ก Router (สำคัญที่สุด)

ต้องมี Port Forwarding บน Router

ต้องตั้งค่า:

ค่า ค่า

External Port 4000

Internal IP IP เครื่อง Ubuntu (เช่น 192.168.1.x)

Internal Port 4000

Protocol TCP

⚠ เปิด UFW อย่างเดียวไม่พอ

Router ไม่รู้จัก Docker / CasaOS

4 เช็ก Firewall เครื่อง (UFW)

ตอนนี้คุณ allow แล้ว แต่ยังไม่ enable

เช็ก:

```
sudo ufw status
```

ถ้ายังขึ้น inactive → ให้ทำ:

```
sudo ufw enable
```

แล้วอนุญาต port:

```
sudo ufw allow 4000/tcp
```

```
sudo ufw reload
```

5 ถ้าเจอกรณีหนัก: ISP ใช้ CGNAT (พบบ่อยมากในไทย)

ถ้า curl ifconfig.me ไม่ตรงกับ 203.172.184.47

แปลว่า:

❌ เปิด port ตาย ยังไงก็เข้าไม่ได้

ทางออก (เลือก 1)

✅ แนะนำที่สุด

- ใช้ Cloudflare Tunnel

- ไม่ต้องเปิด port
- ปลอดภัย
- ใช้กับ OpenWebUI ดีมาก

ทางเลือกอื่น

- ขอ ISP เปิด Public IP (บางเจ้าเสียเงิน)
- ใช้ VPS + Reverse Proxy
- ใช้ Tailscale / ZeroTier

6 คำตอบสั้นที่สุด

? map ออกนอกวง wifi ได้หรือยัง

✗ ยังไม่ได้

ไม่ใช่เพราะ Docker หรือ OpenWebUI
แต่เพราะ Router / ISP / CGNAT

7 ถ้าครูใหญ่บอกว่า:

“ไม่อยากยุ่ง router / ISP เลย”

ผมแนะนำทันที 👉 Cloudflare Tunnel (5 นาทีจบ)

เดี๋ยวผมจัดคำสั่งให้แบบ

- ใช้กับ Docker
- ได้ URL <https://xxx.trycloudflare.com>
- เอาไปใช้จริงได้เลย

👉 บอกผมหน่อยครับ

ผล curl ifconfig.me ออกมาเป็นอะไร

เดี๋ยวผมฟันธงแนวทางให้ตรงจุด ไม่เสียเวลาครูใหญ่แน่นอน 😊

Ok ได้แล้ว ต่อไปเตรียมการลง ComfyUI ใน docker run เฉพาะ เวลา ทำงาน

เยี่ยมเลยครับครูใหญ่ 🔥

งั้นเรามาทำ ComfyUI แบบ “เรียกใช้เฉพาะเวลาทำงาน” ให้สะอาด แข็งแรง และไม่กินเครื่องตอนว่าง ตามแนวคิดสายโปรเลยครับ

ผมจะให้ เป็น **docker run** (ไม่ auto start)

เหมาะมากกับ CasaOS / Server งาน AI

แนวคิดก่อนลง (สำคัญมาก)

- ❌ ไม่รันตอนบูต
- ✅ รันเฉพาะตอนต้องสร้างภาพ
- ✅ แยก model / output / custom_nodes ชัดเจน
- ✅ พร้อมต่อ Ollama / Workflow อื่นภายหลัง
- ✅ หยุดแล้ว RAM / VRAM คืนทันที

1 เตรียมโฟลเดอร์ (แนะนำโครงนี้)

สมมติใช้ดิสก์ /DATA

```
mkdir -p /DATA/comfyui/{models,output,input,custom_nodes}
```

โครงสร้างจะเป็น:

/DATA/comfyui/

├── models/

├── output/

├── input/

└── custom_nodes/

2 เลือก Image (แนะนำ)

ถ้า ไม่มี GPU

ghcr.io/comfyanonymous/comfyui:latest

ถ้า มี NVIDIA GPU

ghcr.io/comfyanonymous/comfyui:latest-nvidia

⚠️ ถ้ามี GPU ต้องลง nvidia-container-toolkit แล้วยก่อน

3 คำสั่ง docker run (ใช้งานจริง)

แบบ CPU

```
docker run -d \  
  --name comfyui \  
  -p 8188:8188 \  
  -v /DATA/comfyui/models:/app/models \  
  -v /DATA/comfyui/output:/app/output \  
  -v /DATA/comfyui/input:/app/input \  
  -v /DATA/comfyui/custom_nodes:/app/custom_nodes \  
  ghcr.io/comfyanonymous/comfyui:latest
```

แบบ GPU (NVIDIA)

```
docker run -d \  
  --name comfyui \  
  --gpus all \  
  -p 8188:8188 \  
  -v /DATA/comfyui/models:/app/models \  
  -v /DATA/comfyui/output:/app/output \  
  -v /DATA/comfyui/input:/app/input \  
  -v /DATA/comfyui/custom_nodes:/app/custom_nodes \  
  ghcr.io/comfyanonymous/comfyui:latest-nvidia
```

เข้าใช้งาน:

<http://<IP-เครื่อง>:8188>

ใช้งานแบบ “เปิด-ปิดตามงาน”

เริ่มทำงาน

```
docker start comfyui
```

เลิกใช้

```
docker stop comfyui
```

 ลบ (ถ้าจะลงใหม่)

docker rm comfyui

Model / output ไม่หาย เพราะอยู่ใน /DATA

5 โหลด Model (วางตรง ๆ ได้เลย)

เช่น:

```
cp *.safetensors /DATA/comfyui/models/checkpoints/
```

หรือโหลดจาก HuggingFace แล้ววางตรงนี้

6 ต่อยอดระดับครูใหญ่ (ไอเดียถัดไป)

ถ้าจะไปต่อ ผมแนะนำลำดับนี้ 🙋

1. 🔌 ComfyUI + API mode (เรียกจาก Web / Next.js)
2. 🧠 LLM → เขียน prompt → ส่งเข้า ComfyUI
3. 🐳 docker-compose เฉพาะช่วง workshop
4. 🧩 Custom Nodes สำหรับการศึกษ / เด็ก
5. 🔒 เปิดเฉพาะใน LAN / Cloudflare Tunnel

7 คำถามชวนคิดนิดเดียว 😊

- จะใช้ ComfyUI คนเดียว หรือ หลายคนพร้อมกัน
- จะเน้น ภาพเด็ก / pastel / anime / realistic
- อยากให้ Ollama → สั่ง ComfyUI อัตโนมัติไหม

ถ้าครูใหญ่ตอบมา 1-2 ข้อ

ผมจะจัด Workflow + Prompt Pipeline + Node set ให้ตรงเป้าเลยครับ 🚀

ลองย้อนไปดูข้อมูลเก่า เครื่องผมมี gpu nvidia

ได้เลยครับ (และดีมาก—มี NVIDIA = ชีวิตง่ายขึ้นเยอะ 😊)

งั้นเราจัด ComfyUI แบบใช้ GPU ให้ตรง ๆ เลย + เช็คให้ชัวร์ว่า “Docker เห็นการ์ดจอ” ก่อน (ไม่งั้น ComfyUI จะวิ่ง CPU แล้วเราจะงงกันทั้งหมู่บ้าน)

0) เช็คก่อนว่า Ubuntu เห็น NVIDIA ไหม

nvidia-smi

ถ้าเห็นตาราง GPU/Driver แปลว่าไดรเวอร์โอเค

1) เช็คว่า Docker เห็น GPU ใหม่ (สำคัญสุด)

ติดตั้ง/ตั้งค่า NVIDIA Container Toolkit (ถ้ายังไม่เคยทำ)

```
sudo apt update
```

```
sudo apt install -y nvidia-container-toolkit
```

```
sudo nvidia-ctk runtime configure --runtime=docker
```

```
sudo systemctl restart docker
```

ทดสอบ:

```
docker run --rm --gpus all nvidia/cuda:12.3.2-base-ubuntu22.04 nvidia-smi
```

ถ้าใน container เห็น nvidia-smi = ผ่าน 

2) เตรียมโฟลเดอร์เก็บโมเดล/งาน (ไว้ใน /DATA ตามที่ทำไว้)

```
mkdir -p /DATA/comfyui/{models,output,input,custom_nodes}
```

```
mkdir -p /DATA/comfyui/models/{checkpoints,vae,loras,controlnet,clip,upscale_models}
```

3) รัน ComfyUI (GPU) แบบ “เปิดเฉพาะเวลาทำงาน”

ตัวเลือก A: ใช้ image ที่มี ComfyUI พร้อม (ง่ายสุด)

```
docker run -d \
```

```
--name comfyui \
```

```
--gpus all \
```

```
-p 8188:8188 \
```

```
-v /DATA/comfyui/models:/app/models \
```

```
-v /DATA/comfyui/output:/app/output \
```

```
-v /DATA/comfyui/input:/app/input \
```

```
-v /DATA/comfyui/custom_nodes:/app/custom_nodes \
```

```
ghcr.io/comfyanonymous/comfyui:latest-nvidia
```

เข้าใช้งาน:

- <http://203.172.184.47:8188>

หยุดเมื่อไม่ใช่:

```
docker stop comfyui
```

เริ่มใหม่:

```
docker start comfyui
```

ดู log:

```
docker logs -f comfyui
```

4) ให้เข้าจากนอกวง Wi-Fi ได้ (เหมือนที่ทำกับ OpenWebUI)

ถ้าต้องการให้เรียกจากภายนอกได้:

- เปิดพอร์ตที่เครื่อง/ไฟร์วอลล์

```
sudo ufw allow 8188/tcp
```

```
sudo ufw status
```

ถ้า ufw ยัง inactive อยู่ ก็ไม่ต้องทำก็ได้ แต่เปิดไว้ก็ดี (ปลอดภัยกว่า)

5) เช็คว่า ComfyUI ใช้ GPU จริงไหม

หลังรันแล้วให้ดู log:

```
docker logs -f comfyui
```

หรือเข้า UI แล้วลองรัน workflow สักอัน—ถ้าใช้ GPU จะเร็วแบบ “คนละโลก”

หมายเหตุสั้น ๆ ที่กันพลาด

- ถ้า docker run ... --gpus all แล้ว error = toolkit ยังไม่พร้อม
- ถ้า ComfyUI รันได้แต่ช้า = มักจะตกไปใช้ CPU (เช็กขั้นตอน 1)

ถ้าส่งรูป nvidia-smi กับผลคำสั่งทดสอบ docker run ... nvidia-smi มา 1 รูป

ผมจะบอกได้เลยว่า “ผ่านทุกขั้น” หรือสะดุดตรงไหนครับ 🙌

อ้อ ลง driver card gpu ก่อนก็ดี

ใช่เลยครับ 👍

ลำดับที่ถูกต้อง = ลง NVIDIA Driver ก่อน → ค่อยไป Docker / ComfyUI / Ollama

คิดถูกมาก ไม่งั้นจะเสียเวลาย้อนแก้ทีหลัง 😊

ผมสรุปให้แบบ **สั้น-ชัด-ปลอดภัย** ตาม Ubuntu 24.04 ที่คุณใช้อยู่

ขั้นที่ 1: ตรวจสอบระบบเห็นการ์ด NVIDIA ไหม

`lspci | grep -i nvidia`

ถ้าเห็นชื่อการ์ด = ฮาร์ดแวร์โอเค

ขั้นที่ 2: ดูไดรเวอร์ที่ Ubuntu แนะนำ (วิธี safest)

`ubuntu-drivers devices`

จะเห็นประมาณนี้:

driver : nvidia-driver-550 (recommended)

ให้เลือกตัวที่ขึ้นว่า recommended เท่านั้น

(ตอนนี้มักเป็น 535 หรือ 550)

ขั้นที่ 3: ติดตั้ง NVIDIA Driver

ตัวอย่าง (สมมติ recommended คือ 550)

`sudo apt update`

`sudo apt install -y nvidia-driver-550`

เสร็จแล้ว **ต้อง reboot**

`sudo reboot`

ขั้นที่ 4: เช็คหลัง reboot (สำคัญมาก)

`nvidia-smi`

ถ้าเห็นตาราง GPU / Driver Version / Memory

= 🎉 **ไดรเวอร์พร้อมใช้งาน**

ขั้นที่ 5: ต่อด้วย Docker + GPU (ที่คุณทำไว้เกือบครบแล้ว)

หลังจาก driver พร้อมแล้ว ค่อยทำ/เช็คสิ่งนี้:

`sudo apt install -y nvidia-container-toolkit`

`sudo nvidia-ctk runtime configure --runtime=docker`

`sudo systemctl restart docker`

ทดสอบ:

```
docker run --rm --gpus all nvidia/cuda:12.3.2-base-ubuntu22.04 nvidia-smi
```

ถ้า container เห็น GPU = ผ่านทุกด้าน

สรุปลำดับทองคำ (จำอันนี้อันเดียวพอ)

1.  ติดตั้ง NVIDIA Driver บน Host
2.  reboot
3.  nvidia-smi บนเครื่องจริง
4.  NVIDIA Container Toolkit
5.  Docker + --gpus all
6.  ComfyUI / Ollama / OpenWebUI

ถ้าคุณอยากให้ผม เลือก driver ให้ตรงรุ่นการ์ดเป๊ะ ๆ

ส่งผล lspci | grep -i nvidia มาได้เลย

ผมจะบอกเลยว่า ควรลงเบอร์ไหนดีที่สุด 🙌

ดีมากครับ ภาพนี้คือ คำตอบครบแล้ว 👍

ผมสรุปให้แบบ “ฟันธง” เลย ว่าควรลงตัวไหน และทำไม

สรุปจากผล ubuntu-drivers devices

บรรทัดสำคัญที่สุดคืออันนี้ 👉

driver : nvidia-driver-580-open - distro non-free recommended

👉 คำว่า recommended คือของจริง

Ubuntu ทดสอบมาแล้วว่า เสถียรที่สุดกับการ์ด + kernel รุ่นนี้

คำตอบสั้น ๆ (เอาไปทำเลย)

✅ ให้ติดตั้งตัวนี้

sudo apt update

```
sudo apt install -y nvidia-driver-580-open
```

```
sudo reboot
```

อธิบายแบบคนทำ Docker / AI / ComfyUI (ทำไมเลือกตัวนี้)

◆ 580-open คืออะไร

- Driver รุ่นใหม่ (R580)
- รองรับ CUDA / AI / Docker GPU เต็มรูปแบบ
- เป็น **Open Kernel Module** → ทำงานเข้ากับ kernel ใหม่ (24.04) ดีมาก
- Canonical แนะนำเอง (ไม่ใช่เดา)

◆ ไม่ควรเลือกอะไร

ตัว	เหตุผล
xserver-xorg-video-nouveau	✗ ช้า / ไม่เหมาะ AI
570	รุ่นเก่าไปนิด
server	เอาไว้ทำ server display-less + GRID
580 (non-open)	ใช้ได้ แต่ open ดีกว่าสำหรับ kernel ใหม่

หลัง reboot ต้องเช็ค 2 อย่างนี้เท่านั้น

```
nvidia-smi
```

ต้องเห็น:

- Driver Version: **580.xx**
- GPU Name
- Memory

แล้วต่อด้วย

```
lsmod | grep nvidia
```

ต้องมี nvidia, nvidia_uvm

หลังจากนี้ (ขั้นถัดไปที่เราจะทำต่อ)

เมื่อ driver ผ่านแล้ว เราจะไป:

1.  NVIDIA Container Toolkit (ถ้ายัง)
 2.  ทดสอบ docker run --gpus all
 3.  ComfyUI (docker run แบบ **on-demand**)
 4.  Map volume ไป /DATA
-

ถ้าคุณอยากให้ผม:

- เขียน docker run ComfyUI (GPU, on-demand, ไม่ auto start)
- หรือ docker-compose แยก Ollama / OpenWebUI / ComfyUI เป็นระเบียบ

บอกได้เลยครับ

ตอนนี้เครื่องคุณ “พร้อมทำ AI เต็มระบบแล้ว” 

ภาพนี้ ยืนยันชัดแล้วครับ 🙌

เครื่องคุณ ยังไม่ได้ลง driver แต่ Ubuntu บอกทางที่ถูกต้องมาให้เรียบร้อยแล้ว

สรุปจากหน้าจอ (พื้นธง)

บรรทัดนี้คือคำตอบสุดท้าย 📌

nvidia-driver-580-open - distro non-free recommended

➡️ แปลตรงตัว:

Canonical แนะนำให้ใช้ตัวนี้กับเครื่องคุณ

คำสั่งที่ควรใช้ (ทำตามนี้ได้เลย)

sudo apt update

sudo apt install -y nvidia-driver-580-open

sudo reboot

🕒 ระหว่างติดตั้งอาจใช้เวลาสักครู (โหลด kernel module + firmware)

หลัง reboot ต้องตรวจ 2 อย่างนี้

1) เช็ค ว่า driver ขึ้นจริง

nvidia-smi

ควรเห็น:

- Driver Version: 580.xx
- เห็นชื่อ GPU
- ไม่มี error

2) เช็ค module

lsmod | grep nvidia

ควรมีอย่างน้อย:

- nvidia
- nvidia_uvm
- nvidia_drm

ถ้า 2 ข้อนี้ผ่าน = GPU พร้อมใช้งาน 100%

ทำไมเลือก 580-open (สำหรับสาย AI / Docker / ComfyUI)

- รองรับ CUDA + Docker GPU เต็มรูปแบบ
- เข้ากับ Ubuntu 24.04 + kernel ใหม่ ดีที่สุด
- เสถียรกว่า nouveau หลายเท่า
- Canonical recommend เอง ไม่ต้องลังเล

ลำดับงานถัดไป (ผมเรียงให้แล้ว)

1.  ติดตั้ง NVIDIA Driver (กำลังทำอยู่ตอนนี้)
2.  ติดตั้ง NVIDIA Container Toolkit
3.  ทดสอบ docker run --gpus all nvidia/cuda
4.  ลง ComfyUI (docker run แบบ on-demand)
5.  Map /DATA ใช้งานจริง

ถ้าลงเสร็จแล้ว

ส่งผล nvidia-smi มาได้เลย

เดี๋ยวผมพาคุณไปขึ้น Docker + GPU + ComfyUI ต่อทันที 

หา driver RTX 5070 Ti + VRAM ~16GB

โอเค ตรงนี้ขอสรุปให้ ชัดแบบไม่อ้อม เลยนะครับ 

(ดูจากบริบทเครื่องคุณ + Ubuntu 24.04 + งาน AI / Docker / ComfyUI)

 RTX 5070 Ti (VRAM ~16GB) ใช้ driver อะไร?

คำตอบเดียวที่ถูกต้องตอนนี้คือ

NVIDIA Driver 580 (Open หรือ Server ก็ได้ แต่แนะนำ Open)

 เหตุผล (สำคัญมาก)

RTX 5070 Ti = สถาปัตยกรรม Blackwell (รุ่นใหม่มาก)

→ ต้องใช้ driver สาขา 57x / 58x ขึ้นไปเท่านั้น

Driver	ใช้ได้ไหม	หมายเหตุ
535 / 545	✗ ไม่ได้	เก่าเกิน
550	✗ ไม่แนะนำ	รองรับไม่ครบ
570	⚠ พอได้	ยังไม่ optimal
580-open	✓ ดีที่สุด	Canonical แนะนำ
580-server	✓ ได้	เน้น server/headless



ตัวที่ควรใช้ (ฟันธง)

nvidia-driver-580-open

เหมาะกับ:

- Ubuntu 24.04
- RTX 50-series
- AI / Stable Diffusion / ComfyUI
- Docker + GPU
- Ollama / LLM / CUDA



VRAM 16GB → ทำอะไรได้บ้าง

- Stable Diffusion XL (SDXL) สบาย
- ComfyUI workflow ใหญ่ ๆ
- LLM 13B–30B (บางตัว 34B Q4)
- ControlNet / AnimateDiff
- Multi-container GPU (แบ่งใช้ได้)



คำสั่งติดตั้ง (ซ้ำอีกครั้ง)

```
sudo apt update
```

```
sudo apt install -y nvidia-driver-580-open
```

```
sudo reboot
```

หลัง reboot:

nvidia-smi

ควรเห็น:

- GPU: RTX 5070 Ti
- Driver Version: 580.xx
- VRAM ~16384 MiB
- No error



สำคัญต่อ Docker / ComfyUI (อย่าข้าม)

หลัง driver ผ่านแล้ว **ต้องลง NVIDIA Container Toolkit**

ไม่งั้น Docker จะไม่เห็น GPU

ขั้นถัดไป (ผมเตรียมให้):

```
sudo apt install -y nvidia-container-toolkit
```

```
sudo nvidia-ctk runtime configure --runtime=docker
```

```
sudo systemctl restart docker
```

ทดสอบ:

```
docker run --rm --gpus all nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi
```

ถ้าเห็น GPU = พร้อมลุย 🚀



ต่อจากนี้ผมแนะนำ roadmap แบบมืออาชีพ

1. NVIDIA 580-open (กำลังทำ)
2. NVIDIA Container Toolkit
3. ComfyUI (docker run เปิดเฉพาะเวลาทำงาน)
4. Map /DATA/models ใช้ VRAM + disk คุ่ม
5. Optimized workflow สำหรับ RTX 5070 Ti

ถ้าคุณอยากให้ผม



จัด docker run / docker-compose สำหรับ ComfyUI + GPU โดยเฉพาะ RTX 5070 Ti

บอกได้เลยครับ ผมจัดให้ตรงเครื่องคุณจริง ๆ ไม่ generic 😊

เยี่ยมมากครับ 🙌 ภาพนี้ยืนยันได้ชัดเจนแล้วว่า

✅ สถานะปัจจุบัน (ผ่านด่านใหญ่แล้ว)

- nvidia-driver-580-open
- เวอร์ชัน: 580.95.05 (Ubuntu 24.04.3)
- ระบบบอกว่า **newest version** → ถูกต้อง 100%
- ตรงรุ่นกับ RTX 5070 Ti (Blackwell, VRAM ~16GB)

👉 ด่าน Driver = ผ่านฉลุย

ตอนนี้เครื่องคุณอยู่ในสถานะ

“พร้อมเป็น AI / GPU Server เต็มตัว” แล้ว 😊

🔴 ขั้นตอนสำคัญมาก (ห้ามข้าม)

1 reboot เพื่อให้ kernel + driver จับกันจริง

sudo reboot

2 เช็ก GPU หลัง reboot

nvidia-smi

ถ้าถูกต้อง คุณควรเห็นประมาณนี้:

- GPU: RTX 5070 Ti
- Driver Version: 580.95.xx
- VRAM: ~16384 MiB
- No error / No fallback

ถ้าเห็นแบบนี้ = จบงานฝั่ง OS 🎯

🔥 ขั้นต่อไปสำหรับ Docker / ComfyUI / Ollama (สำคัญ)

ตอนนี้ Driver มีแล้ว แต่ Docker ยัง “ไม่เห็น GPU”

ต้องลง NVIDIA Container Toolkit

3 ติดตั้ง NVIDIA Container Toolkit

sudo apt install -y nvidia-container-toolkit

```
sudo nvidia-ctk runtime configure --runtime=docker
```

```
sudo systemctl restart docker
```

ทดสอบว่า Docker เห็น GPU จริง

```
docker run --rm --gpus all nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi
```

ถ้า output เหมือน nvidia-smi บนเครื่อง

Docker + GPU = พร้อมใช้งาน

จากตรงนี้ เราไปได้ไกลมาก

ด้วย RTX 5070 Ti + 16GB VRAM คุณสามารถ:

-  ComfyUI (SDXL / ControlNet / AnimateDiff)
-  Ollama (13B–30B models)
-  OpenWebUI + GPU
-  แยก container ทำงานเป็นชุด ๆ
-  เปิดเฉพาะตอนใช้ (ประหยัดไฟ/อายุการ์ด)

soon ถัดไปผมแนะนำ (เลือกได้)

1.  **docker run ComfyUI (GPU) แบบเปิดเฉพาะเวลาทำงาน**
2.  โครงสร้าง /DATA/models แשרระหว่าง Ollama + ComfyUI
3.  ComfyUI workflow ที่ optimize สำหรับ VRAM 16GB
4.  เปิด ComfyUI ออกนอกวง LAN แบบปลอดภัย
5.  ตั้ง preset สำหรับ SDXL / Anime / Photoreal

บอกผมได้เลยว่า

จะไปต่อ “ComfyUI” หรือ “Optimize Ollama” ก่อน

ผมจัดคำสั่งให้ตรงเครื่องคุณทีละขั้นครับ 

1 ผมไม่อยากเข้าหน้า gui ubuntu 2 ยังติดต้อจาก ข้างนอกเข้ามาในวงไม่ได้ แก๊ไขหน่อย

ได้เลยครับ 👍 เรามา จัดเครื่องให้เป็น Headless Server เต็มรูปแบบ และ เข้าจากข้างนอกได้จริง ทีละประเด็นแบบไม่เร่ง ไม่ต้องแตะ GUI Ubuntu อีกต่อไป

เป้าหมาย

1. ❌ ไม่ต้องเข้า หน้า GUI Ubuntu อีก
 2. 🌐 เข้าจากข้างนอกวง (WAN / Internet) ได้จริง
 3. 💬 ใช้เครื่องนี้เป็น AI / Docker / ComfyUI / Ollama Server
-

ข้อ 1: ไม่อยากเข้าหน้า GUI Ubuntu (Headless)

◆ วิธีที่ถูกต้อง (แนะนำ)

เปลี่ยนเครื่องให้บูตเข้า Text mode (CLI only)

```
sudo systemctl set-default multi-user.target
```

```
sudo reboot
```

หลัง reboot:

- ❌ ไม่มีหน้า Desktop
-  เข้าผ่าน SSH อย่างเดียว
- ประหยัด RAM / VRAM / เสถียรกว่า

ถ้าวันไหนอยากกลับ GUI:

```
sudo systemctl set-default graphical.target
```

```
sudo reboot
```

(ทางเลือก) ถอน Desktop ออกเลย (ถ้าอยากคลีน)

ถ้ามั่นใจว่า ไม่ใช้ GUI อีกแล้ว

```
sudo apt purge ubuntu-desktop -y
```

```
sudo apt autoremove -y
```

อันนี้ optional มาก ไม่จำเป็นต้องทำก็ได้

ข้อ 2: เข้าจาก “ข้างนอกวง” ไม่ได้ (แก้ไขจบจริง)

ตอนนี้คุณทำได้แค่ ใน LAN

เราต้องเช็ค 3 ชั้น (สำคัญมาก):

ชั้นที่ 1: เครื่อง Ubuntu (Firewall)

จากรูปก่อนหน้า:

- UFW = inactive 
(แปลว่าเครื่องไม่บล็อก port)

แต่เพื่อความชัวร์ แนะนำตั้งค่าให้ชัด:

```
sudo ufw allow 22/tcp
```

```
sudo ufw allow 4000/tcp
```

```
sudo ufw allow 8188/tcp # เพื่อ ComfyUI
```

```
sudo ufw enable
```

```
sudo ufw status
```

ชั้นที่ 2: Router / Modem (ตัวการหลัก 90%)

สาเหตุที่ WAN เข้าไม่ได้ = ยังไม่ได้ Port Forward

ต้องเข้า Router แล้วทำ Port Forward แบบนี้ 

Service	External Port	Internal IP	Internal Port
SSH	22	192.168.x.x	22
OpenWebUI	4000	192.168.x.x	4000
ComfyUI	8188	192.168.x.x	8188

 **Internal IP** = IP เครื่อง Ubuntu

เช็คด้วย:

```
ip a
```

 แนะนำให้ตั้ง DHCP Reservation ให้ IP เครื่องนี้ “ไม่เปลี่ยน”

ชั้นที่ 3: เช็ค IP ที่ใช้เป็น Public จริงไหม

บน Ubuntu:

curl ifconfig.me

ถ้า IP:

- ❌ ขึ้นต้น 10.x / 100.64.x / 172.16-31 / 192.168.x

→ แปลว่า ติด CGNAT (ของ ISP)

➡ Port forward ทำยังไงก็เข้าไม่ได้ ❌

ถ้า:

- ✅ เป็น IP สาธารณะจริง

→ ใช้ WAN ได้เลย

🧠 ถ้า “ติด CGNAT” (เจอบ่อยมากในไทย)

มี 3 ทางเลือก:

1️⃣ ทางเลือกที่ดีที่สุด

- ขอ Public IP / Static IP จาก ISP (เดือนละนิดเดียว)

2️⃣ ใช้ Cloudflare Tunnel / Tailscale (ไม่ต้องเปิด port)

- ปลอดภัย
- ไม่ต้องยุ่ง router

3️⃣ ทำ Reverse Proxy ผ่าน VPS

- ขั้นสูง (ไว้ค่อยทำ)

ถ้าคุณอยากได้แนว ไม่เปิด port แต่เข้าได้จากทุกที่

ผมแนะนำ Cloudflare Tunnel มาก 👍

➡ soon ผมขอถาม 2 ข้อสั้น ๆ เพื่อพาไปต่อให้ตรงที่สุด

ตอบแค่ตัวเลขก็ได้ครับ:

1 curl ifconfig.me ได้ IP อะไร

2 Router ที่ใช้ เป็นของอะไร (NT / AIS / True / 3BB / อื่น ๆ)

จากนั้นผมจะบอกได้ทันทีว่า:

- เปิด port ต่อได้เลย ✅

หรือ

- ควรใช้ Cloudflare / Tailscale แบบไหน 🚀

ดีมากครับ ภาพนี้ “ฟ็อกซ์” แล้ว 🐼
สรุปสั้นก่อน แล้วค่อยแก้ทีละขั้นแบบไม่อ้อม

✅ สถานะปัจจุบัน (จากรูป)

ฝั่งเครื่อง Ubuntu = ถูกต้อง 100%

- UFW = เปิดแล้ว
- Port ที่จำเป็น ALLOW ครบ
(22, 4000, 8188, 8080, 11434 ฯลฯ)
- Docker / OpenWebUI / Ollama = ทำงาน

👉 เพราะฉะนั้น ปัญหาไม่อยู่ที่เครื่องนี้แล้ว

❌ ปัญหาจริง: “เข้าจากข้างนอกไม่ได้”

เหลืออยู่แค่ 2 ตัวการเท่านั้น

1. Router / Modem
2. ISP (CGNAT) ← ตัวนี้โอกาสสูงมาก

🔍 เช็กให้ชัดภายใน 30 วินาที (ขอให้ทำ)

รันคำสั่งนี้บนเครื่อง Ubuntu:

```
curl ifconfig.me
```

แล้วดู IP ที่ได้

🧠 วิเคราะห์ผลลัพธ์

❌ ถ้า IP เป็นพวกนี้

- 10.x.x.x
- 100.64.x.x – 100.127.x.x
- 172.16.x.x – 172.31.x.x
- 192.168.x.x

สรุปทันที:

👉 คุณอยู่หลัง CGNAT ของ ISP

👉 เปิด port ยังไงก็เข้าไม่ได้จาก Internet ❌

จากบริบทก่อนหน้านี้ ผมเดาว่าเป็น NT / เน็ตองค์กร / ภาครัฐ

→ ติด CGNAT เกือบ 100%

✅ ถ้า IP เป็น Public จริง

เช่น 203.xxx.xxx.xxx

→ ต้องไปตั้ง Port Forward ที่ Router เท่านั้น

❌ วิธีแก้ “แบบจบจริง” (เรียงตามความเหมาะสมกับคุณ)

👉 วิธีที่เหมาะสมที่สุดสำหรับเรื่องนี้ (แนะนำมาก)

👉 Cloudflare Tunnel

- ไม่ต้องเปิด port
- ไม่ต้องยุ่ง router
- ใช้งานจากที่ไหนก็ได้
- เหมาะกับ OpenWebUI / ComfyUI / Dashboard

ตัวอย่าง:

<https://ai.yourschool.or.th> → <http://localhost:4000>

ปลอดภัยกว่าเปิด port ตรง ๆ ด้วยซ้ำ

ถ้าบอกว่า “เอาวิธีนี้”

ผมจะให้คำสั่งติดตั้ง + config ให้ครบใน 10 นาที

👉 ทางเลือกง่ายอีกตัว

👉 Tailscale

- เหมาะกับ “เครื่องส่วนตัว / ทีมเล็ก”
 - SSH / Web เข้าได้เหมือนอยู่ LAN เดียวกัน
-



ทางสายดิบ (ไม่แนะนำถ้าไม่จำเป็น)

- ขอ Public IP / Static IP จาก ISP
- หรือเช่า VPS ทำ reverse tunnel



สรุปชัด ๆ แบบไม่ต้องเดา

- เครื่อง Ubuntu:  ผ่าน
- Docker / Port / Firewall:  ผ่าน
- ติดอยู่ที่ ISP / CGNAT 99%



ขอคำตอบสั้น ๆ 2 ข้อ แล้วผมจัดให้ตรงจุดทันที

1

curl ifconfig.me ได้ IP อะไร

2

อินเทอร์เน็ตที่ใช้: NT / AIS / True / 3BB / องค์กร

แล้วผมจะบอกเลยว่า

“ไป Cloudflare แบบไหน / คำสั่งไหน / ใช้โดเมนยังไง”

เยี่ยมมากครับ 👍 ภาพนี้คือ คำตอบสุดท้ายที่สำคัญมาก

curl ifconfig.me

→ 203.172.184.47

👉 คุณมี Public IPv4 จริง 100%

👉 ไม่ติด CGNAT

👉 ปัญหาเหลือแค่ “การตั้งค่าระบบ” ไม่ใช่เน็ตแล้ว

ต่อไปแก้ 2 เรื่องตามที่คุณขอ แบบตรงจุด 🛠️

✅ เป้าหมายของคุณ

1. ❌ ไม่อยากเข้า GUI Ubuntu (ใช้แบบ server ล้วน)
2. 🌐 อยากให้คนข้างนอกเข้าระบบได้ (SSH / Web / Docker)

ผมสรุปเป็น “ทำทีละคำสั่ง” ให้เลย

🧱 1) ปิด GUI Ubuntu ให้เป็น Server Mode ถาวร

ตอนนี้เครื่องคุณยัง boot เข้า GNOME อยู่

◆ เปลี่ยน default target เป็นแบบ server

```
sudo systemctl set-default multi-user.target
```

◆ ปิด GUI ทันที (ไม่ต้อง reboot)

```
sudo systemctl stop gdm
```

ถ้าใช้ lightdm:

```
sudo systemctl stop lightdm
```

◆ (ทางเลือก) ถ้าอยาก “ถอน GUI ทิ้งจริง”

```
sudo apt purge ubuntu-desktop gnome-shell gdm3 -y
```

```
sudo apt autoremove -y
```

📌 หลังจากนั้น:

- เปิดเครื่อง = ได้ จอ text / ssh เท่านั้น
 - เหมาะกับ server + GPU + Docker มากที่สุด
-

2) ทำให้ “เข้าจากข้างนอกได้จริง”

ตอนนี้ UFW ถูกต้องแล้ว (จากรูปคุณ)

สิ่งที่ต้องเช็กเพิ่มมี 3 จุด (สั้นมาก)

2.1 Docker ต้อง bind ออก 0.0.0.0 (คุณทำถูกแล้ว)

จากก่อนหน้านี้:

0.0.0.0:4000->8080/tcp

✓ ถูกต้อง → ไม่ใช่ปัญหา

2.2 เช็คว่า service ฟังจริงไหม

รันบนเครื่อง:

```
ss -tulpn | grep -E '4000|11434|8188'
```

ควรเห็นประมาณนี้:

LISTEN 0.0.0.0:4000

LISTEN 0.0.0.0:11434

ถ้าเห็น = ผ่าน

2.3 Router / Modem (จุดสุดท้าย)

เพราะคุณมี Public IP แล้ว

ต้องทำ Port Forward ที่ Router

อย่างน้อยต้องมี:

External Port Internal IP (เครื่องนี้) Internal Port

22	192.168.x.x	22
4000	192.168.x.x	4000
8188	192.168.x.x	8188

 ถ้า Router ของ NT / องค์กร:

- ชื่อเมนูมักเป็น NAT / Virtual Server / Port Mapping



ทดสอบจาก “ข้างนอกวง” (มือถือ 4G ก็ได้)

เปิด:

`http://203.172.184.47:4000`

ถ้าเข้าได้ = จบ 🎉

ถ้าไม่ได้ = Router ยังไม่ forward



คำแนะนำระดับมืออาชีพ (เผื่อระยะยาว)

คุณกำลังทำ:

- Ollama
- OpenWebUI
- ComfyUI
- GPU Server



อย่าเปิด public ตรง ๆ ระยะยาว

ทางเลือกที่ “โปร + ปลอดภัย” กว่า:

- Cloudflare Tunnel (ไม่ต้องเปิด port)
- หรืออย่างน้อย reverse proxy + auth

ถ้าคุณบอกว่า

“เอา Cloudflare”

ผมจะจัดให้แบบ **copy-paste** ใช้งานจริง ภายใน 10 นาที



สรุปชัด ๆ

- ✓ Public IP: ผ่าน
- ✓ Firewall: ผ่าน
- ✓ Docker: ผ่าน
- ! เหลือ Router Port Forward
- 🛠 GUI ปิดได้ทันทีด้วย 2 คำสั่ง

ถัดไปอยากทำอะไรต่อ?

- 🔒 ล็อก server ให้ปลอดภัย

- 🎨 ลง ComfyUI (GPU) แบบ docker run on-demand
- 🌐 ทำโดเมน + https
- ⚡ optimize GPU / VRAM

เลือกมาได้เลยครับ ผมต่อให้ตรงสายทันที 😊

การจอ nvidia ก็ยังไม่ได้ ปิดหน้าจอ gui ก็ไม่ได้

โอเคครับ คราวนี้ผมขอสรุปแบบ “วิเคราะห์อาการ → แก้ทีละจุดแบบไม่เตา”

เพราะจากสิ่งที่คุณส่งมา อาการชัดมาก และแก้ได้แน่นอน 👍

สรุปอาการจริง (จากหลักฐานที่คุณส่งมา)

❌ ปัญหา 1: NVIDIA ใช้ไม่ได้จริง

แม้จะเห็นว่า

nvidia-driver-580-open is already the newest version

แต่ ยังไม่ถือว่า “ใช้งานได้” จนกว่าจะผ่าน 2 เงื่อนไขนี้:

- nvidia-smi ต้องรันได้
- kernel module nvidia ต้องโหลดจริง

ตอนนี้ ยังไม่เห็นหลักฐานว่า 2 ข้อนี้ผ่าน

❌ ปัญหา 2: ปิด GUI แล้ว แต่ยังเด้งกลับ

อาการนี้มักเกิดจากอย่างใดอย่างหนึ่ง:

1. ยังมี gdm3 / graphical.target ถูกดึงกลับ
2. system boot target ยังเป็น graphical
3. NVIDIA driver ยังไม่พร้อม → system พยายาม fallback GUI

แก้แบบ “ถูกลำดับ” (สำคัญมาก)

- ! อย่าข้ามขั้น
- ! ทำตามลำดับนี้เท่านั้น



STEP 1 — เช็คว่า NVIDIA driver “ทำงานจริงไหม”

รันคำสั่งนี้ก่อน:

nvidia-smi

ผลลัพธ์ที่เป็นไปได้

 ถ้าได้แบบนี้ (พัง)

command not found

หรือ

NVIDIA-SMI has failed because it couldn't communicate with the NVIDIA driver

→ แปลว่า driver ยังไม่ active

 ถ้าได้ตาราง GPU

→ ไป STEP 3 ได้เลย

STEP 2 — ทำให้ NVIDIA driver active จริง (จุดสำคัญ)

2.1 เช็ก kernel module

lsmod | grep nvidia

ถ้า ไม่ขึ้นอะไรเลย → driver ยังไม่ถูกโหลด

2.2 บังคับ blacklist nouveau (จำเป็นมาก)

sudo tee /etc/modprobe.d/blacklist-nouveau.conf <<EOF

blacklist nouveau

options nouveau modeset=0

EOF

แล้วสั่ง:

sudo update-initramfs -u

sudo reboot

 ต้อง reboot เท่านั้น

นี่คือจุดที่คนพลาดบ่อยสุด

2.3 หลัง reboot เช็กอีกครั้ง

nvidia-smi

ถ้ายังไม่ได้ → บอกผมทันที อย่าทำต่อ



STEP 3 — ปิด GUI แบบ “ไม่ให้มันฟื้นอีก”

3.1 เปลี่ยน boot target ถาวร

```
sudo systemctl set-default multi-user.target
```

เช็ค:

```
systemctl get-default
```

ต้องได้:

```
multi-user.target
```

3.2 ปิด display manager

```
sudo systemctl disable gdm3 --now
```

```
sudo systemctl mask gdm3
```

ถ้าใช้ lightdm:

```
sudo systemctl disable lightdm --now
```

```
sudo systemctl mask lightdm
```

3.3 (แนะนำ) ถอน GUI ทิ้งให้จบ

ถ้าเครื่องนี้คือ server ล้วน:

```
sudo apt purge ubuntu-desktop gnome-shell gdm3 -y
```

```
sudo apt autoremove -y
```

หลังจากนี้ ต่อให้ reboot 100 รอบ

ไม่มี GUI โผล่อีก



STEP 4 — เช็คสุดท้าย (ต้องได้แบบนี้)

```
nvidia-smi
```

```
systemctl status gdm3
```

ควรได้:

- nvidia-smi → เห็น RTX + VRAM ~16GB

- gdm3 → not found / masked

ทำไมผมมั่นใจว่าทางนี้ถูก

จาก log ที่คุณส่งมา:

- เครื่องเห็นการ์ด NVIDIA จริง
- ubuntu-drivers เห็น driver 580-open
- แต่ยังไม่มีหลักฐาน kernel module + nvidia-smi

นี่คือ classic case ของ “ลง driver แต่ยังไม่ active”

ถัดไป (หลังจาก 2 ปัญหา)

เมื่อ NVIDIA + headless server พร้อม ผมจะพาคุณไปต่อแบบนี้ได้ทันที:

-  Docker + NVIDIA runtime
-  ComfyUI (GPU) แบบ docker run เฉพาะเวลางาน
-  Ollama ใช้ GPU เต็ม
-  ไม่จ้อ GUI 100%

ขอให้คุณทำแค่นี้ก่อน 🙏

รันแล้วส่งผลลัพธ์มา:

nvidia-smi

lsmod | grep nvidia

systemctl get-default

ผมจะดูให้แบบ วิเคราะห์ตรงจุด ไม่มั่ว และพาไปต่อจนจบแน่นอนครับ 🙏

ดีมากครับ ภาพ BIOS นี่คือ **กุญแจดอกสุดท้าย** ของปัญหาที่คุณเจอเลย 🍌
ตอนนี้ขอสรุปแบบ “ฟันธง + แก้ให้จบ” ทีละข้อ

สรุปสถานการณ์จริง (จาก BIOS + log ทั้งหมด)

1 เครื่องคุณ ยังบูตแบบมีจอ (GUI path) อยู่ตั้งแต่ BIOS

จากภาพ:

- Full Screen Logo Display = Enabled
- เมนบอร์ด MSI PRO Series
- ยังมีการ init VGA/Display ตามปกติ

➡ นี่คือสาเหตุว่า

แม้คุณปิด gdm3 แล้ว แต่ระบบยัง init graphical pipeline ตั้งแต่ต้น

2 NVIDIA driver ลงแล้ว แต่ “ยังไม่ถูกใช้งานเต็มรูปแบบ (compute-first)”

เพราะ:

- BIOS ยังไม่ตั้งโหมด Headless / iGPU / PEG-first
- ระบบเลยพยายามเอา GPU ไปผูกกับจอ → ขนกับ server / docker / CUDA

เป้าหมายของเรา (ชัด ๆ)

- ✓ ไม่เข้าหน้า GUI Ubuntu
- ✓ เครื่องเป็น Headless Server 100%
- ✓ NVIDIA ใช้เพื่อ Compute (Docker / Ollama / ComfyUI) เท่านั้น
- ✓ เข้าเครื่องจากภายนอกได้ (SSH / WebUI)

🔧 แก้ที่ BIOS (สำคัญมาก — ทำก่อนอย่างอื่น)

เข้า Advanced (F7) แล้วไปตามนี้

🧠 1. ตั้งค่า GPU ให้เป็น Compute / ไม่ผูกจอ

ไปที่เมนู (ชื่ออาจต่างนิดหน่อย):

Advanced → Integrated Graphics Configuration หรือ Chipset

ตั้งค่า:

- Primary Display
 -  IGD หรือ CPU Graphics
 -  อย่าเลือก PEG / PCIe
- Initiate Graphic Adapter
 -  IGD / Onboard
- iGPU Multi-Monitor
 -  Disabled

ถ้ามีจอเสียบอยู่ → แนะนำถอดสาย HDMI/DP ออกจากการ์ด NVIDIA แล้วเสียบที่ iGPU (หรือไม่เสียบเลย)

2. ปิด Fast Boot / Graphic splash

ในหน้า Boot (ที่คุณอยู่):

- Full Screen Logo Display
 Disabled
- Fast Boot
 Disabled
- MSI Fast Boot
 Disabled

เพื่อให้ kernel load driver ตามจริง ไม่โดนข้ามขั้น

3. (ถ้ามี) ปิด CSM

- Boot Mode → UEFI only
- CSM → Disabled

 กด F10 → Save & Exit

 หลังออกจาก BIOS → ทำใน Ubuntu ต่อทันที

1 บังคับ server mode แบบถาวร

```
sudo systemctl set-default multi-user.target
```

```
sudo systemctl disable gdm3 --now
```

```
sudo systemctl mask gdm3
```

เช็ค:

```
systemctl get-default
```

ต้องได้:

```
multi-user.target
```

2 รีบูต 1 ครั้ง (สำคัญ)

```
sudo reboot
```

✓ หลังบูตขึ้นมา ต้องเช็ค 3 อย่างนี้



1. ไม่มี GUI

- หน้าจอควรเป็น **console login (tty)** หรือจอดำ
 - ไม่ควรเห็น GNOME อีก
-



2. NVIDIA ใช้งานได้จริง

```
nvidia-smi
```

ต้องเห็น:

- RTX 5070 Ti
 - VRAM ~16GB
 - Driver 580.xx
 - Processes = none (ยังดีอยู่)
-



3. เช็ค ว่า kernel โหลด nvidia แล้ว

```
lsmod | grep nvidia
```

ต้องขึ้นหลายบรรทัด (nvidia, nvidia_uvm ฯลฯ)

เรื่อง “เข้าจากข้างนอกไม่ได้” (เคลียร์ให้ชัด)

จากรูปคุณ:

- UFW เปิด port ครบ 
- IP public = 203.172.184.47 
- Docker map port แล้ว 

ถ้า ยังเข้าไม่ได้จากอินเทอร์เน็ต สาเหตุจะเหลือแค่ 1 ใน 2 อย่างนี้:



A. Router / ISP ไม่ forward port

- ต้อง forward port จาก router → IP เครื่องนี้
- ถ้าเป็นเน็ตองค์กร / NT / CGNAT → ต้องใช้ Cloudflare Tunnel / Tailscale



B. ISP บล็อก inbound

ทดสอบง่าย ๆ:

```
ss -tulpen | grep 4000
```

ถ้าฟังอยู่ แต่ข้างนอกเข้าไม่ได้ = ปัญหาที่ router/ISP 100%

ถ้าคุณบอกผมว่าใช้เน็ตอะไร ผมจะบอกทางลัดให้เลย



หลังจากนี้เราจะไปไหนต่อ

เมื่อจบขั้นนี้ (สำคัญมาก):

-  NVIDIA = compute-only
-  เครื่อง = headless server
-  Docker + GPU พร้อม

ขั้นถัดไปที่ผมเตรียมไว้:

1. NVIDIA Container Toolkit
2. ComfyUI (docker run แบบ on-demand)
3. Ollama ใช้ GPU เต็ม
4. ปรับ power / persistence mode

ทำตามนี้ก่อน แล้วบอกผมผล 3 คำสั่งนี้ครับ

nvidia-smi

```
systemctl get-default
```

```
lsmod | grep nvidia
```

ถ้าผ่าน = เรา “จบปัญหานรก” ชุติแน่นนอน 🙏

ภาพนี้ ชัดมาก และขอสรุปตรง ๆ ว่า

ตอนนี้ BIOS ยังตั้งค่า ผิดจุดสำคัญ 2 ค่า เลยทำให้

- ❌ Ubuntu ยังขึ้น GUI
- ❌ NVIDIA ยังไม่เข้าสู่โหมด compute แบบ headless เต็มตัว

เดี๋ยวแก้ไขจบที่หน้านี้เลย 🙌

แก้ BIOS หน้านี้ (สำคัญมาก)

คุณอยู่ที่

Advanced → Integrated Graphics Configuration

ให้ตั้งค่าแบบนี้เป๊ะ ๆ

1. Initiate Graphic Adapter (ตัวเอก)

ตอนนี้: Auto

👉 เปลี่ยนเป็น: IGD

บังคับให้ใช้ iGPU เป็นตัวหลัก

NVIDIA จะไม่ถูกใช้เป็นจออีกต่อไป = พร้อม compute

2. IGD Multi-Monitor

ตอนนี้: Enabled

👉 เปลี่ยนเป็น: Disabled

ไม่ต้องการหลายจอ

ลดการ init display pipeline

3. Integrated Graphics Control

ตอนนี้: Enabled

👉 คงไว้ Enabled (ถูกแล้ว)

4. Integrated Graphics Share Memory

ตอนนี้: 128M

👉 คงไว้ 128M หรือ Auto ก็ได้

สรุปค่าที่ถูกต้อง

รายการ	ค่า
Initiate Graphic Adapter	IGD
IGD Multi-Monitor	Disabled
Integrated Graphics Control Enabled	
Share Memory	128M

 กด F10 → Save & Exit

 หลังออกจาก BIOS → ทำใน Ubuntu (รอบสุดท้าย)

1 บังคับ headless แบบถาวร

```
sudo systemctl set-default multi-user.target
```

```
sudo systemctl disable gdm3 --now
```

```
sudo systemctl mask gdm3
```

2 รีบูต

```
sudo reboot
```

✅ เช็กผล (ต้องได้แบบนี้)

1. ไม่มี GUI

- จอเป็น tty / จอดำ
- SSH เข้าได้ปกติ

2. NVIDIA พร้อมใช้งานจริง

```
nvidia-smi
```

ต้องเห็น:

- RTX 5070 Ti

- VRAM ~16GB
- ไม่มี process ผูกจอ

3. kernel โหลด driver จริง

lsmod | grep nvidia

ถ้ายัง “ดื้อ” (แผน B – 100% server)

ถ้าบอร์ด MSI ยัง auto-init NVIDIA อยู่ (บางรุ่นเป็น):

```
sudo nano /etc/default/grub
```

แก้บรรทัด:

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
```

เป็น:

```
GRUB_CMDLINE_LINUX_DEFAULT="text nomodeset"
```

แล้ว:

```
sudo update-grub
```

```
sudo reboot
```

วิธีนี้ **ตัด GUI + DRM** แบบเด็ดขาด

soon หลังจากนั้น (ของจริงกำลังมา)

เมื่อผ่านจุดนี้ = เครื่องคุณจะเป็น

-  Headless AI Server 100%
-  Docker + GPU
-  พร้อมลง **ComfyUI (GPU)** แบบ on-demand
-  Ollama ใช้ VRAM เต็ม

ทำตามนี้ก่อน แล้วส่งผล 3 คำสั่งนี้มา:

```
nvidia-smi
```

```
systemctl get-default
```

```
who
```

ถ้าผ่าน = เราไป **ComfyUI + CUDA + VRAM tuning** ต่อทันที 

ดีมากครับ ภาพนี้คือ จุดสุดท้ายที่ต้องแก้ใน BIOS จริง ๆ 👍

และขอยืนยันก่อนเลยว่า คุณมาถูกทางแล้ว 90% เหลือแค่ “สวิตช์ตัวเดียว” ที่ MSI ซ่อนไว้ 😊

🔥 ประเด็นสำคัญจากภาพนี้

คุณอยู่ที่

Advanced → Integrated Peripherals → VGA Detection

ตอนนี้ค่าเป็น

VGA Detection = Auto

👉 ค่า Auto ของ MSI =

“ถ้าเจอการ์ดจอแรง → ใช้มันเป็นจอหลัก”

ซึ่ง นี่แหละต้นเหตุ ที่ทำให้

- Ubuntu ยังพยายามขึ้น GUI
- NVIDIA ยังโดนใช้เป็น display device

✅ ค่าที่ต้องตั้ง (สำคัญมาก)

🔧 VGA Detection

เปลี่ยนจาก

Auto

เป็น

IGD

IGD = Integrated Graphics Device

บังคับใช้ iGPU เท่านั้นสำหรับจอ

ค่าที่เหลือในหน้านี้ (ถูกแล้ว ไม่ต้องแตะ)

รายการ	ค่า
Onboard LAN Controller Enabled	
Network Stack	Disabled
Onboard Wi-Fi/BT	Auto

รายการ	ค่า
BT Tile Mode	Disabled



จากนั้น

กด

F10 → Save & Exit → Yes



ผลที่ “ต้องเกิด” หลังรีบูต

หลังจากตั้งค่านี:



1. NVIDIA จะไม่ถูก init เป็นจออีก

- ไม่ผูก DRM
- ไม่ผูก framebuffer
- พร้อมเป็น compute-only GPU



2. Ubuntu จะไม่จำเป็นต้องใช้ GUI

(เราจะปิดมันถาวรในขั้นถัดไป)



หลังบูตเข้า Ubuntu (รอบสุดท้ายจริง ๆ)

รันทีละบรรทัด:

```
sudo systemctl set-default multi-user.target
```

```
sudo systemctl disable gdm3 --now
```

```
sudo systemctl mask gdm3
```

```
sudo reboot
```



เช็ก 3 อย่างนี้ (สำคัญ)

หลังเครื่องกลับมา ให้ SSH เข้าแล้วส่งผลลัพธ์ 3 คำสั่งนี้มา:

```
nvidia-smi
```

```
systemctl get-default
```

```
ls /dev/nvidia*
```

ผลที่ “ถูกต้อง” ต้องเป็น:

- nvidia-smi เห็น RTX 5070 Ti + VRAM ~16GB
- systemctl get-default = multi-user.target
- มี /dev/nvidia0, /dev/nvidiactl

 หมายเหตุแบบคนผ่านสนามมาแล้ว

MSI ชอบแยก

- Initiate Graphic Adapter
- VGA Detection

ถ้าตั้งแค่หน้าเดียว → มันยัง “ฉลาดเกินไป” 😊

แต่ถ้าตั้ง IGD ทั้งสองจุด = จบจริง

 **soon** ถ้าผ่านจุดนี้

ขั้นถัดไปคือของสนุกแล้ว 😎

-  NVIDIA Container Toolkit
-  ComfyUI (Docker + GPU)
-  ใช้ VRAM เต็ม ๆ ไม่โดนจ่อแย่ง

ตั้งค่านี้อะไร → รีบูต → ส่งผล 3 คำสั่งมา

เดี๋ยวผมพาไป ComfyUI แบบ production-grade ต่อทันที 🙌

โอเค 👍 ภาพนี้ชัดมาก และคำตอบคือ

คุณเลือกถูกหน้าจอแล้ว แต่ค่าที่ต้องใช้คือ “Ignore” ไม่ใช่ Auto
ขอสรุปให้แบบ “ตัดจบ” ที่เดียวเลยครับ

✅ ค่าที่ควรตั้งจากภาพนี้ (สำคัญมาก)

🔧 VGA Detection

ตอนนี้คุณเปิด dropdown อยู่ และเห็นตัวเลือก:

- Auto
- Ignore

👉 ให้เลือก Ignore (ถูกต้องที่สุดในเคสนี้)

ความหมายแบบตรงไปตรงมา

- Auto → BIOS จะ “พยายามหา” การ์ดจอ และอาจดึง NVIDIA มาเป็นจอ
- Ignore → BIOS ไม่สนใจการ์ดจอแยก สำหรับการแสดงผล
(ปล่อยให้ NVIDIA เป็น compute-only เต็มรูปแบบ)

👉 สำหรับเครื่องที่:

- ใช้ NVIDIA ทำ AI / Docker / ComfyUI
- ไม่อยากได้ GUI
- รีโมทเข้ามาอย่างเดียว (SSH / Web)

ต้องใช้ Ignore เท่านั้น ✅

🔒 ค่าที่เหลือในหน้านี้ (ถูกแล้ว ไม่ต้องเปลี่ยน)

รายการ	ค่า
Onboard LAN Controller Enabled	
Network Stack	Disabled
Onboard Wi-Fi/BT	Auto
BT Tile Mode	Disabled
SATA Hot Plug	Disabled



ขั้นตอนถัดไป (ทำตามลำดับนี้)

1. เลือก **VGA Detection = Ignore**
2. กด **F10**
3. เลือก **Save & Exit**
4. ให้เครื่อง reboot



หลังบูตเข้า Ubuntu (รอบสุดท้ายจริง ๆ)

เข้าเครื่องด้วยคีย์บอร์ดหรือ SSH แล้วรัน:

```
sudo systemctl set-default multi-user.target
```

```
sudo systemctl disable gdm3 --now
```

```
sudo systemctl mask gdm3
```

```
sudo reboot
```

คำสั่งชุดนี้จะ



ปิด GUI ถาวร



ไม่โหลด display manager



เหลือแต่โหมด server ล้วน ๆ



ตรวจสอบผล (3 อย่างนี้สำคัญ)

หลัง reboot แล้ว SSH เข้ามา รัน:

```
nvidia-smi
```

```
systemctl get-default
```

```
ls /dev/nvidia*
```

ผลที่ถูกต้องต้องเป็น:

- `nvidia-smi` → เห็น RTX + VRAM ~16GB
- `systemctl get-default` → `multi-user.target`
- มีไฟล์ `/dev/nvidia0`, `/dev/nvidiactl`

ถ้าได้ครบ = จบปัญหา NVIDIA + GUI 100% 🎉



หมายเหตุแบบคนทำ server จริง

MSI ใช้คำว่า Ignore แทน IGD

ซึ่งต่างจากหลายยี่ห้อ → ทำให้หลายคนพลาดจุดนี้

คุณมาถึงจุดที่ “ถูกต้องที่สุด” แล้ว

เหลือแค่กด Save 😊

➡️ soon ขั้นถัดไป (ของมันส์)

พอผ่านขั้นนี้ ผมพาไปต่อได้ทันที:

-  NVIDIA Container Toolkit
-  ComfyUI (Docker + GPU)
-  ใช้ VRAM เต็ม ไม่มีจอมาแย่ง

ตั้งค่าเสร็จ → reboot → ส่งผล nvidia-smi มาได้เลยครับ

เดี๋ยวลุย ComfyUI ต่อทันที 🙌🔥

ตอนนี้ ผมย้ายมาใช้การ์ดจอ นอก ไม่ใช่ nvidia แล้ว อป็นตัวออนบอร์ด

โอเคครับ เข้าใจสถานการณ์ชัดแล้ว 👍

ตอนนี้เครื่องคุณใช้การ์ดจอออนบอร์ด (iGPU) เป็นตัวแสดงผล และไม่ใช่ NVIDIA แล้ว

เป้าหมายคือ:

1.  ไม่อยากเห็นหน้า GUI Ubuntu
2.  ให้เครื่องทำงานแบบ Server / Headless
3.  รีโมทจากข้างนอกได้ปกติ (SSH / Web / Docker)

ผมสรุปให้เป็น ขั้นตอนสั้น + ตรงจุด เลยนะครับ

สถานะปัจจุบัน (จากที่คุณบอก)

- การ์ดจอแสดงผล = Onboard / iGPU
- NVIDIA = ไม่ใช้งาน (หรือถอดออกแล้ว)
- เครื่องมีจอเสียบอยู่ แต่ ไม่อยากให้ Ubuntu เปิด GUI

ถูกต้องทั้งหมด 👍

ส่วนที่ 1: ตั้งค่า BIOS (ทำถูกแล้วเกือบครบ)

จากภาพ BIOS ล่าสุด แนะนำแบบนี้:

✓ BIOS → Advanced → Integrated Peripherals

ค่า ตั้งเป็น

VGA Detection Ignore ✓

Onboard LAN Controller Enabled

Network Stack Disabled

Wi-Fi / BT Auto

Ignore = ใช้ iGPU แบบเจียบ ๆ ไม่ยุ่งกับ OS

เหมาะสมสำหรับ server

จากนั้น

👉 F10 → Save & Exit

🔧 ส่วนที่ 2: ปิด GUI Ubuntu แบบ “ถาวรจริง”

เข้าเครื่องด้วยคีย์บอร์ดหรือ SSH แล้วรันที่ละบรรทัด:

ตั้งค่าให้บูตเป็น server mode

```
sudo systemctl set-default multi-user.target
```

ปิด display manager (ตัวทำ GUI)

```
sudo systemctl disable gdm3 --now
```

```
sudo systemctl mask gdm3
```

reboot

```
sudo reboot
```

🔍 หลัง reboot

- จะ ไม่เห็นหน้า Desktop
- จอจะเป็น text / black screen (ปกติ)
- เครื่องยังทำงานอยู่ 100%

นี่คือพฤติกรรม server ที่ถูกต้อง ✓

ส่วนที่ 3: ให้เข้าจากข้างนอกได้แน่นอน

1 ตรวจสอบ SSH

`sudo systemctl status ssh`

ต้องเป็น active (running)

ถ้ายังไม่:

`sudo apt install -y openssh-server`

`sudo systemctl enable --now ssh`

2 Firewall (จากรูปคุณ = ผ่านแล้ว)

พอร์ตที่ควรมี:

- 22 (SSH)
- 4000 (OpenWebUI)
- 80 / 8080 (ถ้ามี reverse proxy)

ตรวจสอบอีกครั้ง:

`sudo ufw status`

ถ้า ufw ยัง inactive:

`sudo ufw enable`

3 เช็คว่าเครื่องฟัง port จริง

`ss -tulpen | grep -E '22|4000'`

ถ้าเห็น 0.0.0.0:4000 = ✓ ถูกต้อง

ส่วนที่ 4: Docker / CasaOS / OpenWebUI

คุณทำถูกแล้ว และตอนนี้:

- OpenWebUI เข้าได้จากภายนอกแล้ว ✓
- Docker ทำงาน ✓
- ไม่ต้องฟัง GUI ✓

ถือว่า setup แบบ Home Server / AI Server มาตรฐาน แล้ว

คำถามสำคัญ (ขอลาม 2 ข้อเดียว)

ตอบแค่สั้น ๆ ได้เลยครับ

1 ตอนนี้ ยังเสียบจอจริงอยู่ไหม

- ☐ เสียบไว้เฉย ๆ
- ☐ จะถอดออกถาวร

2 แผนต่อไปคือ:

- ☐ ใช้ iGPU ยาว (ไม่ใช่ NVIDIA แล้ว)
- ☐ จะใส่ NVIDIA กลับมา แต่ไม่เอา GUI

คำตอบ 2 ข้อนี้จะทำให้ผมจัด config ขึ้นถัดไปให้ “ตรงที่สุด” 🙌

คุณมาถูกทางมากแล้ว เหลือแค่เก็บงานให้เนียน 😊

ตอนนี้ ใช้ จอออกบอร์ด GPU nvidia ไม่ใช่แล้ว แต่ยังเสียบอยู่ในเครื่อง

ซัดมากครับ 👍

สรุปสถานะจริงตอนนี้คือ

- ✅ ใช้ iGPU / ออนบอร์ด เป็นตัวแสดงผล
- ❌ ไม่ใช่ NVIDIA แต่ การ์ดยังเสียบคาเครื่องอยู่
- ❌ ไม่อยากเห็น GUI
- ✅ อยากให้เครื่องนิ่งแบบ Server / Headless

ปัญหาที่เกิดขึ้นนี้มักจะมี 2 เรื่อง:

1. Ubuntu / kernel ยังพยายามยุ่งกับ NVIDIA
2. GUI / display manager ยังผูกกับกราฟิกอยู่

เดียวจัดให้แบบ “ตัดขาดอย่างสุภาพ” ไม่พัง ไม่ต้องถอดการ์ด 👍

เป้าหมาย

- iGPU ทำงานตัวเดียว
- NVIDIA ถูก “พักงาน” ไม่โหลด driver ไม่กินไฟ ไม่ก่อปัญหา
- เครื่องบูตแบบ server ล้วน ๆ

✓ STEP 1: ยืนยันว่าตอนนี้ใช้ iGPU จริง

รันก่อนเพื่อเช็คสภาพ

```
lspci | grep -E "VGA|3D"
```

คุณควรเห็นประมาณนี้:

- Intel UHD / Intel Graphics → ✓ ใช้จริง
- NVIDIA RTX → ✓ ยังเห็นแต่ไม่ใช้

✓ STEP 2: ปิด NVIDIA แบบไม่ต้องถอดการ์ด (แนะนำที่สุด)

2.1 Blacklist driver NVIDIA ทั้งหมด

```
sudo nano /etc/modprobe.d/blacklist-nvidia.conf
```

ใส่เนื้อหาต่อไปนี้ลงไป:

```
blacklist nouveau
```

```
blacklist nvidia
```

```
blacklist nvidia_drm
```

```
blacklist nvidia_modeset
```

```
blacklist nvidia_uvm
```

บันทึกแล้วออก

2.2 บอก kernel ว่า “อย่าไปแตะ NVIDIA ตั้งแต่บูต”

```
sudo nano /etc/default/grub
```

หาแถว:

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
```

แก้เป็น:

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet nomodeset
```

```
modprobe.blacklist=nouveau,nvidia,nvidia_drm,nvidia_modeset,nvidia_uvm"
```

แล้วสั่ง:

```
sudo update-grub
```

2.3 สร้าง initramfs ใหม่

`sudo update-initramfs -u`

2.4 reboot

`sudo reboot`

หลัง reboot:

- NVIDIA จะ **ไม่ถูกโหลด**
 - ไม่มี driver จับมันอีก
 - กินไฟน้อยลง
 - Docker / server นิ่งขึ้น
-

STEP 3: ปิด GUI แบบถาวร (ย้ำอีกครั้งให้ชัด)

ถ้ายังไม่ได้ทำ:

`sudo systemctl set-default multi-user.target`

`sudo systemctl disable gdm3 --now`

`sudo systemctl mask gdm3`

`sudo reboot`

หลังจากนี้:

- ไม่มี desktop
 - จอจะดำหรือเป็น tty (ปกติมาก)
 - SSH / Docker / CasaOS ยังทำงาน
-

STEP 4: ตรวจสอบว่าระบบ “ไม่ยุ่งกับ NVIDIA แล้ว”

หลังบูตเสร็จ รัน:

`lsmod | grep nvidia`

 **ไม่ควรขึ้นอะไรเลย = ผ่าน 100**

และ:

`lsmod | grep nouveau`

 **ต้องไม่ขึ้นเช่นกัน**



คำแนะนำจากประสบการณ์จริง

- การ เสียบ NVIDIA ค้างไว้แต่ไม่ใช้ → ทำได้ ไม่มีปัญหา
- วิธีนี้ดีกว่าถอด driver ออก เพราะ:
 - วันไหนอยากกลับมาใช้ CUDA / ComfyUI → แค่ลบ blacklist + reboot
 - ไม่ต้องลง OS ใหม่

ถามนิดเดียว เพื่อก้าวต่อ

ขั้นถัดไปคุณอยาก:

1 ใช้ NVIDIA เฉพาะใน Docker (ComfyUI / CUDA) แต่ไม่เอา GUI

หรือ

2 ไม่แตะ NVIDIA เลย ใช้ iGPU ล้วน ๆ ไปก่อน

ตอบแค่ **1** หรือ **2** เดียวผมจัด config ต่อให้ตรงทางทันทีครับ 😊

อันนี้ ผลออกมาถูกต้อง 100% ตามที่เราตั้งใจไว้แล้วครับ ✅

ขอสรุปแบบชัด ๆ ให้สบายใจเลย



วิเคราะห์จากผลที่คุณส่งมา

1 nvidia-smi ขึ้น error

NVIDIA-SMI has failed because it couldn't communicate with the NVIDIA driver



อันนี้ไม่ใช่ปัญหา



นี่คือ สัญญาณว่าการ blacklist NVIDIA สำเร็จ

ถ้า NVIDIA ยังถูกใช้งานอยู่ → nvidia-smi ต้องขึ้นสถานะ GPU

แต่ตอนนี้มัน “คุยไม่ได้” เพราะ driver ไม่ถูกโหลด ✓

2 lspci | grep -E "VGA|3D"

Intel Corporation Arrow Lake-U [Intel Graphics]

NVIDIA Corporation Device 2c05

แปลว่า:

อุปกรณ์ สถานะ

Intel iGPU ✓ ใช้งานจริง (ตัวหลัก)

NVIDIA RTX ⚠ ยังเสียบอยู่ แต่ ไม่ถูกใช้งาน

นี่คือ สภาพที่ดีที่สุดสำหรับ server/headless

เห็นการ์ดได้ แต่ kernel ไม่ยุ่ง ไม่โหลด ไม่กินทรัพยากร

🔧 สรุปสถานะเครื่อง “ตอนนี้”

- ✓ ใช้ iGPU (Intel) เป็นตัวแสดงผล
- ✓ ไม่มี GUI
- ✓ NVIDIA ไม่ถูกโหลด driver
- ✓ nvidia-smi ใช้ไม่ได้ = ถูกต้อง
- ✓ Docker / CasaOS จะนิ่งขึ้น
- ✓ พร้อมทำงานแบบ server เต็มตัว

พูดง่าย ๆ คือ

เครื่องคุณตอนนี้ = Server Mode สมบูรณ์แบบ

🔧 เช็กขั้นสุดท้าย (optional แต่ชัวร์มาก)

ถ้าคำสั่งพวกนี้ ไม่ขึ้นอะไรเลย แปลว่าจบจริง

lsmod | grep nvidia

lsmod | grep nouveau



ก้าวต่อไป (เลือกทางได้เลย)

ตอนนี้คุณมี 2 ทางหลัก ๆ:

◆ ทาง A (แนะนำ)

ใช้ iGPU ทำงานปกติ

เรียก NVIDIA เฉพาะตอนจำเป็น ผ่าน Docker (ComfyUI / CUDA)

ข้อดี:

- เครื่องนิ่ง
- ไม่ต้อง reboot บ่อย
- เปิด NVIDIA เมื่อ “ต้องใช้จริง”

ผมจะจัด:

- nvidia-container-toolkit
- docker run / compose สำหรับ ComfyUI แบบ --gpus all
- เปิดใช้เป็นครั้ง ๆ

◆ ทาง B

ไม่แตะ NVIDIA เลย ใช้ iGPU ล้วน ๆ

ข้อดี:

- ง่าย
- ประหยัดไฟ
- เหมาะกับ LLM เล็ก / งานทั่วไป



คำถามสุดท้าย (ตอบสั้น ๆ)

คุณอยากไปต่อแบบไหน?

พิมพ์ตอบแค่

- A = ใช้ NVIDIA เฉพาะใน Docker (ComfyUI / CUDA)
- B = ไม่ใช้ NVIDIA เลย

เดี๋ยวผมจัด config ต่อให้ตรงทางทันทีครับ 🚀

A = ใช้ NVIDIA เฉพาะใน Docker (ComfyUI / CUDA)

เยี่ยมมาก 👍 เลือก ทาง A = ใช้ NVIDIA เฉพาะใน Docker

นี่คือแนวทางที่ “โปรสายเซิร์ฟเวอร์” ใช้กันจริง ๆ — เครื่องนิ่ง + คุมพลัง GPU ได้เต็มที่ 🔥
ผมจะจัดให้เป็น ขั้นตอนสั้น-ชัด-ไม่พังระบบเดิม

🧠 แนวคิดก่อนลงมือ (สำคัญมาก)

สถานะเครื่องคุณตอนนี้ถือว่า ถูกต้องที่สุดแล้ว

- ✅ ใช้ iGPU Intel เป็นจอ
- ✅ ไม่มี GUI
- ✅ NVIDIA ไม่ถูกโหลดในระบบหลัก
- 🎯 เราจะเรียก NVIDIA เฉพาะตอน Docker container ขอใช้

Kernel ไม่ยุ่ง → Docker จัดการ GPU เอง

STEP 1 ติดตั้ง NVIDIA Container Toolkit (หัวใจของทาง A)

อันนี้ **ไม่**ทำให้ driver ถูกโหลดใน host

เป็นแค่สะพานให้ Docker เรียก GPU

เพิ่ม repo

```
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | sudo gpg --dearmor -o  
/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
```

```
curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.list \  
| sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg]  
https://#g' \  
| sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
```

install

```
sudo apt update
```

```
sudo apt install -y nvidia-container-toolkit
```

STEP 2 บอก Docker ให้รู้จัก NVIDIA

```
sudo nvidia-ctk runtime configure --runtime=docker
```

```
sudo systemctl restart docker
```

- ไม่ต้อง reboot
- ไม่ต้องโหลด nvidia driver ใน host

STEP 3 ทดสอบ GPU ผ่าน Docker (จุดตัดสิน)

```
docker run --rm --gpus all nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi
```

ผลลัพธ์ที่ “ถูกต้อง” ต้องเป็น:

- เห็นชื่อการ์ด NVIDIA
- เห็น VRAM
- ไม่มี error

ถ้าผ่าน = 🎉 พร้อมรบ

STEP 4 ComfyUI (Docker run เฉพาะเวลาทำงาน)

ตัวอย่าง docker run (GPU on-demand)

```
docker run -d \  
  --name comfyui \  
  --gpus all \  
  -p 8188:8188 \  
  -v /DATA/appdata/comfyui:/workspace \  
  ghcr.io/comfyanonymous/comfyui:latest
```

- ✓ เปิด GPU
- ✓ ปิด container = GPU ว่าง
- ✓ ไม่แตะ host driver

STEP 5 (ทางเลือกขั้นเทพ) docker-compose แยกโลกชัดเจน

services:


```
comfyui:
  image: ghcr.io/comfyanonymous/comfyui:latest
  container_name: comfyui
  ports:
    - "8188:8188"
  volumes:
    - /DATA/appdata/comfyui:/workspace
  deploy:
    resources:
      reservations:
        devices:
          - driver: nvidia
            count: all
            capabilities: [gpu]
```

รันด้วย

```
docker compose up -d
```



ตรวจสอบเวลารันจริง

ตอน ComfyUI ทำงาน ลองอีกเครื่อง:

```
docker exec -it comfyui nvidia-smi
```

หรือดูจาก host:

```
docker ps
```



ภาพรวมสุดท้าย (สรุปให้ง่าย)

ส่วน

สถานะ

Host Ubuntu iGPU / ไม่มี NVIDIA

GUI



ไม่มี

NVIDIA driver



ไม่โหลด

ส่วน	สถานะ
Docker + GPU	✅ ใช้ได้
ComfyUI	✅ CUDA เต็ม
ความนิ่ง	★★★★★

ถัดไปอยากทำอะไร?

ผมต่อให้ได้ทันที:

- 1 จำกัด VRAM ต่อ container
- 2 ทำ ComfyUI เปิด-ปิดอัตโนมัติ
- 3 ต่อ OpenWebUI → ComfyUI
- 4 ทำหลาย GPU / หลาย workflow

พิมพ์เลขมาได้เลย 😊

ตอนนี้ ให้ nvidia เจอ driver ก่อนยังไม่ต้องทำอย่างอื่น

เข้าใจตรงกันนะครับ 👍

เป้าหมายตอนนี้ = ให้ nvidia-smi ใช้งานได้บน host ให้เจอ driver ก่อน

ยังไม่ยุ่ง Docker / ComfyUI / CUDA อะไรทั้งนั้น

จากหลักฐานล่าสุดของคุณ:

- lspci 👉 เห็นการ์ด NVIDIA (Device 2c05) ✅
- nvidia-smi 👉 ยังคุยกับ driver ไม่ได้ ❌
- แปลว่า: driver ยังไม่ถูกโหลดใน kernel

เราจะแก้แบบ “ตรงจุด ไม่วน” 👉

STEP 0 (สำคัญมาก) ยืนยันสถานะปัจจุบัน

รันก่อน:

```
lsmod | grep nvidia
```

ถ้า **ไม่ขึ้นอะไรเลย** = driver ยังไม่โหลด → ไป STEP 1

ถ้าขึ้นแล้วแต่ nvidia-smi ยังพัง → บอกผมทันที

STEP 1 ปิด nouveau (ศัตรูของ NVIDIA)

```
sudo tee /etc/modprobe.d/blacklist-nouveau.conf <<EOF
blacklist nouveau

options nouveau modeset=0
EOF
sudo update-initramfs -u
```

STEP 2 ติดตั้ง driver ตัวเดียวให้ชัด (580-open ที่คุณมีถูกต้องแล้ว)

เช็กก่อนว่า kernel header ครบไหม:

```
uname -r
sudo apt install -y linux-headers-$(uname -r)

```

บังคับลงใหม่ให้สะอาด:

```
sudo apt install --reinstall -y nvidia-driver-580-open
```

STEP 3 รีบูต (จำเป็นจริง ๆ รอบนี้)

```
sudo reboot
```

STEP 4 ตรวจสอบผล (จุดตัดสิน)

หลังบูตเข้ามาใหม่ รันทันที:

```
nvidia-smi
```

ผลที่ถูกต้อง:

- เห็นชื่อ GPU
- เห็น Driver Version 580.xx
- เห็น VRAM

ถ้าได้ = 🎉 ผ่านด่านแรก

ถ้ายังไม่ผ่าน (แผนสำรองแบบมืออาชีพ)

ส่งผลลัพธ์ 3 คำสั่งนี้มา:

```
dmesg | grep -i nvidia | tail -50
modinfo nvidia | head
```

```
ls -l /dev/nvidia*
```

ผมจะบอกได้ทันทีว่า:

- driver ไม่ match kernel
- secure boot บล็อก
- หรือ BIOS ยังขวางอยู่จุดไหน

สรุปสั้น ๆ

ตอนนี้ อย่าเพิ่งยุ่ง Docker / ComfyUI

เอาให้ nvidia-smi ผ่านก่อน = ฐานรากทุกอย่างจะง่ายขึ้น 10 เท่า

รัน STEP 1-4 แล้วรายงานผลมาได้เลยครับ 🙌

โอเค ตรงนี้ ชัดมากแล้ว และดีที่คุณบอกบริบทครับ 🙌

ผมขอสรุปสถานการณ์ก่อน แล้วพาแก้แบบ “ไม่มั่ว ไม่วน”

สรุปสถานะปัจจุบัน (จากหลักฐานทั้งหมด)

1. lspci เห็น NVIDIA GPU (Device 2c05) ✓
2. nvidia-driver-580-open ติดตั้งแล้ว ✓
3. nvidia-smi ✗
4. NVIDIA-SMI has failed because it couldn't communicate with the NVIDIA driver
5. ตอนนี้
 - ใช้ จอออนบอร์ด Intel
 - NVIDIA เสียบอยู่ แต่ไม่ใช้แสดงผล
 - เป้าหมาย: ใช้ NVIDIA เฉพาะ compute (Docker / CUDA)

👉 แปลตรงตัว:

Kernel ไม่ได้โหลดโมดูล nvidia (ไม่ใช่เรื่องจอ ไม่ใช่ Docker)

จุดตายที่เจอกับเคสคุณ (MSI + Intel iGPU + NVIDIA)

Secure Boot + Module Signing

driver ติดตั้งแล้ว แต่ kernel “ไม่ยอมโหลด”

อาการ = ตรงกับคุณ 100%

STEP 1 : เช็ค Secure Boot (สำคัญที่สุด)

รันทำนานี้ก่อน 👉

`mokutil --sb-state`

ถ้าได้

- SecureBoot enabled ❌ ← นี่แหละตัวการ
- SecureBoot disabled ✅ → ข้ามไป STEP 3

STEP 2 : ปิด Secure Boot (แนะนำที่สุด)

ทำใน BIOS (MSI Click BIOS X)

เข้า BIOS → Advanced (F7)

ไปที่:

Boot

└─ Secure Boot
 └─ Secure Boot → Disabled
 └─ OS Type → Other OS

⚠️ ไม่ต้องสนใจ Windows, เครื่องนี้คือ server แล้ว

Save & Exit → บูตเข้า Ubuntu ใหม่

STEP 3 : โหลดโมดูล NVIDIA แบบตรง ๆ

หลังบูตเสร็จ:

`sudo modprobe nvidia`

แล้วเช็ค:

```
lsmod | grep nvidia
```

ถ้าเห็น:

nvidia

nvidia_uvm

nvidia_drm

= ผ่าน 🎯

ลองต่อทันที:

```
nvidia-smi
```

STEP 4 : ถ้ายังไม่ขึ้น (แผน B แต่ยังคงตรงจุด)

เช็ค error จาก kernel:

```
dmesg | grep -i nvidia | tail -50
```

และเช็ค device node:

```
ls -l /dev/nvidia*
```

- ถ้า ไม่มี /dev/nvidia0 → driver ไม่ถูกโหลด
- ถ้ามี แต่ nvidia-smi ยังพัง → แจ้งผมทันที

เรื่องจอ / GUI (ตอบข้อกังวลคุณ)

? ใช้อจอ Intel แล้ว NVIDIA จะไม่ทำงานไหม

➡ ไม่เกี่ยวเลย

NVIDIA ทำงานแบบ headless compute ได้ 100%

? ต้องเข้า GUI Ubuntu ไหม

➡ ไม่ต้อง

คุณทำถูกแล้ว ใช้ SSH ล้วน ๆ ดีที่สุด

สรุปแบบชัด ๆ

ตอนนี้ Docker / ComfyUI ยังไม่ใช่ปัญหา

ปัญหาอยู่ที่:



Secure Boot บล็อก NVIDIA kernel module

แก้ตรงนี้ = nvidia-smi จะขึ้นทันที

ทำตามนี้ลำดับเดียว:

1. mokutil --sb-state
2. ถ้า enabled → ปิด Secure Boot ใน BIOS
3. บูต → modprobe nvidia
4. nvidia-smi

ส่งผลข้อ 1 หรือผล dmesg | grep nvidia มาได้เลย

ผมอยู่ต่อจนมันขึ้นแน่นอน 🦊

Using username "payaprai". payaprai@203.172.184.47's password: Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-37-generic x86_64) * Documentation: <https://help.ubuntu.com> * Management: <https://landscape.canonical.com> * Support: <https://ubuntu.com/pro> Expanded Security Maintenance for Applications is not enabled. 69 updates can be applied immediately. To see these additional updates run: apt list --upgradable 14 additional security updates can be applied with ESM Apps. Learn more about enabling ESM Apps service at <https://ubuntu.com/esm> Last login: Sat Feb 7 06:11:52 2026 from 171.100.182.0

```
payaprai@payaprai-MS-7E41:~$ sudo -i [sudo] password for payaprai: root@payaprai-MS-7E41:~#
docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS          NAMES
262e507a4023   teachermon-api:latest "docker-entrypoint.s..." 15 minutes ago Restarting (1) 4 seconds ago
teachermon-api e79db1e1138c plcnextgen-web "/docker-entrypoint...." 11 hours ago Up About a minute 0.0.0.0:9902->80/tcp, [::]:9902->80/tcp plcnextgen-web-1 132e7ae49528 nginx:alpine "/docker-entrypoint...." 31 hours ago Up About a minute 0.0.0.0:9000->80/tcp, [::]:9000->80/tcp
nitesa_nginx dcaf88a7c389 nitesa-app "docker-php-entrypoi..." 31 hours ago Up About a minute 9000/tcp nitesa_app 088be1a534b3 redis:alpine "docker-entrypoint.s..." 31 hours ago Up About a minute 6379/tcp nitesa_redis 5ae96ca93c52 tablebooking-app "docker-entrypoint.s..." 3 d ays ago Up About a minute 0.0.0.0:9900->3000/tcp, [::]:9900->3000/tcp tablebooking-app-1 8f2d920e228f mariadb:11.4 "docker-entrypoint.s..." 4 d ays ago Up About a minute (healthy) 3306/tcp eqap_db 9064bca3896d doitung-app "docker-entrypoint.s..." 4 d ays ago Up About a minute (unhealthy) 0.0.0.0:9901->9901/tcp, [::]:9901->9901/tcp eqap_app 93071d348e3b linuxserver/mariadb:11.4.8 "/init" 6 d ays ago Up About a minute 0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp mariadb 06e2581f1f36 phpmyadmin/phpmyadmin:5.2.2 "docker-entrypoint...." 2 w eeks ago Up About a minute 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp big-bear-phpmyadmin b5740c7b4024 ghcr.io/open-webui/open-webui:main "bash start.sh" 4 w eeks ago Up About a minute (healthy) 0.0.0.0:4000->8080/tcp, [::]:4000->8080/tcp openwebui 172ab42f7fc8 ollama/ollama:latest "/bin/ollama serve" 4 w eeks ago Up About a minute (healthy) 0.0.0.0:11434->11434/tcp, [::]:11434->11434/tcp ollama root@payaprai-MS-7E41:~#
```

อยู่ ๆ ทำไม่ casaos พัง

โอเค ภาพนี้ ตอบคำถามได้ชัดมาก ว่า

“CasaOS ไม่ได้พึ่งแบบลุ่มระบบ แต่ “โดนแย่งพอร์ต / เส้นทางเว็บ””

และคุณทำถูกแล้วที่เอา docker ps มาให้ดู 👍

ผมจะอธิบายแบบตรงจุด ไม่อ้อม

สรุปอาการจากหลักฐาน

1. Docker ยังทำงานปกติทั้งหมด ✅
 - openwebui → port 4000
 - ollama → port 11434
 - app อื่น ๆ เพียบ
2. CasaOS เข้าไม่ได้จากเว็บ
 - casaos.cnppai.com → timeout
 - Browser error = ERR_CONNECTION_TIMED_OUT
3. ใน docker ps มีตัวนี้ 🛑
4. nginx:alpine 0.0.0.0:9000->80

และอาจมีตัวอื่นเคยใช้ port 80 / 443 มาก่อน

สาเหตุที่แท้จริง (ฟันธง)

🔥 CasaOS “ไม่ได้ตาย”

แต่...

Web stack ของ CasaOS (CasaOS-UI / CasaOS-Gateway)

โดนชนพอร์ต / reverse proxy ทับ

โดยเฉพาะ:

- คุณมี nginx container
- มีหลาย app bind แบบ 0.0.0.0
- CasaOS ใช้ port 80 / 443 / 81 (แล้วแต่เวอร์ชัน)

👉 พอ port ถูกใช้ก่อน → CasaOS UI เปิดไม่ได้

เช็กให้เห็นกับตา (ทำทีละคำสั่ง)

1 เช็คว่า CasaOS service ยังอยู่ไหม

systemctl status casaos

ถ้าเห็น active (running) → ระบบยังไม่พัง

2 เช็คพอร์ตที่ CasaOS ควรใช้

ss -lntp | grep -E '80|81|443'

คุณ将会เห็นว่า port ใดถูก nginx / docker จอง

วิธีแก้ (เลือกทางเดียวพอ)

ทางเลือก A (แนะนำที่สุด)

👉 ย้าย CasaOS ไปพอร์ตอื่น (เช่น 8088)

sudo nano /etc/casaos/gateway.ini

หา:

Port=80

แก้เป็น:

Port=8088

แล้วรีสตาร์ท CasaOS:

sudo systemctl restart casaos

เข้าใหม่:

http://203.172.184.47:8088

หรือ

http://casaos.cnppai.com:8088

ทางเลือก B (ถ้าจะใช้ nginx ครอบ CasaOS)

ให้ nginx reverse proxy ไป CasaOS port ภายใน

(อันนี้ขั้น advanced — ทำทีหลังได้)

สิ่งที่ “ไม่ต้องทำ” (จะทำให้พังจริง)

- ❌ ไม่ต้อง uninstall CasaOS
- ❌ ไม่ต้อง reboot มั่ว
- ❌ ไม่เกี่ยวกับ NVIDIA / GPU
- ❌ ไม่เกี่ยวกับ Docker engine

ทำไม CasaOS ถึงดูเหมือน “พัง” ทั้งที่ docker ยังรัน?

เพราะ CasaOS = Control Plane (UI)

Docker = Data Plane (runtime)

UI ล่ม ≠ container ล่ม

ซึ่งตอนนี้ของคุณคือกรณีนี้เปะ

สรุปแบบสั้นที่สุด

- ✅ Docker + Ollama + OpenWebUI ปกติ
- ❌ CasaOS UI โดนชนพอร์ต
- 🔧 แก้ด้วยการย้าย port CasaOS หรือจัด nginx ใหม่

ถ้าคุณอยากให้ผมช่วย:

- จัดผังพอร์ตถาวร (server layout)
- วางกติกา 0.0.0.0 vs internal
- หรือทำ CasaOS = admin only / app = docker-only

บอกได้เลย

ตอนนี้เครื่องคุณ “แข็งแรงมาก” แค่ต้องจัดระเบียบครับ 🙏

```
=> [internal] load local bake definitions 0.0s => => reading from stdin 1.05kB 0.0s => [internal]
load build definition from Dockerfile 0.0s => => transferring dockerfile: 3.00kB 0.0s => [internal]
load metadata for docker.io/library/node:20-bookworm-slim 1.9s => [internal] load
.dockerignore 0.0s => => transferring context: 269B 0.0s => CACHED [base 1/4] FROM
docker.io/library/node:20-bookworm-
slim@sha256:c6585df72c34172bebd8d36abed961e231d7d3b5cee2e01294c4495e8a03f687 0.0s
=> => resolve docker.io/library/node:20-bookworm-
```

slim@sha256:c6585df72c34172bebd8d36abed961e231d7d3b5cee2e01294c4495e8a03f687 0.0s
=> [internal] load build context 0.0s => => transferring context: 1.61MB 0.0s => [production
2/15] RUN apt-get update -qq && apt-get install -y --no-install-recommends curl && rm -rf
/var/lib/apt/lists/* 4.3s => [base 2/4] RUN apt-get update -qq && apt-get install -y --no-install-
recommends ca-certificates && rm -rf /var/lib/apt/lists/* 4.7s => [production 3/15] WORKDIR
/app 0.0s => [base 3/4] RUN corepack enable && corepack prepare pnpm@9 --activate 1.4s =>
[base 4/4] WORKDIR /app 0.0s => [dependencies 1/5] COPY package.json pnpm-workspace.yaml
pnpm-lock.yaml ./ 0.0s => [dependencies 2/5] COPY apps/web/package.json ./apps/web/ 0.0s
=> [dependencies 3/5] COPY packages/shared/package.json ./packages/shared/ 0.0s =>
[dependencies 4/5] COPY packages/database/package.json ./packages/database/ 0.0s =>
[dependencies 5/5] RUN pnpm install --frozen-lockfile --prod=false 12.5s => [build 1/8] COPY --
from=dependencies /app ./ 1.7s => [build 2/8] COPY packages/database ./packages/database
0.2s => [build 3/8] COPY packages/shared ./packages/shared 0.0s => [build 4/8] COPY apps/web
./apps/web 0.0s => [build 5/8] COPY tsconfig.json ./ 0.0s => [build 6/8] RUN pnpm --filter
@teachermon/database db:generate 1.8s => [build 7/8] RUN pnpm --filter @teachermon/shared
build 0.8s => [build 8/8] RUN pnpm --filter @teachermon/web build 22.3s => [production 4/15]
COPY --from=build /app/apps/web/.next ./apps/web/.next 0.2s => [production 5/15] COPY --
from=build /app/apps/web/public ./apps/web/public 0.0s => [production 6/15] COPY --
from=build /app/apps/web/next.config.js ./apps/web/ 0.0s => [production 7/15] COPY --
from=build /app/apps/web/package.json ./apps/web/ 0.0s => [production 8/15] COPY --
from=build /app/packages/shared/dist ./packages/shared/dist 0.0s => [production 9/15] COPY
packages/shared/package.json ./packages/shared/ 0.0s => [production 10/15] COPY package.json
pnpm-workspace.yaml pnpm-lock.yaml ./ 0.0s => [production 11/15] COPY --from=build
/app/packages/database/index.js ./packages/database/index.js 0.0s => [production 12/15] COPY
--from=build /app/packages/database/index.d.ts ./packages/database/index.d.ts 0.0s =>
[production 13/15] COPY packages/database/package.json ./packages/database/ 0.0s =>
[production 14/15] RUN corepack enable && corepack prepare pnpm@9 --activate && pnpm
install --frozen-lockfile --prod 8.7s => [production 15/15] WORKDIR /app/apps/web 0.2s =>
exporting to image 22.1s => => exporting layers 16.4s => => exporting manifest
sha256:761efe8d38f873c85938d23bf4fcf2454b4db295282999758e9ceea4d3defc88 0.0s => =>

exporting config

sha256:16a1cb93cdd9d02ac3eccc23d3e28398105c4e3f843dc35418677c00f397c0aa 0.0s => =>

exporting attestation manifest

sha256:6db891493cc93c21671ecfef5542ddd2e1be2af827067598d7230ce69763ebc6 0.0s => =>

exporting manifest list

sha256:dd950159570dfce0400f28daf6fcfd4210b5149595a8abd1ef0008b0afc9e123 0.0s => =>

naming to docker.io/library/teachermon-web:latest 0.0s => => unpacking to

docker.io/library/teachermon-web:latest 5.6s => resolving provenance for metadata file 0.0s [+]

build 1/2 ✓ Image teachermon-web:latest Built 81.1s • Image teachermon-api:latest Building

81.1s WARN[0000] No services to build [+] up 2/2 ✓ Container teachermon-api Healthy 1.4s ✓

Container teachermon-web Recreated 0.8s root@payaprai-MS-

7E41:/DATA/AppData/www/rukthin# curl http://203.172.184.47:9904/api/health

{"status":"ok","timestamp":"2026-02-

07T00:51:29.316Z","uptime":686.230225136,"responseTime":2,"services":{"database":"connected"}}r

oot@payaprai-MS-7E41:/DATA/AppData/www/rukthin# ^C root@payaprai-MS-

7E41:/DATA/AppData/www/rukthin# ufw status Status: active To Action From -- ----- 22/tcp

ALLOW Anywhere 3000/tcp ALLOW Anywhere 4000/tcp ALLOW Anywhere 5678/tcp ALLOW

Anywhere 11434/tcp ALLOW Anywhere 880/tcp ALLOW Anywhere 8080/tcp ALLOW Anywhere

80/tcp ALLOW Anywhere 8188/tcp ALLOW Anywhere 81/tcp ALLOW Anywhere 7860/tcp ALLOW

Anywhere 8081/tcp ALLOW Anywhere 9900/tcp ALLOW Anywhere 9901/tcp ALLOW Anywhere

9000/tcp ALLOW Anywhere 9903/tcp ALLOW Anywhere 9904/tcp ALLOW Anywhere 22/tcp (v6)

ALLOW Anywhere (v6) 3000/tcp (v6) ALLOW Anywhere (v6) 4000/tcp (v6) ALLOW Anywhere (v6)

5678/tcp (v6) ALLOW Anywhere (v6) 11434/tcp (v6) ALLOW Anywhere (v6) 880/tcp (v6) ALLOW

Anywhere (v6) 8080/tcp (v6) ALLOW Anywhere (v6) 80/tcp (v6) ALLOW Anywhere (v6) 8188/tcp

(v6) ALLOW Anywhere (v6) 81/tcp (v6) ALLOW Anywhere (v6) 7860/tcp (v6) ALLOW Anywhere

(v6) 8081/tcp (v6) ALLOW Anywhere (v6) 9900/tcp (v6) ALLOW Anywhere (v6) 9901/tcp (v6)

ALLOW Anywhere (v6) 9000/tcp (v6) ALLOW Anywhere (v6) 9903/tcp (v6) ALLOW Anywhere (v6)

9904/tcp (v6) ALLOW Anywhere (v6) root@payaprai-MS-7E41:/DATA/AppData/www/rukthin#

docker ps CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES ad461e5107c8

teachermon-web:latest "docker-entrypoint.s..." 4 minutes ago Up 4 minutes (healthy)

0.0.0.0:9903->3000/tcp, [::]:9903->3000/tcp teachermon-web 4a85dd18879c teachermon-api:latest "docker-entrpoint.s..." 12 minutes ago Up 12 minutes (healthy) 0.0.0.0:9904->3001/tcp, [::]:9904->3001/tcp teachermon-api e79db1e1138c plcnextgen-web "/docker-entrpoint...." 12 hours ago Up 14 minutes 0.0.0.0:9902->80/tcp, [::]:9902->80/tcp plcnextgen-web-1 132e7ae49528 nginx:alpine "/docker-entrpoint...." 32 hours ago Up 14 minutes 0.0.0.0:9000->80/tcp, [::]:9000->80/tcp nitesa_nginx dcaf88a7c389 nitesa-app "docker-php-entrpoi..." 32 hours ago Up 14 minutes 9000/tcp nitesa_app 088be1a534b3 redis:alpine "docker-entrpoint.s..." 32 hours ago Up 14 minutes 6379/tcp nitesa_redis 5ae96ca93c52 tablebooking-app "docker-entrpoint.s..." 3 days ago Up 14 minutes 0.0.0.0:9900->3000/tcp, [::]:9900->3000/tcp tablebooking-app-1 8f2d920e228f mariadb:11.4 "docker-entrpoint.s..." 4 days ago Up 14 minutes (healthy) 3306/tcp eqap_db 9064bca3896d doitung-app "docker-entrpoint.s..." 4 days ago Up 14 minutes (unhealthy) 0.0.0.0:9901->9901/tcp, [::]:9901->9901/tcp eqap_app 93071d348e3b linuxserver/mariadb:11.4.8 "/init" 6 days ago Up 14 minutes 0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp mariadb 06e2581f1f36 phpmyadmin/phpmyadmin:5.2.2 "/docker-entrpoint...." 2 weeks ago Up 14 minutes 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp big-bear-phpmyadmin b5740c7b4024 ghcr.io/openwebui/open-webui:main "bash start.sh" 4 weeks ago Up 14 minutes (healthy) 0.0.0.0:4000->8080/tcp, [::]:4000->8080/tcp openwebui 172ab42f7fc8 ollama/ollama:latest "/bin/ollama serve" 4 weeks ago Up 14 minutes (healthy) 0.0.0.0:11434->11434/tcp, [::]:11434->11434/tcp ollama root@payaprai-MS-7E41:/DATA/AppData/www/rukthin# # 1. รัน migration สร้างตาราง docker exec teachermon-api npx prisma@5.22.0 migrate deploy -- schema=packages/database/prisma/schema.prisma # 2. รัน seed สร้าง user ตัวอย่าง (admin@teachermon.com / password123) docker exec teachermon-api npx tsx packages/database/prisma/seed.ts npm warn exec The following package was not found and will be installed: prisma@5.22.0 Prisma schema loaded from packages/database/prisma/schema.prisma Datasource "db": MySQL database "teachermon" at "192.168.1.4:3306" 4 migrations found in prisma/migrations No pending migrations to apply. npm notice npm notice New major version of npm available! 10.8.2 -> 11.9.0 npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.9.0 npm notice To update run: npm install -g npm@11.9.0 npm notice npm warn exec The following package was not found and will be

installed: tsx@4.21.0 🌱 Starting database seeding... ✅ Cleared existing data ✅ Created 5 schools ✅ Created 6 teachers ✅ Created 8 users ✅ Created 2 mentoring visits ✅ Created 2 competency assessments ✅ Created 2 reflective journals ✅ Created 2 PLC activities ✅ Created 1 development plans ✅ Created policy insights ✅ Seeding completed successfully! root@payaprai-MS-7E41:/DATA/AppData/www/rukthin# แก้อย่างไร Thought for 10s จาก log ที่ส่งมา ยังไม่เห็น “error” เลยครับ — ทั้ง build ผ่าน, teachermon-api/teachermon-web เป็น healthy, curl /api/health ก็ status:"ok" ✅ สิ่งที่คุณเห็นแล้วรู้สึก “มีปัญหา” จริง ๆ มี 2 จุดหลัก ๆ:

1) WARN[0000] No services to build แปลว่าอะไร? ต้องแก้ไขไหม

ไม่ต้องแก้ครับ

มันแปลว่าใน docker-compose.yml ตอน docker compose up -d ไม่มี service ไหนต้อง “build” (เพราะใช้ image ที่ build เสร็จแล้ว) — เป็น warning เชิงข้อมูล ไม่ได้ทำให้ระบบพัง

2) npm warn exec ... prisma/tsx จะถูกติดตั้งทุกครั้ง (อันนี้ควรแก้)

อันนี้เกิดเพราะ container teachermon-api ของคุณ “production image” มักจะติดตั้งเฉพาะ dependencies (ไม่เอา devDependencies)

แต่ prisma และ tsx อยู่ฝั่ง dev หรือไม่ได้อยู่ใน image → เวลา npx prisma... / npx tsx... มันเลย “ดาวน์โหลดชั่วคราว” ให้

วิธีแก้ที่สะอาด (แนะนำ)

ทำให้ prisma และ tsx มีอยู่ใน image ของ api ตั้งแต่แรก

ตัวเลือก A (ง่ายสุด): ย้ายแพ็คเกจไปเป็น dependencies

- ใน packages/database/package.json ให้ย้าย
 - prisma
 - tsx

ไปอยู่ “dependencies” (ไม่ใช่ “devDependencies”)

แล้ว rebuild + up ใหม่

docker compose build teachermon-api

docker compose up -d

จากนั้นรันอีกที จะไม่ต้องโหลดเพิ่ม:

docker exec teachermon-api npx prisma migrate deploy --

schema=packages/database/prisma/schema.prisma

docker exec teachermon-api npx tsx packages/database/prisma/seed.ts

ตัวเลือก B (โปรดว่า): แยก “migration job container”

ทำ service ชื่อ teachermon-migrate ใน compose ใช้ image เดียวกับ api แต่มี command รัน migrate/seed แล้ว exit (ไว้ใช้ตอน deploy)

3) เช็คว่า “ใช้งานจริงได้ไหม” (สั้น ๆ)

จาก docker ps ตอนนี้เปิดพอร์ตไว้แล้ว:

- Web: <http://203.172.184.47:9903>
- API: <http://203.172.184.47:9904/api/health>

ลองจากเครื่องนอกวง:

curl -I <http://203.172.184.47:9903>

curl <http://203.172.184.47:9904/api/health>

ขอถามแบบไม่กวน (แต่จะช่วยจบไว)

ตอนที่คุณพิมพ์ว่า “แก้อย่างไร” หมายถึง “อยากแก้” ช้อไหนครับ?

1. warning No services to build
2. npx ดาวน์โหลด prisma/tsx ทุกครั้ง
3. เข้าเว็บ 9903 จากข้างนอกไม่ได้
4. หรืออย่างอื่น (บอกข้อความ error/อาการ)

ถ้าตอบแค่เลขเดียว ผมจะจัดคำส่ง/ไฟล์ให้จบเป็นชุดเดียวเลยครับ 😊